

TECHNISCHE UNIVERSITÄT DRESDEN

CHAIR OF SOFTWARE TECHNOLOGIES

PROF. DR. RER. NAT. UWE ASSMANN

Großer Beleg

LRDMSim with fragmented Network optimization -
Snowflake

Benjamin-Elias Probst

(Mat.-No.: 4510512)

Professor: Prof. Dr. rer. nat. Uwe Aßmann

Tutor: Dr. Sebastian Götz

Dresden, September 17, 2025

Contents

1	Introduction	6
1.1	Background and Task Context	6
1.2	Research Questions and Resulting Work	6
1.3	Objective and Scope	7
1.4	Research Methodology Aligned with the Research Questions	8
1.4.1	Research-Adapted Logical Framework (ZOPP-lite)	8
1.4.2	Methodological Principles	9
1.4.3	System and Topology Model	10
1.4.4	Instrumentation, Metrics, and Invariants	10
1.4.5	Generic Experimental Procedure	10
1.4.6	Data Management and Reproducibility	11
1.4.7	Validity Considerations	11
1.4.8	From Methodology to RQs	11
1.5	Methodology of Research Questions	11
1.5.1	Methodology for RQ1: Failover Behaviour and Cut Sizes on k -Connected Backbones with Periodic Corona Attachments	11
1.5.2	RQ2: Runtime Reconfiguration under WAN-like Constraints	13
1.5.3	RQ3: Metric Design for LRDM Objectives Across Topology Variants and Pattern Fusion	15
1.5.4	RQ4: Bandwidth–latency trade-offs and Pareto-efficient topology settings	18
1.5.5	RQ5: Framework suitability and design patterns for recursive LRDM structures	20
1.5.6	Methodology for RQ6 (Suitability of Java Graph Frameworks)	23
2	Fundamentals and Definitions	26
2.1	Graph-Theoretic Concepts	26
2.2	Network and Performance Concepts	27
2.3	Simulation Semantics	27
2.4	Structural Compositions Used in This Thesis	27
2.5	Metrics and Indicators	28
2.6	Software-Engineering Terms (Adapter-Based Integration)	28
2.7	Assumptions and Boundaries	28
3	Problem Analysis and Requirements	29
3.1	Problem Analysis and Requirements - Plan and Contribute examples using LRDM Simulator	29
3.1.1	Occurring Problems	29
3.1.2	Consequence	29
3.1.3	Solution	30
3.2	Problem Analysis and Requirements - Extension concept and reproducibility	30
3.2.1	Occurring Problems	30
3.2.2	Consequence	33
3.2.3	Solution	33
3.3	Problem Analysis and Requirements - We present: A snowflake pattern	33
3.3.1	Occurring Problems	33
3.3.2	Consequence	34

3.3.3	Solution	34
3.4	Problem Analysis and Requirements - Architecture, Design and Effective project transformation	34
3.4.1	Occurring Problems	34
3.4.2	Consequence	36
3.4.3	Solution	37
4	Concept	40
4.1	Concept - Plan and Contribute examples using LRDM Simulator	40
4.1.1	Network	40
4.1.2	TimedRDMSim	40
4.1.3	DataPackage	40
4.1.4	Mirror	41
4.1.5	MirrorCursor	41
4.1.6	Link	41
4.1.7	Action	41
4.1.8	MirrorChange	41
4.1.9	TargetLinkChange	42
4.1.10	TopologyChange	42
4.1.11	Effector	42
4.1.12	VisualizationStrategy	42
4.1.13	GraphVisualization	42
4.1.14	Probe	43
4.1.15	LinkProbe	43
4.1.16	MirrorProbe	43
4.1.17	TopologyStrategy	43
4.1.18	BalancedTreeTopologyStrategy	44
4.1.19	FullyConnectedTopology	44
4.1.20	NConnectedTopology	44
4.2	Concept - Extension concept and reproducibility	44
4.2.1	StructureNode	45
4.2.2	MirrorNode	45
4.2.3	TreeMirrorNode	45
4.2.4	BalancedTreeMirrorNode	45
4.2.5	DepthLimitTreeMirrorNode	46
4.2.6	FullyConnectedMirrorNode	46
4.2.7	LineMirrorNode	46
4.2.8	NConnectedMirrorNode	46
4.2.9	StarMirrorNode	46
4.2.10	BuildAsSubstructure	46
4.2.11	DepthInsertionStrategy	47
4.2.12	LineTopologyStrategy	47
4.2.13	StarTopologyStrategy	47
4.2.14	SnowflakeTopologyStrategy	47
4.2.15	TreeTopologyStrategy	47
4.2.16	DepthLimitTreeTopologyStrategy	48
4.2.17	SnowflakeTopologyValidator	48
4.2.18	SubstructureFactory	48
4.3	Concept - We present: A snowflake pattern	48
4.4	Concept - Architecture, Design and Effective project transformation	49

5	Evaluation, Results and Conclusion	51
5.1	Findings from the three measurement plots (snowflake run).	51
5.2	Interpretation per research question	51
5.2.1	RQ1: How does a k -connected backbone with periodic corona attachments affect failover behavior and cut sizes under node/link failures?	51
5.2.2	RQ2: Can run-time reconfiguration (adding/removing mirrors/links) maintain service continuity without global restart under WAN-like latencies/capacities? What needs to be changed on the architecture and implementation to support this feature?	51
5.2.3	RQ3: Which metrics best capture LRDM objectives (consistency, convergence, utilization) across topology variants, transformations and fusion of structure patterns?	52
5.2.4	RQ4: Under bandwidth constraints, which composed backbone/attachment settings strike the best trade-off between link usage and time-to-write across the network?	52
5.2.5	RQ5: Which implementation strategy (design patterns, architecture design suggestions, custom core vs. general graph frameworks) supports recursive LRDM structures best? How can we still maintain mirror stability and tick semantics to sample the use of the given framework [Göt23, GBS24]? Which design patterns are most effective to yield results?	52
5.2.6	RQ6: Which Java graph framework (JGraphT, GraphStream, Guava <code>common.graph</code>) or a custom adjacency structure is most suitable to extend <i>LRDMSimulator</i> [Göt23], judged by estimated implementation effort, ID stability and determinism, predictable per-tick mutation costs, fixed-tick hooks/instrumentation, lean dependency footprint, and adapter/visualization options?	52
5.3	Hypotheses mapping and evaluation (snowflake run).	52
5.4	Summary of Results	54
5.5	Lessons Learned	55
5.6	Limitations	55
6	Outlook and Future Directions	58
6.1	Next Steps	58
6.2	Broader Applications: From Information Flow to Multi-Resource Supply Chains	58
6.3	Architectural Refactoring: Making <i>StructureNode</i> the Primary Plan and <i>MirrorNode</i> the Runtime Anchor	59
6.4	Closing Remark	60
	Bibliography	61
A	Figure Appendix	63
B	Preset Schema and Scenario Manifest	85
B.1	Purpose	85
B.2	Top-level Structure (Informal JSON Schema)	85
B.3	Endpoint and Naming Conventions	86
B.4	Minimal Working Example	86
B.5	Indicator Notes	86
C	Reference Scenarios & Batch Runner outlook	88
C.1	Purpose	88
C.2	Runner Design Goals	88
C.3	Command-Line Interface (CLI)	88

C.4	Execution Model	88
C.5	Reference Scenario Catalog	88
C.6	Output Layout	90
C.7	Reproducibility & Recommended Defaults	90
C.8	Batch Runner—Integration Sketch	90
C.9	Indicator Interpretation	91
D	Configuration Listings	92
D.1	Baseline (R1) Configuration Future Example	92
D.2	WAN Bridge (R2) Configuration Future Example	92
E	Future CLI Commands	93
E.1	Build	93
E.2	Run Scenarios R1–R5	93
E.3	Post-processing (example)	93
F	Pseudocode for future Core Routines	94
F.1	Two-Phase Tick (Effects then Measurements)	94
F.2	Substructure-Permutation Attachment (Period- k)	94
	Declaration of Authorship	95

Abstract

Latency-aware Remote Data Mirroring (LRDM) requires network designs that keep replicated data consistent while remaining robust under failures and topology evolution. We present LRDMSimulator+, a discrete-time, graph-based simulator for mirror networks that supports live topology adaptation and domain-level composition. At its core, a recursive structure building template, proved with a N-connected ring as the main structure, provides predictable path redundancy and bounded cut sizes; on top, a substructure-permutation factory attaches domain subgraphs at periodic offsets to the core (modulo mirror IDs), enabling controlled heterogeneity (e.g., edge domains, WAN-connected sites). Unlike previous prototypes that required full restarts for any change, LRDMSimulator+ models mirror state machines, link capacities, and timed effectors to add/remove mirrors and links at run-time. Domains can be bridged via high-latency, capacity-limited links to emulate Internet-scale interconnects, allowing planners to explore trade-offs between bandwidth constraints, resilience, and update convergence. We evaluate design alternatives by tracking mirror-state transitions, link utilization, and end-to-end propagation delay, and we report system-level outcomes (e.g., recovery and convergence times, cut sizes) rather than microbenchmarks. We also discuss why we avoided generic graph frameworks in the core (to preserve mirror stability, timing semantics, and lean dependencies) while still integrating visualization libraries. Experiments show that N-connected backbone combined with periodic substructure attachment yields predictable failover behavior and scalable update dissemination, and that runtime reconfiguration achieves service continuity without global restart.

1 Introduction

1.1 Background and Task Context

Latency-aware Remote Data Mirroring (LRDM) enables researchers and users to determine an optimum mirror distribution by simulating connection-, stability and scalability behavior of self aware mirror networks with the LRDM Simulation framework [Göt23, GBS24]. The concept requires backbones with guaranteed vertex connectivity and predictable failover paths. The current state of art only allows us to simulate single network domain structures with a complete restart of each simulation. Our goal is to extend the framework to converge to a real setup of internet WAN domains and sub networks. We therefore adopt k -connected graphs (vertex connectivity $\kappa(G) \geq k$) as the backbone model to mock the general concept of the framework extension.

This includes keeping the concept of failure tolerance on multi network domains. By Menger’s theorem [Men27], any two mirrors are joined by at least k internally vertex-disjoint paths, implying tolerance to up to $k-1$ node failures without disconnecting the backbone. We do *not* assume edge minimality. To attach heterogeneous domains in a structured way, we use the *corona product* to decorate the backbone with periodic subgraphs (“snowflake” attachments), which keeps degrees and distances tractable while supporting WAN/edge domains [FH70]. Building on the original simulator by Götz et al. [Göt23, GBS24], we extend the core to support on-the-fly creation and removal of mirrors and links under fixed-tick timing and share experience made by planning, documenting and implementing on top of the given framework baseline.

Before we take a look on the implementation of the extension, we require to analyze several research questions leading to a knowledge complement. From this point we will then define requirements specifications *RS*, an object oriented analysis *OOA*, we suggest changes to the graphical user interface *GUI* and finally document an object oriented design to be implemented on top of the existing framework.

1.2 Research Questions and Resulting Work

RQ1: How does a k -connected backbone with periodic corona attachments affect failover behavior and cut sizes under node/link failures?

Resulting work: Implementation and formal analysis of connectivity properties on composed graphs. Experimental study measuring cut sizes and post-failure path availability across k , attachment periodicity, and domain sizes.

RQ2: Can run-time reconfiguration (adding/removing mirrors/links) maintain service continuity without global restart under WAN-like latencies/capacities? What needs to be changed on the architecture and implementation to support this feature?

Resulting work: Conceptualization, Planning and implementation of mirror appending, timed effectors in *LRDMSim+*. Evaluation of recovery/convergence times and delivery ratios during reconfiguration scenarios.

RQ3: Which metrics best capture LRDM objectives (consistency, convergence, utilization) across topology variants, transformations and fusion of structure patterns?

Resulting work: Comparison of the given metric set of the given framework [Göt23, GBS24] using the snowflake pattern in combination with different substructures to quantify metrics behavior of mirror-state transitions, end-to-end propagation delay, link utilization, delivery ratio on compound structures. Offer example reproducible pipelines (configs, seeds, log-plots) to compute and compare.

RQ4: Under bandwidth constraints, which composed backbone/attachment settings strike the best trade-

off between link usage and time-to-write across the network?

Resulting work: Parameter sweep (tick size, k , attachment periodicity, WAN latency/capacity) with multi-objective reporting (utilization vs. propagation/convergence delay); identification of Pareto-efficient regimes.

RQ5: Which implementation strategy (design patterns, architecture design suggestions, custom core vs. general graph frameworks) supports recursive LRDM structures best? How can we still maintain mirror stability and tick semantics to sample the use of the given framework [Göt23, GBS24]? Which design patterns are most effective to yield results?

Resulting work: Comparative engineering study (prototype adapters vs. bespoke core). Documentation of trade-offs (determinism, dependency footprint, performance), and pattern catalogue (e.g., builder/factory for topology composition, command/effectors for reconfiguration, strategy for initiation of single topology). Suggestions to architecture refactoring and improved maintainability of the implementation by using the current node domain extension as an example.

RQ6 (Suitability): Which Java graph framework (JGraphT, GraphStream, Guava `common.graph`) or a custom adjacency structure is most suitable to extend *LRDMSimulator* [Göt23], judged by estimated implementation effort, ID stability and determinism, predictable per-tick mutation costs, fixed-tick hooks/instrumentation, lean dependency footprint, and adapter/visualization options?

Resulting work: Qualitative requirements-to-capabilities mapping. Lightweight adapter prototypes. Discussion for the selected core (framework or custom) and positioning property-graph stacks (e.g., TinkerPop/Neo4j) as *adapters* rather than as the simulation core.

1.3 Objective and Scope

This thesis presents *LRDMSim+*, a discrete-time, graph-based simulator that (i) supports live topology adaptation (add/remove mirrors and links at run-time), (ii) composes domain subgraphs at periodic off-sets over a k -connected backbone via corona-style attachments, and (iii) preserves mirror stability and fixed-tick timing needed for convergence analysis. The focus is enabling functionality and structural exploration, detailed indicator design is addressed insofar as it serves RQ1–RQ4.

Framework scope and suitability. Instead of runtime benchmarks, we conduct an exploration and qualitative suitability analysis for extending the mother project *LRDMSimulator* [Göt23]. We assess whether and how well candidates support (i) stable mirror interconnections and deterministic iteration orders, (ii) mutability with predictable per-tick costs (add/remove nodes/edges), (iii) hooks for fixed-tick timing and instrumentation, (iv) a low dependency/footprint overhead, and (v) adapter and visualization options. Concretely, we consider JGraphT, GraphStream, Google’s Guava `common.graph` and a slim project-specific adjacency structure. The property-graph stacks (e.g., TinkerPop/Neo4j) are positioned as persistence/analysis *adapters* in memory, not as the core simulation engine.

- **Determinism & mirror stability:** Dynamic iterative node IDs, reproducible traversal orders.
- **Mutability per tick:** Predictable costs for adding/removing nodes/edges, listener hooks.
- **Timing/Instrumentation:** Integration into the fixed-tick loop, events/probes for metrics.
- **Dependencies/Footprint:** Low transitive dependencies, simple build integration.
- **Adapters & Visualization:** Reuse of existing plot pipelines and positioning of the graph role on the simulation framework

Objectives. O1: Formalize and use a k -connected backbone ($\kappa(G) \geq k$) without assuming edge minimality from RQ1. Show examples of other base-topologies. O2: Plan and implement *LRDMSim+* with timed effectors and ID-stable, no-restart reconfiguration with the object oriented design from RQ5

and findings from RQ6. O3: Design, implement and compose heterogeneous domains via periodic corona attachments from RQ4 with the planning of RQ5. O4: Evaluate under WAN-like constraints using the metric set from RQ3, the LRDM extension from RQ2 and project design from RQ6.

Approach In *LRDMSim+*, time advances in fixed ticks; mirrors follow state machines; links enforce capacities and latencies. Topology variants are supplied as pre-run configurations (properties and Java examples) for reproducibility; run-time effectors perform controlled reconfiguration. The k -connected backbone plus corona attachments provides a tractable basis to reason about connectivity, path diversity, and bounded cut sizes [FH70]. The framework comparison is executed as a qualitative suitability analysis (requirements-to-capabilities mapping) rather than as a performance benchmark, and feeds into the architectural decision for the simulation core (cf. RQ5–RQ6). The architecture of the simulation core needs to be adjusted and split to keep the complexity of each implementation aspect low and to offer room for the domain extensions using a custom approach.

Contributions. C1: A set of LRDMSim simulator examples to show separate single domain simulations. C2: A reproducibility bundle of architecture design (java project extension, configs, plots) enabling and showing parameter sweeps. Plans to transform the LRDM simulator structure. C3: A structured domain-composition method (periodic corona attachments) on k -connected backbones. An extension tailored to LRDM semantics with run-time reconfiguration with an example compound simulation. C4: A qualitative suitability analysis and reasoned selection of core data structure/framework (with adapter positioning of property-graph stacks) aligned with LRDM needs [Göt23, GBS24].

1.4 Research Methodology Aligned with the Research Questions

This section consolidates our research-oriented methodology. We adopt a *research-adapted logical framework* (ZOPP-lite) to make explicit how assumptions and inputs drive activities, produce outputs, and translate into measurable outcomes that answer the research questions (RQ1–RQ6). Detailed per-RQ procedures remain in Section 1.5.

1.4.1 Research-Adapted Logical Framework (ZOPP-lite)

Purpose: Set up a reproducible reasoning chain for scientific studies and the experimental setup.

Layer	Definition (research context)	Examples in this work
Assumptions & Constraints	Fixed ticked simulation; deterministic scheduling; bounded resources; graph is the core abstraction; mutations occur at tick boundaries.	Discrete time Δt ; single-threaded tick loop; random mirror crash generation; bounded graph sizes.
Inputs	What we configure in config file and example java setup before a run.	Topology class (ring, k -connected, star, B^+ -tree, line), sizes, link latency/capacity, failure/churn schedule, effector scripts, framework choice (custom vs. JGraphT/GraphStream/Guava).
Activities	What the system does under those inputs.	Initialize graph; run T ticks; apply add/remove node/edge; measure metrics; assert invariants; export traces.
Outputs (Artifacts)	Immediate, inspectable byproducts.	Command line and graphical metrics per tick; logs; topology snapshots; adapter prototypes.
Outcomes (Findings)	Aggregated, validated results answering RQs.	Convergence times, mutation costs per tick, ID stability evidence, trade-offs bandwidth/latency, suitability assessment of frameworks.
Impact (Scope)	What we <i>do not</i> claim in this paper.	Deployment guidance for production networks; economic impact; long-term organizational change.

1.4.2 Methodological Principles

- **Reproducibility & Determinism:** Fixed seeds, deterministic tick order, explicit tie-break rules; all runs described by a machine and human-readable java example preset.
- **Isolation of Effects:** Mutations (add/remove nodes/edges) occur atomically at tick and planned mirror boundaries; metrics sampled after state settles for a tick.
- **Measurement Validity:** Definitions for reconfiguration latency (ticks until structural invariants are restored) and convergence latency (ticks until steady metrics can be configured by setting up an optimizer as a java example).
- **Fair Comparisons:** Same workloads and preset families across implementations (custom vs. frameworks); report any adapter overheads on similar examples.
- **Lean Footprint:** Minimal dependencies in the simulation core by keeping strategy design pattern; property-graph stacks (e.g., TinkerPop/Neo4j) are positioned as *adapters*, not core.
- **Transparency:** Publish presets, seeds, and plotting scripts; keep raw per-tick exports alongside derived figures.

1.4.3 System and Topology Model

Graph abstraction. A simulation state at tick t is a directed or undirected graph $G_t = (V_t, E_t)$ with attributes on V (roles, IDs) and E (latency, capacity, reliability). Node roles (e.g., “mirror”) are *labels*, not classes.

Dynamics. Mutations are effectors scheduled at known ticks: `addNode`, `removeNode`, `addEdge`, `removeEdge`, `setAttr`. Effectors and Probes are atomic and linearized at the tick boundary.

Topologies. Base families include: rings and k -connected rings; stars; lines; B^+ -trees with depth/branching limits; and compound graphs (backbone + periodic attachments). Connectivity $\kappa(G)$, degree distributions, and cut properties are tracked as structural invariants.

Timing. The engine advances in fixed steps Δt . Within a tick, events are processed in a deterministic order; metrics are recorded after all scheduled mutations for that tick.

Randomness. All randomized choices (placement, failures, tie-breaks) draw from repeatable graph construction, but spontaneous mirror crashes and are therefore not stable and repeatable, as far as a mirror crash occurs.

1.4.4 Instrumentation, Metrics, and Invariants

- **Operational metrics:** time-to-write / completion ticks, link utilization, active-links ratio.
- **Manual adaptation metrics:** reconfiguration latency (ticks to restore required connectivity), convergence latency (ticks to steady state).
- **Structural checks:** required $\kappa(G)$, degree bounds, component count $|\mathcal{C}(G)| = 1$ if connectedness is required.
- **Cost of change:** per-tick mutation cost (amortized operations), ID stability under mutations, determinism of traversal/iteration.

All metrics are exported per tick (command line) with run metadata (mirror count, link count, main metrics).

1.4.5 Generic Experimental Procedure

1. **Preset & mirror crash probability.** Select a scenario preset and mirror crash probability; fix Δt .
2. **Initialize.** Construct G_0 and register example workloads/graphs; register invariant probes and metrics.
3. **Execute.** Run for T ticks; at scheduled ticks apply effectors (add/remove) atomically; sample metrics after each tick.
4. **Validate.** Assert invariants; tag violations; store snapshots around mutation windows for command line logs and graphical feedback.
5. **Analyze.** Aggregate to medians/IQRs and confidence intervals; derive trade-offs (e.g., bandwidth vs. latency).

1.4.6 Data Management and Reproducibility

Each run yields: (i) manifest (commit, preset name, seed), (ii) per-tick logs, (iii) plots. Figures in the paper are regenerated from these artifacts; presets and seeds are versioned.

1.4.7 Validity Considerations

Internal: deterministic scheduling; atomic mutations; single source of truth for evolving IDs. *Construct:* minimal, explicit definitions for “reconfiguration” and “convergence”; metrics reflect hypotheses. *External:* parameter ranges (latency/capacity/failure rates) reported with validity domains; framework adapter overheads are disclosed.

1.4.8 From Methodology to RQs

This section provides the common backbone. Section 1.5 specifies, for each RQ, the concrete factor sets, presets, effectors, and analysis windows. For framework suitability (RQ6), we map requirements (ID stability, determinism, mutation costs, hooks/instrumentation, dependency footprint, visualization) to observed capabilities using lightweight adapters rather than using the framework as the simulation core.

1.5 Methodology of Research Questions

1.5.1 Methodology for RQ1: Failover Behaviour and Cut Sizes on k -Connected Backbones with Periodic Corona Attachments

Research question. How does a k -connected backbone with periodic corona-style attachments affect failover behaviour and cut sizes under node/link failures?

Context. We build on the original simulator by Götz et al. [Göt23, GBS24]. The backbone is k -connected (vertex connectivity $\kappa(G) \geq k$), implying at least k internally vertex-disjoint paths between any two backbone mirrors (Menger’s theorem [Men27]). Heterogeneous domains are attached periodically via a corona-like scheme [FH70]. Attachments are realised with `BuildAsSubstructure` (single attachment port $a=1$) in a `SnowflakeTopologyStrategy` over an `NConnectedTopology`. Failures delete mirrors and their incident links; optional link resets are configurable.

Hypotheses.

- H1** Increasing k reduces the probability that random vertex removals disconnect the backbone and lowers empirical cut sizes.
- H2** With single-port attachments ($a=1$), the *global* graph remains vulnerable at attachment mirrors even if the backbone subgraph maintains $\kappa(G) \geq k$; isolation risk drops only with multi-port attachments (out of scope to implement, analysed conceptually).
- H3** Denser attachment (smaller period p) shortens post-failure paths and improves reachability, at the cost of higher link load.
- H4** Under fixed-tick timing with bandwidth accumulation in `Link`, structural robustness correlates with dynamic behaviour (active links, utilisation, time-to-write).

Design factors (independent variables).

- Backbone connectivity $k \in \{2, 3, 4\}$ (num_links_per_mirror).
- Backbone size $m \in \{32, 64, 128, 256, 512\}$ mirrors (supported range 5–512).
- Attachment periodicity $p \in \{1, 2, 4, 8\}$ (attach a domain every p -th backbone mirror).
- Domain template: ring size $s \in \{3, 5, 7\}$ ($s \geq 3$).
- Attachment ports per domain: $a=1$ (current implementation).
- WAN/Link settings: latency/capacity from properties (static per run).

Failure models (disturbances).

1. **F1 Random burst:** remove $r \in \{1, \dots, k\}$ mirrors uniformly at tick t_f .
2. **F2 Random drift:** apply fault_probability per tick over $[t_f, t_f + \Delta]$.
3. **F3 Targeted degree:** remove the r highest-degree mirrors at t_f (adversarial proxy).

Response variables (dependent metrics). *Structural (per tick):* reachability fraction ρ ; largest connected component (LCC) size; average shortest-path length $\bar{d}$ in LCC; articulation flags; empirical cut-size estimator $\hat{\kappa}$ via sampled removals on saved snapshots; domain isolation rate δ (fraction of domains disconnected after attachment-node failure).

Dynamic (per tick): active-links ratio; bandwidth utilisation (accumulating in Link); time to write (TTW) to reachable mirrors; recovery time τ_{settle} (ticks from t_f until metrics stabilise without further reconfiguration).

Instrumentation and logging.

1. **ConnectivityProbe** (runtime on Network): ρ , LCC, $\bar{d}$; articulation detection (Tarjan for $k \leq 2$; degree bounds otherwise).
2. **CutSweepProbe** (offline on snapshots at $t \in \{t_f - 1, t_f, t_f + \Delta\}$): sample S subsets for $r=1, \dots, k+1$ (e.g. $S=100$) to estimate $\hat{\kappa}$ and $P(\text{disconnect} \mid r)$.
3. **DomainIsolationProbe:** track attachments from BuildAsSubstructure and compute δ at/after t_f .

All probes are just conceptional and manually evaluated; built-in charts (active links, utilisation, TTW) are exportable as PNG.

Procedure.

1. **Topology init:** build NConnectedTopology with size m and connectivity k ; attach ring domains of size s every p -th backbone mirror using SnowflakeTopologyStrategy + BuildAsSubstructure ($a=1$).
2. **Warmup:** simulate to steady state (or fixed warmup), e.g.:

```
debug=true, sim_time=500, num_mirrors=m, num_links_per_mirror=k,
startup_time_min=5..10, ready_time_min=2..20, stop_time_min=2..5,
link_activation_time_min=5..10, fileSize=80, min_bandwidth=2..8,
max_bandwidth=8, fault_probability as configured
```
3. **Failure injection:** apply F1/F2/F3 at t_f ; collect metrics for Δ ticks (e.g. $\Delta=100$).
4. **Replication:** run 10 PRNG seeds per configuration; record seeds for reproducibility.

Analysis plan.

- Compare ρ , LCC, $\bar{d}$, TTW before/after t_f across (k, p, s, m) and F1–F3; report mean $\pm$ 95% CI across seeds.
- Fit logistic curves to $P(\text{disconnect} \mid r)$ to compare effective robustness across k and p ; summarise $\hat{\kappa}$.
- Separate *backbone* vs. *global* resilience (domains included) to expose attachment-articulation effects (H2).
- Trade-off visualisations: heatmaps of ρ over (k, p) at fixed (m, s) ; Pareto plots of TTW vs. ρ .

Evaluation criteria. Evidence for H1/H3: monotone improvement of ρ and lower disconnect probability with larger k and smaller p (subject to load). Evidence for H2: under $a=1$, non-zero δ and global $\hat{\kappa} \approx 1$ despite backbone $\kappa(G) \geq k$.

Threats to validity. Sampling-based $\hat{\kappa}$ may miss worst-case cuts for $k > 2$; targeted failures use degree, not centrality; link speeds are static during a run.

1.5.2 RQ2: Runtime Reconfiguration under WAN-like Constraints

Research question. RQ2: Can run-time reconfiguration (adding/removing mirrors/links) maintain service continuity without global restart under WAN-like latencies and capacities? What architectural and implementation changes are required to support this feature?

Hypotheses.

- **H5 (Continuity under additions).** Staggered additions of mirrors/links via timed effectors preserve a high delivery ratio $\text{DR}(w) \geq 0.99$ and bounded time-to-write (TTW), if the backbone remains k -connected and link capacities stay within configured limits.
- **H6 (Graceful removals).** Removing a single mirror (and its incident links) does not cause global stalls if $\kappa(G) \geq 2$ and removals are serialized (one effector per ≥ 1 tick); transient impact is bounded by detour length and link activation times.
- **H7 (WAN bridges).** Inserting high-latency, capacity-limited inter-domain bridges inflates local TTW but does not trigger global restart; steady-state $\text{DR}(w)$ remains ≥ 0.98 with periodic attachments that avoid hotspotting.

Operational definition (service continuity). We say *service continuity* holds during a reconfiguration scenario if:

1. No global restart (no system-wide reinitialization; mirrors do not simultaneously return to STARTUP).
2. Rolling-window delivery ratio $\text{DR}(w)$ never drops below τ_{DR} (default 0.98) for more than s consecutive ticks (default $s = 5$).
3. Per-window TTW(w) stays within $\leq 2 \times$ baseline median.

Design and scenarios. We instrument *LRDMSim+* with timed *effectors* that apply topology changes at tick boundaries (compute $\rightarrow$ apply). Baseline is a k -connected ring (NConnectedTopology), decorated by periodic domain attachments (SnowflakeTopologyStrategy) via BuildAsSubstructure. Unless noted, crashes are disabled to isolate reconfiguration effects.

S1: Staggered add. From a baseline ring ($k \in \{2, 3\}$, $|V| \in \{32, 64\}$), attach one snowflake subgraph every a -th backbone mirror ($a \in \{1, 4\}$) using an ADD_MIRROR+ADD_LINKS effector once per tick for m ticks ($m \in \{8, 16\}$). Link activation obeys configured `link_activation_time_*`.

S2: Serialized removal. Remove a single backbone mirror at tick t_0 (REMOVE_MIRROR); verify continuity for $\kappa(G) \geq 2$ vs. $\kappa(G) = 1$.

S3: WAN bridge insertion. At t_0 , add one/two inter-domain bridges with $(L_{\text{WAN}}, C_{\text{WAN}})$; $L_{\text{WAN}} \in \{40, 80, 160\}$ ms, $C_{\text{WAN}} \in \{2, 4, 8\}$ (simulation units). Periodicity $a \in \{1, 4\}$.

S4: Mixed sequence. ADD for m ticks, then one REMOVE, then WAN_BRIDGE—each separated by at least one tick.

Controls and ranges.

- Nodes: $|V| \in [5, 512]$ (report 32, 64, 128; 512 for stress).
- Connectivity: $k \in \{1, 2, 3\}$ ($k=1$ as negative control).
- Periodicity: $a \in \{1, 2, 4, 8\}$ (domain ring size ≥ 3).
- Timing: fixed tick $\Delta t = 250$ ms; use `startup/ready/stop/link_activation_time_*` from properties.
- Bandwidth: `min_bandwidth, max_bandwidth` in $[2, 8]$; static during runs.
- Faults: `fault_probability=0` unless stated.

Instrumentation and metrics. We log per tick: (i) delivery ratio $\text{DR}(w)$ per rolling window w , (ii) TTW (p50/p95/max), (iii) bandwidth utilisation, (iv) active links and path availability. Effector events are marked in CSV and charts. A no-restart indicator flags any global transition to STARTUP.

Procedural steps (per scenario).

1. Seed and initialize baseline; record baseline metrics for T_{warmup} ticks.
2. Schedule effectors at designated ticks; for additions: max one addition per tick; for removals: enforce ≥ 1 idle tick between operations.
3. Run for `sim_time` ticks; export CSV (`metrics.csv`, event log) and optional plots.
4. Post-process: compute $\text{DR}(w)$, $\text{TTW}(w)$, stall durations; compare across $k, a, (L_{\text{WAN}}, C_{\text{WAN}})$.

Analysis. Test H5–H7 by verifying (i) the no-restart indicator remains false, (ii) $\text{DR}(w)$ stays above thresholds (except brief transients), (iii) TTW inflation versus baseline remains bounded. Summarize across seeds (median/IQR) and report sensitivity to k, a , and WAN parameters. For S2, contrast $\kappa(G) = 1$ vs. $\kappa(G) \geq 2$ to show redundancy necessity.

Threats to validity. Tick-boundary aliasing; capacity ramp-up warm-up bias; chart overhead for large sweeps; determinism enforced via fixed PRNG seeds and ID-stable builders.

Evaluation Criteria.

- **H5:** In S1, $DR(w) \geq 0.99$ and median TTW $\leq 1.5 \times$ baseline during additions; no-restart true.
- **H6:** In S2 with $\kappa(G) \geq 2$, no contiguous stall $> s$ ticks and $DR(w) \geq 0.98$; with $\kappa(G) = 1$ at least one window below threshold.
- **H7:** In S3/S4, WAN bridges increase TTW locally but global $DR(w) \geq 0.98$ and no restart; higher periodicity ($a \geq 2$) reduces hotspotting vs. $a = 1$.

Reproducible configuration (example).

```

debug=true
sim_time=500
num_mirrors=64
num_links_per_mirror=2
startup_time_min=5
startup_time_max=10
ready_time_min=2
ready_time_max=20
stop_time_min=2
stop_time_max=5
link_activation_time_min=5
link_activation_time_max=10
fileSize=80
min_bandwidth=2
max_bandwidth=8
fault_probability=0.000
# effectors/ticks are scheduled in the Snowflake example harness

```

1.5.3 RQ3: Metric Design for LRDM Objectives Across Topology Variants and Pattern Fusion

Research question. **RQ3:** Which metrics best capture LRDM objectives (consistency, convergence, utilization) across topology variants, topology transformations (attach/detach, rewiring), and fusion of structure patterns (Snowflake with mixed substructures)?

Hypotheses.

- **H8 (Sufficiency).** A compact core set—Delivery Ratio DR, Time-To-Write TTW, Convergence Latency CL, Bandwidth Utilization U , End-to-End Propagation Delay PD, and Mirror-State Churn MSC—is sufficient to distinguish regimes induced by topology variants, transformations, and pattern fusion.
- **H9 (Sensitivity to structure/fusion).** DR, TTW, CL, and PD respond predictably to (i) higher backbone connectivity k , (ii) higher attachment periodicity $a > 1$, and (iii) mixing substructures (ring/star/line/ B^+ -like) in Snowflake composites; WAN bridges increase local TTW/PD without system-wide collapse.
- **H10 (Non-redundancy).** $\{DR, TTW, CL, U, PD, MSC\}$ provide complementary information (pairwise $|\rho| < 0.8$); no single metric can be robustly inferred from the others.
- **H11 (Structural linkage).** Offline structural proxies (e.g., estimated vertex connectivity $\hat{\kappa}$, path stretch) explain a significant share of the variance in DR, TTW, and PD across k , a , and fusion patterns.

Candidate metrics (operational definitions). Ticks have fixed duration Δt (e.g., 250 ms).

- **Delivery Ratio** over a rolling window w :

$$\text{DR}(w, t) = \frac{\text{msgs_delivered}(t-w+1:t)}{\text{msgs_attempted}(t-w+1:t)} \in [0, 1].$$

- **Time-To-Write (TTW):** ticks from write initiation until all reachable mirrors commit (report p50/p95/max).
- **Convergence Latency (CL):** ticks from a topology change/failure spike to the first tick with no backlog and stable metrics.
- **Bandwidth Utilization** per link e : $U_e(t) = \text{throughput}_e(t)/C_e$; report per-link median and the 95th-percentile tail.
- **End-to-End Propagation Delay (PD):** difference (in ticks) between the send time of an update at the source and arrival/commit at destination mirrors (p50/p95).
- **Mirror-State Churn (MSC):** number of state transitions per mirror and window (or mean dwell time per state); proxy for stability/consistency.

Offline structural proxies from periodic graph snapshots (every s ticks):

- **Estimated vertex connectivity** $\hat{\kappa}$ via sampling source–target pairs and computing the maximum number of internally vertex-disjoint paths using unit-capacity flow with vertex splitting (report median across samples).
- **Path stretch:** ratio of current shortest-path length to the baseline (pre-disturbance) length for sampled pairs.

Design and coverage. Backbone generated by `NConnectedTopology`; attachments via `SnowflakeTopology` and `BuildAsSubstructure`. We vary:

- **Topology variants & fusion:** pure rings/stars/lines/ B^+ -like subtrees; mixed Snowflake composites (e.g., alternating ring/star domains).
- **Size:** $|V| \in \{32, 64, 128\}$ (stress: 512).
- **Backbone connectivity:** $k \in \{1, 2, 3\}$.
- **Attachment periodicity:** $a \in \{1, 2, 4, 8\}$; ring domains with ≥ 3 mirrors.
- **WAN bridges:** $(L_{\text{WAN}}, C_{\text{WAN}}) \in \{(40, 8), (80, 4), (160, 2)\}$.
- **Transformations:** none (baseline), staggered adds, serialized removes, mixed sequences (cf. RQ2).

Instrumentation. At each tick t , `collectMetrics(t)` logs deliveries, link throughputs/capacities, mirror states (for MSC), active links, and timestamps of effector events; charts (`activeLinksChart`, `bandwidthChart`, `timeToWriteChart`) mark events. We export `metrics.csv` and periodic graph snapshots (node/edge lists) for $\hat{\kappa}$ and path-stretch computation.

Procedural steps.

1. **Warm-up:** Run the baseline for T_{warmup} ticks; exclude from analysis to stabilize capacity ramp-up in Link.
2. **Measure:** For each configuration and fusion variant, run for sim_time ticks; compute $\text{DR}(w)$, TTW, CL, U , PD, MSC online; $\hat{\kappa}$ and path stretch offline.
3. **Compare:** Aggregate by median/IQR across seeds; analyze sensitivity to k , a , WAN parameters, and fusion patterns; use nonparametric tests and effect sizes (e.g., Cliff's δ).
4. **Redundancy check:** Pairwise Spearman ρ and mutual information among $\{\text{DR}, \text{TTW}, \text{CL}, U, \text{PD}, \text{MSC}\}$; PCA to check dimensionality.
5. **Structure linkage:** Regress outcomes on $\hat{\kappa}$ and path stretch; report explained variance and monotonic trends.

Analysis and reporting. **H8** is supported if the core set separates regimes (topology, transformation, fusion) with clear effect sizes. **H9** holds if median DR increases and TTW/CL/PD decrease with k and $a > 1$ (while WAN bridges inflate TTW/PD locally but do not cause global instability). **H10** holds if $|\rho| < 0.8$ (and MI below threshold) across scenarios. **H11** holds if higher $\hat{\kappa}$ predicts higher DR and lower TTW/PD across snapshots.

Threats to validity. Tick-boundary aliasing; transients due to capacity ramp-up; chart overhead for large sweeps; sampling error in $\hat{\kappa}$; absence of explicit queue-length metrics (left for future work).

Evaluation Criteria.

- **H8:** $\{\text{DR}, \text{TTW}, \text{CL}, U, \text{PD}, \text{MSC}\}$ separates configurations/fusions with at least medium effects ($|\delta| \geq 0.33$).
- **H9:** median(DR) is non-decreasing and median(TTW), median(CL), median(PD) are non-increasing with k and $a > 1$; WAN bridges raise TTW/PD but keep $\text{DR} \geq 0.98$.
- **H10:** $\max_{X \neq Y} |\rho(X, Y)| < 0.8$ for $X, Y \in \{\text{DR}, \text{TTW}, \text{CL}, U, \text{PD}, \text{MSC}\}$; the first two PCA components explain $< 90\%$ (indicating multiple necessary metrics).
- **H11:** $\hat{\kappa}$ correlates positively with DR ($\rho > 0.5$) and negatively with TTW/PD ($\rho < -0.5$).

Reproducible pipeline (example).

```
# Config (excerpt)
debug=true
sim_time=600
num_mirrors=64
num_links_per_mirror=2
startup_time_min=5; startup_time_max=10
ready_time_min=2; ready_time_max=20
stop_time_min=2; stop_time_max=5
link_activation_time_min=5; link_activation_time_max=10
fileSize=80
min_bandwidth=2; max_bandwidth=8
fault_probability=0.000
# Topology: NConnectedTopology + SnowflakeTopologyStrategy (BuildAsSubstructure)
# Fusion: alternating Ring/Star domains; snapshots every s=10 ticks.
# Seeds: 10 fixed PRNG seeds; export metrics.csv + plots + node/edge snapshots.
```

1.5.4 RQ4: Bandwidth–latency trade-offs and Pareto-efficient topology settings

Research question. RQ4: Under bandwidth constraints, which composed backbone/attachment settings strike the best trade-off between link usage and time-to-write across the network?

Hypotheses.

- **H12 (Trade-off surface).** For fixed traffic and tick size, composed topologies parameterized by (k, a) yield a non-trivial Pareto front between *link usage* and *Time-To-Write (TTW)*.
- **H13 (Connectivity leverage).** Increasing k improves TTW and delivery ratio under the same capacity budget up to a saturation point; beyond that, extra edges do not materially reduce TTW.
- **H14 (Attachment pacing).** Higher attachment periodicity $a > 1$ reduces backbone load and provisioned capacity while keeping TTW comparable to $a=1$ away from WAN bottlenecks.
- **H15 (WAN sensitivity).** Higher WAN latency / lower WAN capacity locally inflates TTW but does not eliminate Pareto-efficient settings if (k, a) are tuned.

Objective functions and constraints. We evaluate multi-objective efficiency using:

- **Time-To-Write TTW:** median/p95 ticks from write initiation to global commit (cf. Sec. 1.5.3).
- **Link usage (budget)** $C_\Sigma = \sum_{e \in E} C_e$ (sum of configured per-link capacities; steady-state samples account for capacity ramp-up in Link).
- **Utilization** summaries $\tilde{U}$ (median and 95th percentile across links) and **active-links ratio** (fraction of links with non-zero throughput) to diagnose waste vs. saturation.

Two complementary optimization views:

1. *Epsilon-constraint:* minimize TTW subject to $C_\Sigma \leq B$, sweeping budget levels B .
2. *Pareto enumeration:* collect (C_Σ, TTW) (plus $\tilde{U}$) for each configuration and extract the non-dominated set; report knee points.

Design and coverage. We synthesize backbones with `NConnectedTopology` and attach subgraphs via `SnowflakeTopologyStrategy` using `BuildAsSubstructure` (single-attachment by default). Failures are modeled as mirror removals (with incident edge deletions) to reflect simulator semantics.

- **Scale:** $|V| \in \{32, 64, 128, 256\}$ (stress: 512 with charts disabled).
- **Connectivity:** $k \in \{1, 2, 3\}$.
- **Attachment periodicity:** $a \in \{1, 2, 4, 8\}$; ring domains require ≥ 3 mirrors (other domains ≥ 1).
- **WAN bridges:** $(L_{\text{WAN}}, C_{\text{WAN}}) \in \{(40, 8), (80, 4), (160, 2)\}$.
- **Capacity budgets:** induce C_Σ strata via `min_bandwidth/max_bandwidth`, e.g., $\{(1, 4), (2, 8), (4, 16)\}$; tick size fixed per run, optionally varied across sweeps.
- **Effectors:** none (steady state) for core fronts, then scripted sequences (cf. RQ2) to confirm robustness under reconfiguration.

Instrumentation. Per tick, `collectMetrics(t)` logs deliveries, link throughputs/capacities, mirror states, and active links; effector events are timestamped and chart-marked. For each run we export `metrics.csv` and compute $(C_\Sigma, \text{TTW}, \tilde{U}, \text{active-links ratio})$. Graph snapshots are optional here (no $\hat{\kappa}$ required for the core trade-off).

Procedural steps.

1. **Warm-up and steady state.** Execute T_{warmup} ticks (excluded) to stabilize capacity ramp-up in Link; verify stable $\tilde{U}$.
2. **Budget sweep.** For each budget stratum, set capacities via `min_bandwidth/max_bandwidth` and run all (k, a) combinations across seeds.
3. **WAN sensitivity.** Repeat for each $(L_{\text{WAN}}, C_{\text{WAN}})$ pair; keep other parameters fixed.
4. **Aggregation.** Report median/p95 TTW and median C_Σ across seeds; include $\tilde{U}$ and active-links tails to diagnose under/over-provisioning.
5. **Front extraction.** Build empirical Pareto fronts per budget/WAN setting; compute knee points (max curvature on the convex hull in (C_Σ, TTW) space).

Analysis and reporting.

- Validate **H12** by exhibiting non-empty non-dominated sets with ≥ 3 distinct (k, a) configurations.
- Test **H13**: along fixed budgets, increasing k reduces TTW until a plateau; report ΔTTW vs. ΔC_Σ .
- Test **H14**: compare $a=1$ vs. $a>1$ at equal budgets; expect lower C_Σ with similar TTW except where WAN bridges dominate.
- Test **H15**: under higher L_{WAN} /lower C_{WAN} , fronts shift upward (larger TTW) but preserve ordering of (k, a) ; recommend knee configurations for low/medium/high budgets.

Threats to validity. Capacity ramp-up bias if warm-up is too short; visualization overhead at large $|V|$; variance from randomized capacities (control with fixed seeds); inability to change link speeds during a run (future work); TTW sensitivity to initiator placement (randomize writers).

Evaluation Criteria.

- **H12**: Non-empty Pareto fronts with ≥ 3 non-dominated points across budgets; fronts stable across seeds.
- **H13**: For at least two budgets, increasing k from $1 \rightarrow 2$ reduces median TTW by $\geq 10\%$; $2 \rightarrow 3$ adds $\leq 5\%$ (saturation).
- **H14**: For $a \in \{2, 4\}$, C_Σ decreases by $\geq 10\%$ vs. $a=1$ at similar TTW (within 5%) in non-WAN-bound regimes.
- **H15**: Under worst WAN, median DR remains ≥ 0.98 at recommended knee points; ordering of (k, a) on TTW is preserved.

Reproducible configuration (example).

```

debug=true
sim_time=500
num_mirrors=128
num_links_per_mirror=2
startup_time_min=5
startup_time_max=10
ready_time_min=2
ready_time_max=20
stop_time_min=2
stop_time_max=5
link_activation_time_min=5
link_activation_time_max=10
fileSize=80
# Budget tier (edit per sweep):
min_bandwidth=2
max_bandwidth=8
fault_probability=0.000
# Topology: NConnectedTopology (+k), SnowflakeTopologyStrategy (+a),
# substructures via BuildAsSubstructure; ring domains  $\geq 3$  mirrors.
# Charts disabled for sweeps; metrics.csv collected for Pareto analysis.

```

1.5.5 RQ5: Framework suitability and design patterns for recursive LRDM structures

Research question. RQ5: Which implementation strategy (design patterns, architecture design suggestions, custom core vs. general graph frameworks) best supports recursive LRDM structures? How can we maintain mirror stability (ID determinism) and fixed-tick semantics when sampling the given framework [Göt23, GBS24]? Which design patterns are most effective to yield robust results?

Scope (no micro-benchmarking). Comparative *engineering* study: semantic fit, determinism, maintainability. No throughput/latency micro-benchmarks. Evidence stems from compile-time checks, semantic conformance tests, and minimal adapter prototypes.

Assumptions from the codebase. Recursive composition via `BuildAsSubstructure` layered on `SnowflakeTopologyStrategy` over `NConnectedTopology`; mirror removal deletes incident links; ring domains require ≥ 3 mirrors (others ≥ 1). Link capacity is configured and ramps up per tick; run-time speed changes are out of scope.

Candidates. (i) *Bespoke core* (status quo with refactorings/patterns), (ii) lightweight adapters for general-purpose Java graph libs (e.g., `JGraphT`, `GraphStream`, `Guava common.graph`). Property-graph stacks (e.g., `TinkerPop/Neo4j`) are considered as *adapters* rather than as the simulation core.

Working hypotheses.

- **H16 (Ports & Adapters).** A hexagonal “Ports & Adapters” boundary (`TopologyPort`) reduces integration risk and preserves determinism across both bespoke core and library-backed graphs.
- **H17 (Tick atomicity).** A two-phase tick scheduler (*compute* $\rightarrow$ *apply*) with *Unit-of-Work* guarantees atomic add/remove of mirrors/links without global restart.

- **H18 (Stable identity).** Combining a deterministic ID service (seeded PRNG or monotone allocator) with an *Identity Map* yields stable mirror/link IDs across runs and across adapters.
- **H19 (Composable recursion).** *Builder/Factory* + *Strategy* + *Composite* encode recursive attachments (periodicity, domain minima) with compile-time validation and fewer runtime errors than ad-hoc construction.

Pattern catalogue (to be evaluated).

- **Strategy** (topology generation: ring/ k -connected, snowflake attachments).
- **Builder/Factory** (validated construction of subgraphs and attachments).
- **Command (Effectors)** for `ADD_MIRROR`, `REMOVE_MIRROR`, `ADD/REMOVE_LINKS`.
- **Unit-of-Work & Two-Phase Apply** at tick boundaries (batch, atomicity, rollback).
- **Identity Map & Deterministic ID Service** (ID stability, replay).
- **Adapter/Port** (`TopologyPort` abstraction to swap core/framework).
- **Observer/Hooks** (pre/post effector, logging, metrics).
- **Composite/Decorator** (node-domain extension without breaking IDs).

Evaluation criteria and rubric (0–2 per criterion).

1. Recursive composition fidelity (periodicity, domain minima).
2. Dynamic reconfiguration without restart (atomic per tick).
3. ID stability & determinism (no hidden reindexing/iteration nondeterminism).
4. Tick semantics (clean compute/apply split; gating via scheduler).
5. Cut/connectivity inspection (degrees, cuts, disjoint paths for RQ1).
6. Event ordering & rollback hooks (pre/post, transactional scope).
7. Integration footprint (deps, licensing, classpath, team learning curve).
8. Visualization bridge (reuse existing charts; optional viewers).
9. Testability (mock-friendly, deterministic iteration order).
10. Maintainability (API stability, docs, adapter/shim size/complexity).

We sum raw scores and flag *hard blockers* (score 0, not reasonably shimmed).

Instruments (probes and harnesses).

- **TopologyPort** (port boundary): `addMirror(id)`, `removeMirror(id)`, `addLink(u, v, cap, lat)`, `removeLink(u, v)`, `neighbors(id)`, `hasPath(u, v)`, `degree(id)`.
- **Composition probe** reproducing `SnowflakeTopologyStrategy` over `NConnectedTopology` (periodicity a , domain minima).
- **Tick scheduler** enforcing two-phase *compute* $\rightarrow$ *apply* with batched Commands (*Unit-of-Work*).
- **Determinism check** (same seed $\Rightarrow$ same IDs, edge sets, effector order).
- **Failure semantics check** (mirror removal deletes incident edges in the same tick).
- **Node-domain extension probe** (Decorator/Composite on mirror roles/attributes without ID churn).

Design (tasks per candidate).

1. Implement `TopologyPort` and a minimal `SnowflakeAdapter` that mirrors `BuildAsSubstructure` semantics.
2. Encode periodicity a and domain minima with `Builder` validation; reject invalid configs at construction.
3. Provide a `TickScheduler` (two-phase) and transactional `Command` batch with rollback on failure.
4. Introduce a deterministic ID service + `Identity Map`; forbid library-driven vertex key relabeling.
5. Add pre/post hooks (`Observer`) for logging/metrics and optional `Memento` for rollback of partial batches.

Procedural steps.

1. **Fit-first pass:** Compile probes against each candidate; record compile-time blockers.
2. **Conformance tests:** Canonical run ($|V|=32$, $k \in \{1, 2\}$, $a \in \{1, 4\}$) with scripted add/remove; assert ID stability and tick atomicity.
3. **Complex case:** $a=8$, ring domain ≥ 3 ; remove a hub mirror and verify expected cut/isolations.
4. **Maintainability pass:** Measure adapter LOC, cyclomatic complexity, number of workarounds, test coverage.
5. **Visualization sanity:** Confirm charts/CSV unchanged regardless of backend.

Analysis and reporting. Present the rubric table (criteria $\times$ candidates) with 0–2 scores and concise justifications; highlight hard blockers and high-risk workarounds. Synthesize trade-offs (framework convenience vs. determinism/ID stability) and recommend either (i) bespoke core with the pattern set, or (ii) a specific framework with a thin adapter and guardrails.

Threats to validity. Adapter quality bias; mitigate via shared acceptance tests. Library updates can change capabilities; pin versions. Determinism may be broken by unspecified iteration order; enforce ordered collections at the adapter boundary. No performance claims.

Evaluation Criteria.

- Each candidate passes the conformance suite (ID stability, tick atomicity, correct failure semantics) or is flagged with a hard blocker.
- A clear recommendation emerges (highest score without blockers) or the bespoke core is retained with rationale (determinism, reduced dependency footprint).
- Pattern implementations compile and run in the current build; metrics/charts remain identical.

Deliverables. Rubric table with justifications; `TopologyPort` interface; per-candidate adapters; tick scheduler + `Command` batch; determinism/rollback tests; and an engineering note documenting refactoring suggestions (`Strategy/Builder` for topology composition, `Command/Unit-of-Work` for reconfiguration, `Adapter` boundary for graph backends, `Identity Map` for IDs, and `Decorator/Composite` for node-domain extension).

1.5.6 Methodology for RQ6 (Suitability of Java Graph Frameworks)

Problem statement and scope. We seek a *qualitative* suitability assessment—not a runtime benchmark—of candidate Java graph cores to extend the mother project *LRDMSimulator* [Göt23] with recursive substructures and live reconfiguration. Candidates must preserve (i) stable mirror IDs and deterministic, reproducible iteration inside a fixed-tick loop; (ii) predictable per-tick mutation costs for adding/removing mirrors and links; (iii) straightforward hooks for instrumentation; (iv) a lean dependency footprint; (v) adapter/visualization options; and (vi) *reasonable implementation effort* to integrate. The evaluation is performed against concrete usage in `SnowflakeTopologyStrategy` (backbone + periodic attachments), `NConnectedTopology` (backbone construction), `BuildAsSubstructure` (recursive attachment), and `ExampleSimulationSnowflake` (end-to-end scenarios).

Artefacts under consideration. (J1) `JGraphT`, (J2) `GraphStream`, (J3) Guava `common.graph`, (J4) slim custom adjacency (project-specific). Property-graph stacks (e.g., Apache TinkerPop/Neo4j) are *explicitly positioned as adapters*, not as the simulation core; they are considered only for optional persistence/visualization bridges.

Evaluation criteria (C1–C6).

- **C1 Determinism & ID stability:** fixed mirror IDs; reproducible traversal (ID-sorted iteration guards if needed); no hidden concurrency that perturbs per-tick order.
- **C2 Mutability per tick:** predictable envelopes for `addMirror`, `removeMirror`, `addLink`, `removeLink`; ability to *batch-apply* mutations at tick boundaries (effectors).
- **C3 Timing & instrumentation:** clean integration with a single-threaded, fixed-tick loop; hook points to emit metrics without global locks.
- **C4 Dependencies & footprint:** low transitive dependency count/size; headless operation in CI (no mandatory GUI/runtime).
- **C5 Adapters & visualization:** straightforward reuse of existing CSV/plot pipelines; optional bridges to viewers/persistence without dictating the core.
- **C6 Implementation effort:** estimated effort to integrate (adapter LOC/complexity, learning curve, required workarounds), categorized as *Low/Medium/High*.

Study design. We combine a requirements-to-capabilities mapping with minimal, runnable adapter spikes and conformance tests embedded in the simulator.

S1 Requirements matrix. Build a table (candidates × C1–C6) with evidence types: API inspection (ID control, iteration guarantees), documentation checks (mutation semantics), dependency tree (`mvn dependency:tree`), adapter options, and a qualitative effort rating. Cells record “*Meets* / *Meets with guard* / *Does not meet*” plus short rationale.

S2 Minimal adapter boundary. Define a small, project-facing interface decoupling the simulator from the graph core:

```
interface GraphCore {
    void addMirror(int id);
    void removeMirror(int id);
    void addLink(int u, int v, double capacity, double latency);
    void removeLink(int u, int v);
    IntStream neighbors(int id); // sorted by mirror ID
```

```

void beginTick(long t); // optional hook
void applyBatchMutations(); // effector barrier (atomic)
GraphCore snapshotReadOnly(); // read-only view for probes
}

```

For each candidate (J1–J4), implement a thin adapter that (i) preserves caller-supplied IDs, (ii) enforces deterministic neighbor iteration by sorting by ID when the library provides no order, and (iii) collects mutations in an effector queue that is atomically applied at tick boundaries.

S3. Conformance Test Protocol. Scenario-driven tests reflecting real use:

- T1 Backbone build:** `NConnectedTopology` constructs a k -connected ring for $n \in \{5, \dots, 512\}$; verify $\kappa(G) \geq k$ via $k-1$ mirror removals without disconnect (structural oracle).
- T2 Periodic attachments:** `BuildAsSubstructure` attaches domain subgraphs with periodicity $a \in \{1, 2, 4, \dots\}$; verify degree/attachment counts match configuration.
- T3 Determinism:** with fixed seed and identical effector schedule, two runs produce identical ID sets, edge sets, and per-tick mutation logs (*byte-identical* CSV of events).
- T4 Live reconfiguration:** in `ExampleSimulationSnowflake`, apply add/remove of mirrors/links mid-run; verify atomicity at `applyBatchMutations()` and no global restart.
- T5 Failure semantics:** mirror deletion cascades incident link removals per configuration; verify complete cleanup in the same tick.
- T6 Scale feasibility:** run $n=512$, $k \in \{2, 3\}$, $a \in \{1, 4\}$ headless; tests complete and instrumentation remains functional.
- T7 Link semantics passthrough:** adapters do not alter `Link.java` behavior (bandwidth accrual, capacity/latency fields).
- T8 Probe integration:** per-tick probes (active links, TTW, utilization) read from `snapshotReadOnly()` to avoid mid-tick mutation races.

S4. Qualitative decision. Using the matrix and tests, classify each candidate as *Core-suitable*, *Adapter-only*, or *Unsuitable*. Record minimal guards (e.g., “sort neighbors by ID”) and the effort rating (Low/Medium/High).

Instrumentation and evidence. Adapters emit a per-tick event log `{tick, op, mirrorId[, u, v]}`. A diff across two runs with the same seed must be empty (determinism evidence). Dependency evidence is the textual output of `mvn dependency:tree`. All tests run headless with the existing CSV/plot pipeline.

Evaluation Criteria (acceptance). A candidate is *Core-suitable* iff:

- A1 (C1)** It preserves caller-assigned IDs and yields reproducible iterations (with ID-sorting guard if needed).
- A2 (C2)** It supports per-tick add/remove with predictable costs and batch-application at tick boundaries (passes T3–T4).
- A3 (C3)** It integrates cleanly into the fixed-tick loop with probe hooks (passes T8) without extra threads.
- A4 (C4)** It runs headless in CI with a lean dependency tree.

A5 (C5) It does not constrain existing CSV/plot adapters; optional visualization/persistence is via adapters.

A6 (C6) Implementation effort is *Low* or *Medium* given current code structure.

Candidates failing any of A1–A3 are *Unsuitable* as core. Those meeting A1–A3 but violating A4/A5 or rated *High* effort are *Adapter-only*.

Threats to validity. We intentionally avoid throughput/latency microbenchmarks; results target *engineering fit*, not performance. Library defaults may change; we mitigate with explicit guards (ID-sorted iteration) and adapter isolation. Scale tests at $n=512$ exercise structure/determinism, not absolute run-time.

Deliverables. (1) Requirements matrix (C1–C6) with evidence links; (2) minimal adapters `GraphCore_J1..J4`; (3) conformance results (T1–T8) and per-tick event logs; (4) a short rationale selecting the core (framework or custom) and positioning property-graph stacks (e.g., TinkerPop/Neo4j) as *adapters* rather than the simulation core.

2 Fundamentals and Definitions

This chapter introduces graph-theoretic, networking, simulation, and software-engineering terms used throughout the thesis. Where appropriate we cite standard sources (e.g., [Men27, Har62, FH70]).

2.1 Graph-Theoretic Concepts

Graph $G = (V, E)$: Finite simple graph with vertex set V and edge set $E \subseteq \{\{u, v\} \mid u \neq v \in V\}$ unless stated otherwise.

Vertex/edge ($v \in V, e \in E$): Nodes and undirected links of G .

Degree $\deg(v)$: Number of incident edges of v ; $\Delta(G)$ and $\delta(G)$ denote max./min. degree.

Neighborhood $N(v)$: Set of vertices adjacent to v .

Path/Cycle: A path is a sequence of distinct vertices with consecutive vertices adjacent; a cycle is a closed path of length ≥ 3 .

Distance $d(u, v)$: Length of a shortest u - v path. **Diameter** $\text{diam}(G) = \max_{u, v \in V} d(u, v)$.

Connected graph: Every pair of vertices is joined by a path.

Vertex cut / edge cut: A vertex (edge) set whose removal disconnects G . A *minimum* cut has minimum cardinality among all such sets.

Vertex connectivity $\kappa(G)$: Size of a minimum vertex cut (or $|V| - 1$ for complete graphs). G is **k -connected** if $\kappa(G) \geq k$.

Edge connectivity $\lambda(G)$: Size of a minimum edge cut; G is **k -edge-connected** if $\lambda(G) \geq k$.

Menger's theorem: Two distinct vertices are connected by at least k internally vertex-disjoint paths iff every vertex cut separating them has size at least k [Men27].

Vertex-/edge-disjoint paths: Paths sharing no internal vertices / no edges.

Ring C_n : Cycle graph on $n \geq 3$ vertices (a simple “backbone” in this work).

Harary graph $H_{k,n}$: A k -connected graph on n vertices with minimum possible number of edges (edge-minimal for given k) [Har62]. *Note*: We do not require edge-minimality in this thesis, but use $H_{k,n}$ as a reference concept.

Corona product $G \circ H$: For each $v \in V(G)$, attach a fresh copy of H and join v to every vertex in that copy [FH70].

Snowflake composition: Informal term in this thesis for periodic attachment of domain subgraphs to a backbone (Section 2.4).

2.2 Network and Performance Concepts

Latency / propagation delay: Time for a unit of data to traverse a link or path.

Bandwidth / capacity: Maximum data rate a link can carry (per time unit); *effective bandwidth* may accrue over time as modeled in `Link.java`.

Throughput: Achieved data rate end-to-end.

Utilization: Fraction of link capacity used over a window.

Delivery ratio: Delivered / sent messages (or bytes) over a period.

Consistency (mirroring): Degree to which replicas hold the same data; **time to write** denotes ticks until an update is fully mirrored.

Failover / resilience: Preservation or rapid recovery of connectivity and service under failures.

WAN bridge: Inter-domain edge with high latency and limited capacity (Internet-scale link).

2.3 Simulation Semantics

Discrete time / tick: Simulation advances in fixed quanta Δt (here typically 250 ms), each tick invoking `timeStep(t)`.

Mirror: A simulation entity (node) with an explicit state machine (e.g., startup/ready/stop) and a stable identifier (ID).

Link: Simulation edge with latency/capacity semantics (data transfer governed by `handleDataTransfer`).

Effector: A batched mutation (add/remove mirror or link) applied atomically at tick boundaries (no global restart).

Runtime reconfiguration: Application of effectors during execution (evolution of topology over time).

Determinism / ID stability: Fixed IDs and reproducible iteration orders for equal seeds and schedules.

Seeded PRNG: Pseudorandom generator with explicit seed to ensure reproducibility.

Headless mode: Running without UI/visualization to enable automated sweeps/CI.

2.4 Structural Compositions Used in This Thesis

k -connected backbone: A connected core with $\kappa(G) \geq k$ (e.g., C_n with additional chords or other k -connected constructions).

Domain subgraph: A (heterogeneous) graph attached to the backbone representing an edge site or sub-network.

Periodic attachment (periodicity a): Attach a domain subgraph to every a -th backbone vertex (offset modulo the backbone size n).

Snowflake topology: Backbone plus periodic attachments, built via `NConnectedTopology` and `BuildAsSubstructure`.

2.5 Metrics and Indicators

Active-link fraction $\rho(t)$: Ratio of links currently up/usable at tick t .

Reconfiguration latency: Ticks between effector invocation and system reaching a quiescent post-change state.

Convergence time: Ticks from disturbance to restoration of specified invariants (e.g., delivery ratio or connectivity thresholds).

Cut size estimator $\hat{\kappa}$: Empirical lower bound on vertex connectivity inferred from failure experiments.

Scenario key / manifest: Encodes configuration (seed, k , n , periodicity a , WAN parameters) for reproducibility.

2.6 Software-Engineering Terms (Adapter-Based Integration)

Adapter: Thin layer that maps project-specific operations to a graph library while preserving IDs and determinism.

Core vs. framework: Choice between a bespoke adjacency structure and a general-purpose library (e.g., JGraphT, GraphStream, Guava `common.graph`, Apache TinkerPop, Jung).

Instrumentation: Probes and logs collected per tick (CSV, plots) without perturbing timing semantics.

Design patterns: Builder/Factory (topology composition), Command (effectors), Observer (probe hooks).

Parameter sweep / Pareto front: Systematic variation of inputs and multi-objective analysis of trade-offs.

2.7 Assumptions and Boundaries

Unless stated otherwise, the simulator advances in fixed-size ticks of 250 ms and employs deterministic scheduling. Mirror identifiers remain stable over time; when a mirror is deleted, all incident links are removed according to configuration. Regarding domain sizes, ring backbones are defined only for $n \geq 3$, whereas the other topology families admit domains of size at least one. Attachments follow a periodicity parameter $a \in \{1, 2, 4, \dots\}$, where $a=1$ denotes attachment at every backbone vertex. Wide-area parameters—latency and capacity—are fixed prior to each run; dynamic link-speed changes are not modeled and are left for future work.

3 Problem Analysis and Requirements

This chapter suggests research papers and known experience on the LRDM simulation topic and prepares its Research questions/phases by summarizing known best practice. We will try to reflect this knowledge in connection with the learning's of the project.

3.1 Problem Analysis and Requirements - Plan and Contribute examples using LRDM Simulator

This sections shows how topology examples were formally introduced to the project, why they are chosen as base examples to the predecessor work in this project and how they affect the development process. Further we will touch research papers that suggest these topologies in network architectures for different reasons in the past. Finally we want to summarize the process to find a reasonable base topology for introducing a recursive LRDM simulation structure to prove basic extended functionality like adding and removing mirrors without a topology reset. The base Question of this topic is Research Question 1 and 2.

3.1.1 Occurring Problems

While trying to prove and explore the usability, extendability and integrity of the base LRDM simulation project, the first occurring problem is the fundamental and dynamic base topologies that cannot be reduced or split into subgraphs. The existing fundamental graph simulation of the previous LRDM project, includes a `nConnected`, fully connected and a 2 children-node b-tree graph. All these fundamental topologies were only able to add mirrors to the structure, measure metrics like bandwidth, total links used and time to write, compare their state to a predictive measure what the metrics should be and finally reset to an uninitialized state. The prediction of these topologies was set to static calculations, that do not consider dynamic changes or crashes of mirrors on the graph. When we consider a crashing mirror on an `nConnected` ring with a connectivity of 1, we receive a line. This topology is missing as a standard building block for a fall-back and `nConnected` repair method. Another relevant topology to model compound structures is a star topology, that has one mirror at its center. It is true that a star-topology is a special case of a B+-Tree with only one level and a fixed set of n child-nodes and the root at its center, but it also allows us to simplify testing compound structures with a possible star in the middle, without configuration overhead of the B+-Tree. On the original LRDM Project [Göt23, Göt] the root node of all structures is not allowed to crash and disabled from taking effects from the graph. This is an unrealistic assumption. We are also only able to define a single root as source, but usually submitting data from multiple endpoints is possible in the real world. Publishing the same version of a release at the same time is known as an option of a sabotage attack on larger corporate systems, resulting in data inconsistencies. By those means we should support such a feature as an important research goal to prevent such incidents.

3.1.2 Consequence

Without adjustable B+-Trees with a depth limit or a balance limit, the LRDM simulation has a very limited capability of simulating real world network problems. Without the star topology, testing compound structure building is hard. Without a line, linear Connection of structures is hard to represent and causes an overhead of configuration too. The suggested missing implementations prevent efficient planning of

topologies, that are required to simulate network situations like a local program version mirroring system or a relay structure build on network switches. Without multiple roots we cannot simulate graph segmentation and data inconsistencies.

3.1.3 Solution

We require implementations of B+-Trees with either a depth limit or a balance limit and a defined number of maximum children n . Structures like B-Trees need to be generalized, serving as a base structure to either of the 2 concepts. nConnected structures need to be derived from a ring pattern, instead of separating the concepts of rings and nConnected structures. For ring patterns we use the research from I-Shyan Hwang et al. [HHC05] to understand the transition and sub-structuring of rings into nConnected topologies and their fault tolerance for rerouting on mirror fault and while adding mirrors. A line pattern is additionally introduced to interconnect topologies to simulate custom coupling of structures, also indirectly shown in Yu Feng et al. [FWLC24] - the most basic fault resistant interconnection between subgraphs is a line. We need to implement star-topologies and lines for test purposes and structure fallback and repair. Since the root node is indestructible on the original LRDM project, a star offers a stable environment to append different structures to a root center. Interesting fault resistant topologies on star topologies are shown by Valter Filipe Silva et al. [SFF06], that also build the fundament to simulate star-connected subgraphs. All structures must also be able to dynamically add and remove mirrors. Finally we would need a data registry together with a conflict detector that crashes mirror on detecting data inconsistencies. We had to leave this last point out due to time constraints.

3.2 Problem Analysis and Requirements - Extension concept and reproducibility

This section discusses the configuration of architecture and design patterns of the original LRDM design [Göt23, GBS24], the stumbling blocks on frameworks, reasoning and known work on the topic. Here we contribute to improve the analysis and implementation of the original LRDM Simulation to the solution we have found, after we conceptualize the setup in the last section. This is done by suggesting analysis and implementation findings of the current project on the LRDM extension of this specific topic. The base question of this task is research question 5 and 6.

3.2.1 Occurring Problems

In the original setup of figure 1 Figure A.1, we can see that the UML diagram of the project shows, that the core of analysis is a strategy pattern, setting topologies and effectors. Each topology strategy also considers a setup to build a graph, but there is no conceptual split between target graph architecture and simulated graph architecture. We are not able to set up a graph architecture that defines a repairable base state that we are able to return to in case of mirror crashes - no matter where they appear. Each mirror is not distinguishable from one another in case of graph and subgraph membership. In case we would construct multiple subgraphs that overlap each other, substructures are not able to be split and merge into each other. There is no hierarchical backbone to append graphs onto one another on a lower level of reference or set structures into compound graph collections on the same level of reference.

When looking at available frameworks, we realize that there are different targets to why these frameworks are created and maintained:

- **JGraphT [MKNS20]:** JGraphT is an in-memory Java library centered on a generic $\text{Graph}\langle V, E \rangle$ abstraction that cleanly separates graph capabilities from their concrete storage. Capabilities — directed or undirected edges, allowance of self-loops, multi/pseudograph semantics, and optional edge weights — are expressed through type variants (e.g., `WeightedGraph`) and a `GraphType` descriptor, while concrete implementations such as `SimpleGraph`, `DirectedPseudograph`, or `WeightedMultigraph` encapsulate the actual adjacency maps/sets. Construction is typically handled via

fluent builders (`GraphTypeBuilder`, `GraphBuilder`), and a rich set of “views” (e.g., `AsSubgraph`, `AsUnmodifiableGraph`, `AsWeightedGraph`, `EdgeReversedGraph`) enables derived graphs without copying. Beyond the core data structures, `JGraphT` offers a large algorithms module (shortest paths, flows, matchings, centralities, etc.), I/O utilities for formats like DOT/GEXF/GML/CSV, and adapters to external toolkits. Modifications and traversals can be instrumented through `ListenableGraph`, `GraphListener`, and `TraversalListener`, which emit events on node/edge changes and traversal progress—useful for simulation hooks. Default implementations are optimized for predictable iteration and algorithmic performance but are not inherently thread-safe; concurrent access can be wrapped with `AsSynchronizedGraph`. Equality, cloning, and serialization semantics are documented to keep vertex/edge identity stable while copying internal structures.

- **GraphStream [PDGO08]:** `GraphStream` is designed around the idea that a graph is best represented as a stream of events over time. Its core architecture uses sources (which produce events), sinks (which consume events), and pipes (which can do both). A `Graph` itself is a pipe, so you can connect file/network sources $\rightarrow$ graphs $\rightarrow$ analytics $\rightarrow$ visualization as a chained pipeline, even across threads or processes. The data model supports directed/undirected and multi-edges, with flexible attribute maps on the graph, nodes, and edges to attach dynamic metadata. A built-in UI subsystem provides a live viewer with layout engines and CSS-like styling so visual state (colors, sizes, styles) reacts to attribute changes in real time. Subprojects cover the core graph model, algorithms for dynamic scenarios, `UI/renderer`, and `NetStream` for shipping event streams over the network, allowing distributed simulations and remote visualization. Storage backends emphasize either fast access or compactness, and the event API (add/remove nodes and edges, attribute changes, time steps) provides natural hooks for stepwise simulation, instrumentation, and reproducible demos.
- **Guava `common.graph` [The25]:** Guava’s `common.graph` supplies lightweight, composable graph primitives focused on modeling rather than algorithms or persistence. It exposes three peer abstractions that differ in how they treat edges: `Graph<N>` models graphs with anonymous edges, `ValueGraph<N,V>` attaches a value to each edge, and `Network<N,E>` gives edges their own identity as first-class elements. Each abstraction comes with mutable and immutable implementations and fluent builders (`GraphBuilder`, `ValueGraphBuilder`, `NetworkBuilder`) to specify directedness, self-loops, expected sizes, and multigraph behavior. The package intentionally avoids dictating a particular storage; adjacency lists/maps are an internal detail, enabling compact or fast implementations as needed. Utilities cover common operations (successors, predecessors, induced subgraphs) and well-defined equality semantics (e.g., two graphs are equal if they share the same node set and adjacency structure). Mutable implementations are not synchronized; the immutable ones are thread-safe by construction. Because there is no heavy algorithms or I/O layer, `common.graph` keeps the dependency footprint small and is often used as a clean domain model for relationships inside larger systems.
- **Apache TinkerPop / Gremlin [Rod15]:** Apache TinkerPop is a vendor-neutral graph computing stack that standardizes the property-graph model and provides the Gremlin traversal language and runtime. At its foundation are interfaces such as `Graph`, `Vertex`, `Edge`, and `Traversal` that providers implement, making applications written against Gremlin portable across backends (e.g., in-memory `TinkerGraph`, `JanusGraph`, or cloud services). The “Gremlin traversal machine” defines a step-wise execution model that can target two modes: OLTP (interactive, transactional traversals over an online graph store) and OLAP (batch analytics via `GraphComputer` on systems like Spark/Hadoop). Gremlin exists as a language and a bytecode format, so traversals can be constructed in multiple host languages (GLVs) and shipped over the wire. Gremlin Server exposes a remote endpoint (`WebSocket/HTTP`) with configurable authentication, SSL, and execution policies, allowing external clients and tools to submit traversals for execution. The architecture is intentionally extensible: providers contribute storage/indices and execution strategies, and users

can register custom traversal steps, serializers, and security models—yielding an interoperable, horizontally scalable graph layer for both operational queries and analytics.

- **Neo4j [RWE15]:** Neo4j is a native property-graph database engineered for transactional workloads and expressive querying over highly connected data. Its storage engine organizes labeled nodes and typed, directed relationships with key-value properties, backed by ACID transactions and a cost-based query planner for the Cypher language. The runtime includes advanced execution strategies (e.g., pipelined/slotted operators), schema features (indexes, constraints), and a page cache that maps on-disk stores and indexes into memory for predictable performance. Clients connect primarily via the Bolt protocol (with official drivers for major languages), and administrative endpoints support HTTP and browser-based tooling. In clustered deployments, Neo4j’s causal clustering assigns write and read roles to members, provides replication and failover, and can be tailored for read-heavy or analytics-oriented topologies. The platform is extensible through procedures and user-defined functions (e.g., APOC), and the Graph Data Science ecosystem layers in-memory projections and algorithm libraries on top of the core engine for scalable graph analytics. Overall, Neo4j’s architecture emphasizes operational robustness, query ergonomics, and ecosystem integrations rather than being a general in-memory algorithms toolkit.

JGraphT offers a great range of algorithms and simulation tools for compound graphs, as well as cascading recursive subgraphs. In case we would use the framework for extending the LRDM simulation to subgraph capability, we would require to build many Adapter classes to match our existing event based Effect pipeline, matching multiple defined changes on each of our custom events. *JGraphT* focuses on performing single events on vertices and edges. The current implementation is based on different design patterns, the most important is the command design pattern on formulating examples over a time based strategy pattern for Topologies and a time based flyweight pattern to formulate events and set up probes. The approach of *JGraphT* is based on builders that handle nodes and edges, but we already have implemented mirrors and links instead, that would be required to match *JGraphT* definitions. The mirrors need to be generated, deleted or crashed by defined terms. *JGraphT* is focused on building graphs directly, but not on splitting planning and construction to calculate changes required to return to the original structure set in the beginning.

GraphStream offers a great library to set up event based graph configuration. Once more we already have Mirrors and Links, further an event pipeline that we need to adjust to the framework. *GraphStream* does focus on analytics, but less on the compound nature that we aim to design. Besides reworking and adapting the entire event pipeline, we would also require to rework the cascading nature of defining subgraphs and define our own topology levels.

Guava offers simple graph composition to represent connection models of the real world. It is not capable to create views on first class subgraphs, level architecture and views. Its main focus is the visualization of datasets like excel sheets and custom data input. Therefore almost none of its functionality is useful to construct an event based and timed LRDM simulation. We would only be able to construct simple graphs that were already supported in the previous version of [GBS24, Göt] that we are improving upon. *TinkerPop / Gremlin* offers an entire graph language with gremlin server backend and some Java, Python, .Net, JavaScript, and Groovy frameworks on the client side. Even though graphs can be generated recursively, they remain in the base graph construct as a single graph. The *TinkerPop* framework is also vast in functionality to support many use cases. Since we would have to implement the server side and the client side for a simulation, that was supposed to run locally, we expect a lot of planning and implementation overhead by doing so. The main use case for the framework is more focused on creating a fundament of corporate planning and management process tools that can be access by a wide range of users within a company.

Neo4j is an entire graph database. We did not even want to persist graphs right now. Our goal is to prepare a research setup for some custom combinations of graphs to get insights into their robustness behavior. We do not require administrative support, graph processing in pipelines, fancy Cypher extra languages to define our problem space and an entire server client approach, where we did not consider

any client construction yet.

3.2.2 Consequence

The discussed problems cause consequences on the suggested use of a graph framework:

JGraphT is not suitable as a project framework, since it would require us to implement many adapters to match our previous architecture while maintaining new concepts of compound event handling.

GraphStream is not suitable as a project framework, because besides finding and implementing a representation of subgraphs, we would need to adapt our existing event based architecture to the framework architecture, implying extra work.

Guava is also not suitable as a project framework, because it cannot handle the complexity of levels, topology assignments and subgraphs, as well as not being suitable for event based processes.

TinkerPop / Gremlin is an excessive graph application framework, creating effective planning and implementation on medium and large scale projects exceeding at least 2 software engineers for the backend and the frontend. Therefore it is not suitable for a small scale research project.

Neo4j is not a match for our event- and recursive-graph based project, since we do not want to persist graphs and results right now, since we can copy the results from the command line. An entire graph database completely exaggerates and exceeds our use case.

3.2.3 Solution

Finally, we can summarize that, in terms of showing results, building on top of the work of the existing LRDM simulation project may be the most effective way. The implementation is already event based, running without any overhead and offering basic metrics and probes to show comparable results to the initial work in the process - over calculation latency to changes taking effect. Graphs presenting hundreds of nodes, can reach a connection degree of thousands of edges, requiring a hardened framework without overhead for the designated use case.

Another advantage on reusing the initial architecture is reusing its already constructed test framework and -structure to validate the feasibility of concept. In case we would diverge from already tested scientific facts, we may create ourselves a hard time reaching a comparable quality of the results. Using any framework will increase the potential of functionality and maturity under the requirement of setting up a larger development team, producing a more sophisticated approach of robust graph LRDM framework.

3.3 Problem Analysis and Requirements - We present: A snowflake pattern

In this section we discuss the evolution of the previous project [GBS24, Göt] into an recursive example structure with the help of a framework extension under the decision of section 3.2. To find valuable corona ring / "snowflake" combinations and an example that we can rely on for construction, we require a discussion about important metrics combinations and their connection to such a compound structure. Our goal is to find a good compromise fundament to plan and develop the desired changes. The base questions of this task is research questions 3 and 4.

3.3.1 Occurring Problems

In the previous LRDM simulation, each simulation example was designed to test a single topology under a specific and fixed event schedule, under the use of measurement probes. Now as we are extending the previous project by hand to support compound structures, we also need to take a look at how the initial

metrics deliver useful results to analyze such structures, since creating graphs with subgraphs will generally stretch the problem size. Increasing problem sizes may lead metrics like the measurement of active links to show no significant result or graphical representation, since the use of data transfer and utilization on larger compound structures is relatively low - even though this analysis is a common phenomenon within large scale data centers to simulate the effect of hardware extensions. Further, when analyzing topologies, we cannot distinguish between the metrics of different subgraphs and the framework does not yet have a concept to spread probes among defined limited graph shards and subgraphs. This leads to less useful results on simulating big compound topologies, since we can neither analyze the metrics and isolated stability of a single subgraph, nor can we measure the metrics of data flow bandwidth and links between two subgraphs, that are part of the master structure. Finally when there is no measurement of subgraph metrics, there are also no dedicated subgraph graph visualization metrics on our head panel to begin with.

3.3.2 Consequence

Since the initial project was designed to simulate a single topology structure, it is not suitable to create valid results for our compound extension. This leads to the need to not only extend the framework to a recursive event and topology structure, but also develop BuildAsSubstructure dedicated probes and metrics.

3.3.3 Solution

Instead of using another abstract class on top of the definition of TopologyStrategy to define a subgraph, as we showed in our implementation, we should implement the network itself as a recursive structure, carrying mirrors and links for each substructure. Since this would create a big overhead of work by merging the event pipeline of the LRDMSimulation and the Network class into a network singleton carrying flyweight classes, together with creating a distribution architecture of probes running on different networks, we constructed a minimal invasive BuildAsSubstructure for now. This substructure carries the information of custom set observers, that analyze the substructure and return metrics when requested for now. BuildAsSubstructure also carries information of its specific mirror connections over a new MirrorNode and StructureNode Mirror-wrapping, planning implementation, that is capable of analyzing compound target architecture beside the instable mirror connection reality. Due to time constraints this solution leads to the necessity of evaluating the subgraph substructure metrics observers, against the overall structure metrics, to calculate the in between subgraph metrics results.

3.4 Problem Analysis and Requirements - Architecture, Design and Effective project transformation

In this section we discuss the findings on project architecture of the original LRDM implementation with the decisions of the last 3 section stages, single topologies and concept, frameworks and design patterns, metrics and "snowflake" pattern construction (Figure A.24). From the findings we evaluate what the LRDM simulation is capable to evolve into from its core and how it can be developed. We will clarify how it should not be build and why. This section also gathers and summarizes research papers and reflects on the empiric project progress by gathering further suggestions to future developments from already existing research projects. The base question of this task is research question 2 and 5.

3.4.1 Occurring Problems

As we already mentioned in section 3.3, there are some architectural implications of the LRDM simulation project, that make it hard to use and extend functionality or lift it to an industrial standard. We added

some examples how extensions could be extended further on Appendix B and Appendix C. Let's check the top ten of architecture flaws:

- **Flaw 1 - the TimedRDMSim has a network Figure A.4:** In the original LRDM simulation architecture we can find a global network variable in TimedRDMSim. If the TimedRDMSim has a network it master administrates the network behavior. This leads to the static assumption, that the simulation usually dictates the events on the network, instead of the network being a model of the real world that requires to check in with an updated state of a running distributed system. In industrial terms this approach is not flexible enough since the users main target is simulation in a first step and most of the time measurement/testing of real environments in a second step.
- **Flaw 2 - flyweight done wrong - graphical maintenance on design Figure A.5 and Figure A.6:** In case we carry entire network objects as a flyweight to external interchangeable objects (like an implementation of a graph VisualizationStrategy or Action), we may hurt maintainability of the project. The only task of any graphical user interface is to represent available data to the user. Since the network object is a central concept that will be open for discussion in the future, we need to expect changes to its structure, as well as to its setters and getters.
- **Flaw 3 - mirror crash is not an action Figure A.7 and Figure A.8:** In the mirror class we can find a crash(simTime) function that randomly crashes mirrors by a certain probability when handling mirrors on each tick. This implies that the crashing events on a mirror do not communicate with the Effector of the network and do not count as an action pattern. There is no concept of sending update messages along the architecture on the mirror crashing event.
- **Flaw 4 - selective probing Figure A.9:** When measuring probes on distributed networks and sub networks, we require to extend the base of the project to probe every single substructure. Acquiring extended context information on base graphs and their relations is key to understand improvement of test graphs.
- **Flaw 5 - links and streams Figure A.10:** On each link there is a possible transmission range from a bandwidth minimum to a bandwidth maximum, but there is no model for real world link performance limit. A link of a graph could be a communication connection, a transport line of physical goods or an abstract concept connection to visualize quantic means. Links are the construct that make the resources to be measured "flow" along the graph. In this case also a data package is too specific, since there are multiple concepts of flows and transportation: The common concepts of flows are steady flow, logarithmic maximum threshold flow, biased flow and random flow (optional between a min/max as we have observed on the old project). For transportation we should consider unidirectional, mono directional, symmetric or asymmetric transportation. There are no factory methods making link behavior available on creating any link and the rules cannot be controlled by setting these link-building-properties to any TopologyStrategy. Finally a link connection cannot be segmented into fine grained streams, that recursively define link behavior as described for flows and transportation. For now any transport is a single file with a fixed size to be synchronized along the network. It is not possible to represent a stream of multiple files as a package or several files being synchronized in parallel with a simulation of fault behavior, relevant to the field of software fault tolerance.
- **Flaw 6 - dynamic bandwidth Figure A.11:** Analyzing the previous LRDM simulation project, we cannot estimate and simulate behavior on bandwidth, since its behavior is completely random by the base model between the limit of a minimum and a maximum threshold. Changing the limits of bandwidth or link throughput, using time checkpoints, is currently not available. In the real world, bandwidth means, usage, faults and industrial production follow principles, that can be models using basic mathematical functions and limits within ranges, that we cannot model in the current approach regarding link configuration. Currently we can remove links from the structure at a specific time step, canceling the transmission to zero.

- **Flaw 7 - topology strategy with effect prototype Figure A.12:** The paper from Götz et al [GBS24] proposes a change prediction on calculating the graph link stability of B-trees, fully connected topology and n-connected topologies. The estimations of Actions can be calculated in advance, like changing the number of target links, number of mirrors or the topology. These actions are unified in one single Effect class. The update on Effect hierarchy was not implemented due to time constraints, we focused on measuring the effects of change.
- **Flaw 8 and 9 - singleton link and mirror list, and no flyweight mirror creation Figure A.13 and Figure A.14:** In the old LRDM simulation [Göt] implementation, the createMirrors function happens to be inside the TopologyStrategy class. This is problematic, since mirrors in the real world are created and then registered to the network. Created mirrors is also non linear in creation, since single mirrors can crash, interrupting the continuity of mirrors. Therefore the function is in the wrong spot with missing capabilities. Since the handling of mirrors and links is distributed among the project and not unified, we cannot extend it to recursive substructures (Flaw 8). Because a TopologyStrategy does not register its mirrors and links with a class state, we cannot distinguish probe metrics and fault behavior of segmented substructures, as well as their reconnection rules (Flaw 9).
- **Flaw 10 - no dataset split and fault recovery simulation:** To analyze WAN fault behavior with rejoining downloads, we require different modes to rejoin data streams and started downloads. In the real world a data transmission is either dropped on a link fault or rejoined using checkpoints to stop loss of progress. Further data packages can be transmitted via multiple links in parallel by splitting the load of one data package in to different sections.
- **Flaw 11 - master effector:** The network carries a single effector to check into the timestep events and update the network towards their actions. Usually, in a remote setup we can find multiple network based effectors that are activated on many occasions regarding the overall model state.

3.4.2 Consequence

- **Flaw 1 - the TimedRDMSim has a network:** Since the project structure does not include an exchangeable event and signal handling against real world signals, the adoption rate for this research topic may stay very low. The industry is unable to adopt frameworks that have no interchangeable signal source, due to restricted cost and project time. This is already fundamentally explored by Dahmann et al [DCW99]. The design also prevents us to recursively distinguish sub networks to activate events or measure recursive mirror crashes and associate the event to the main and sub-structures. In this architecture it is not possible to establish flyweight sub structure defined mirror views on the targeted measurements of the network.
- **Flaw 2 - flyweight done wrong - graphical maintenance on design:** Since we expect a well defined data stream from the network to update the state of any VisualizationStrategy or derivation of an Action, implementation changes on the network object must never affect any external structures. Else the maintenance overhead of testing and verifying changes will not outweigh the potential of the tool.
- **Flaw 3 - mirror crash is not an action:** Since crashing a mirror is a specialized Action to set a random number and pattern of network mirrors on updating the effector, the changes on the network are committed in the wrong place. We need a new concept, as also discussed in flaw 11, to extend the Effector class to compound actions. If a mirror crashes, the TopologyStrategy requires to check its own state and detect changes. In case we want to test a new implementation, we are confronted with many Heisenbugs or even Mandelbugs. This is because the mirror crashing events cannot be tested under the same circumstances when crashes are being generated randomly. By this flaw we cannot set up crash patterns up front in a static timed fashion, leading to unreliable

and ineffective bug fixing. We need to retry running the code, until the bug appears again. This may take time, leading to frustration and low adoption rates.

- **Flaw 4 - selective probing:** In case we are just able to measure probes on the total structure, we will not be able to deliver the flexibility to simulate datacenter load maps or industrial process quantity flows. The project may not be used as a fundament to any use case. If there is no separation into identifiable substructures with a set of unique probes, a split graphical representation on graphs, as well as logging becomes unthinkable to develop on top of the framework and we are not able to recursively filter and evaluate chosen and combined performance shards of the graph.
- **Flaw 5 - links and streams:** Being limited, we cannot show the impact of reserved bandwidth, asymmetric internet connections, transmission parallelization, fault tolerance or parallel processes of manufacturing, that would make this tool powerful in the face of digital measurement and automation on many use cases. Since the tool is specialized on data transfer, we are not providing solutions to other heavy industries. Because we cannot recursively model link segmentation, research teams and companies might find it hard to model relations and their effects. Possible fields might be, for example, modeling cloud customers (B2B) and their end consumers (2 levels), leading to low adoption rates and low community traction for further developing the idea of this tool.
- **Flaw 6 - dynamic bandwidth:** We find it hard to model any valuable transport of objects/data transfer on any graph, because we cannot get close to situations found in the real world. Without timed steps on the link structure, events like link failure or breaking industrial assembly lines between manufacturing steps cannot be modeled. This limits the old implementation of the LRDM simulation to show mirror crashes and their effects.
- **Flaw 7 - topology strategy with effect prototype:** Since every TopologyStrategy generates its unique effects in the real world, that can be calculated by recursively connected topology structure effects, we are confronted with a software architectural problem. We cannot calculate the Effect of substructures, because generating an Effect object of a substructure is not returned by any derivation of the TopologyStrategy class.
- **Flaw 8 and 9 - singleton link and mirror list, and no flyweight mirror creation:** Because links and mirrors are not organized as separate constructs, we have a hard time adapting changes to the substructure handling and snowflake implementation. Further we are not able to remove mirrors without crashing the id alignment of mirrors in the Network class, resulting in a disruption of the getNextMirror() function. As soon as we try to get more new mirrors, after other mirrors have already crashed, the function fails. Therefore getNextMirror() is not usable for spontaneous mirror creation and deletion on different substructures.
- **Flaw 10 - no dataset split and fault recovery simulation:** Without being able to set up dynamic transmission processes, that automatically adapt themselves from a collection of multiple sets of active links, to transmit data from a mirror to another target, we cannot simulate real world network behavior. Further we also lose the potential to simulate equivalent economic models like oil transport lines and disaster simulation.
- **Flaw 11 - master effector:** Since there is only one Effector class as a singleton pattern within the network class, all events are checked and executed on the same input tick. There is no option to conditionally exclude an Action per definition under a registered time step.

3.4.3 Solution

- **Flaw 1 - the TimedRDMSim has a network:** In case we would consider a network as the top architecture structure of the project to be configured in an example simulation, we could introduce the TimedRDMSim as a strategy pattern with a new "ActionManager" interface base to manipulate

the network over the Action and Effect objects applied to a second introduced TopologyStrategy strategy pattern within the network object. In this case the roles of Network and TimedRDMSim swap and the network is setting itself to the chosen ActionManager aka TimedRDMSim as a flyweight global variable. The timing and runStep(int t) of TimedRDMSim/ActionManager can be called via a network function runUpdate(), where TimedRDMSim must be a stateful timed object and an implementation of ActionManager. In case we were planning to transfer the project to the real world and build a measurement tool for our datacenter by connecting to different nodes on the system, we would need to write another ActionManager that may read real measurements of the current local machine or receive measurements of a defined remote node target on the system. The runUpdate() function in network would allow us to define an external heartbeat timing, that is statically set to 250 ms in case of using the TimedRDMSim. In case we would process real measurements, we could also use dynamic event based timing strategies to reduce our energy footprint, only updating after a maximum threshold and when events force us to do so.

If a network carries a TopologyStrategy, each TopologyStrategy should be extended to carry a new set of networks, in case this TopologyStrategy is separated into many flyweight views of the master network. Events should update effects via a visitor design pattern throughout the recursive networks and sub networks. We showed this design fundament by extending TopologyStrategy with the abstract BuildAsSubstructure recursive class. It allows us to show recursive network structure organization and updates. Apparently implementing the pure principle of the ActionManager was too costly on the implementation side to be realized within this project.

- **Flaw 2 - flyweight done wrong - graphical maintenance on design:** In case we need information on an object subject, we should always try to go the extra mile to introduce an observer design pattern. Since the network may change throughout events, it produces metrics that are dependent on its recursive BuildAsSubstructure views. In case we require information about our network or any subnetwork, we should initialize a list of observers on the required networks, which become registered into accessors like Actions and VisualizationStrategy objects. With this design we can ensure the consistent transfer of updated data, stats, topology and system information beyond critical implementation concepts, that will evolve on this topic over time. We have shown an observer pattern on BuildAsSubstructure.
- **Flaw 3 - mirror crash is not an action:** The Effector should support compound actions - like in flaw 11 - by splitting into dedicated and specialized Effector classes. In case we want to make an Action notify our architecture on multiple mirror crashes, we can use a decorator design pattern to extend existing Action classes like MirrorChange, repeating it on multiple occasions. By setting up mirror crashes as an event or Action class, we are now able to set this Action up as a static test case, to prevent unnecessary/unwilling fuzzing tests, were a straight test on functionality requires to be well defined.
- **Flaw 4 - selective probing:** To enable selective probing, we need to build a hybrid solution with a structure of probe observers that are set on change of the BuildAsSubstructure class to be read by the VisualizationStrategy and we require recursive probes that relate values along several leaf-probes on their B+-evaluation-tree. The leaf-probes and their recursive probe counterparts should be defined in advance to any example run or should be graphically defineable, as soon as the tool reaches a higher state of maturity.
- **Flaw 5 - links and streams:** To offer a solution to most link limitations, we should create a flow class, defined by the settings of the TopologyStrategy(new flowRules()), we should offer TopologyStrategy(new transportRules()) and we may also define a class ConnectionOverride(mirror1,mirror2,flow,transport), to define a single direction set of rules within the graph to model special cases. These implementations are suggestions, due to time limits they were not implemented.

- **Flaw 6 - dynamic bandwidth:** In the last point, we have suggested simulating streams on flaw 5. To simulate real world events, we should offer a LinkFunction interface that is assigned to a substructure with a set of mirror to mirror connections or directly assigned to a stream or a set of streams of a link connection. Alternatively we can extend the properties file by setting specific links, link streams, timed mathematical functions for each stream and fault rates. This point was also not implemented.
- **Flaw 7 - topology strategy with effect prototype:** We need to review the Effect class and replace it with an Effect interface that offers the necessary functions to be implemented for every TopologyStrategy. Each derived Effect class implementation should be placed inside a TopologyStrategy as a singleton called by Effect containing network with the current topology strategy. Since BuildAsSubstructure is recursive, an Effect implementation should carry the designated BuildAsSubstructure as a prototype pattern to call substructure Effect implementation leafs.
- **Flaw 8 and 9 - singleton link and mirror list, and no flyweight mirror creation:** To make mirror and link creation more flexible, we approach the problem from two sides. Firstly, the Network class should be segmented more clearly to concentrate on handing through mirror requests, since their functionality is limited on changing the list of mirrors and returning requests. To reduce the growing complexity of Network, we have created a MirrorCursor class after a singleton pattern, that handles and manages all mirrors on the network (Flaw 8). We suggest the same procedure for links, by managing links on the TopologyStrategy and recursive substructure instead, because links are the result of an applied TopologyStrategy. The Network class should manage what does exist, but the TopologyStrategy should rule, how the resource is virtually connected. This concept can be understood by comparing the physical existence of mirrors, that are virtually fully connected with unlimited network bandwidth, to the virtual nature of links representing the actual activated and limited use of chosen connections. Mirrors are computers in a real world holding data and network connections are a concept about enabling transmission/transport processes in between them. In an ideal case we would manage all links and all mirrors in the Network class via function calls to the TopologyStrategy, to sync their existence via a flyweight pattern to the recursive substructures, so that each substructure is aware of storing/registering the mirrors and links recursively (Flaw 9). In our implementation we focused on the principle of isolating the functionality of mirror creation by using a MirrorCursor class and registering MirrorNode objects to substructures to demonstrate the architecture. Mirrors can be created and iterated over to get the next mirror. In case a mirror is deleted, we apply the rule that the iterator on the MirrorCursor skips stopping mirrors to hold consistency with the id structure. New mirrors are created and appended to the end of the list.
- **Flaw 10 - no dataset split and fault recovery simulation:** To support extended simulations and probing, the Network class should be extended by a TransmissionProcess singleton, that is available on every substructure as an observer pattern to register and evaluate each transmission process from a set of links. We have not implemented this principle due to a limit of time and resources.
- **Flaw 11 - master effector:** The Effector should be split into multiple specialized effectors, that derive from the same interface Effector, to support different logging and messaging routines on each implementation. This is due to different effects or registered faults may require actions of different significance by a DevOps team adapting the project to their needs. In most cases, mature software will try to automate logging and software fault tolerance.

4 Concept

This chapter takes action on the results of the "Problem Analysis and Requirements" section. We will plan and describe all major tasks and sub-tasks in this section.

4.1 Concept - Plan and Contribute examples using LRDM Simulator

Before estimating a design and a plan to prove results, we need to go into the details of the existing project design to gain design understanding for the transformation steps suggestions performed in chapter 3. For the design we used clues of the first approaches of self stabilizing systems, that are researched far before the year of 2005, resembling local network "remote" data mirroring and self-stabilizing distributed file system [DK05], contradicting the claim of novelty by [GBS24].

4.1.1 Network

Network is the central graph and state container that coordinates mirrors, links, and the execution of actions and effects. It exposes a time-stepped or event-driven update hook that advances the simulation and mediates message propagation across links. Key operations include Network, getCurrentTimeStep, getProps, registerProbe, with further methods documented in the UML. Important state includes props: Properties, mirrors: List, links: Set, probes: List, which defines identity and configuration. In terms of relationships, it associates with Mirror; associates with Link; associates with Effector; associates with Probe. From the perspective of other components, it is used by Probe; is associated with TimedRDMSim. Design-wise it leverages patterns such as Mediator to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.4, subsection 4.1.6, subsection 4.1.11, subsection 4.1.14, subsection 4.1.2.

4.1.2 TimedRDMSim

TimedRDMSim is a core participant in the LRDM simulator supporting construction, execution, or observation of experiments. Its responsibilities are cohesive and align with the modular strategy-based architecture. Key operations include TimedRDMSim, TimedRDMSim, setHeadless, getProps, with further methods documented in the UML. Important state includes DEFAULT_CONFIG_NAME: String, log: Logger, lastTimeStep: int, network: Network, which defines identity and configuration. In terms of relationships, it associates with Network; associates with GraphVisualization. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.1, subsection 4.1.13.

4.1.3 DataPackage

DataPackage is a core participant in the LRDM simulator supporting construction, execution, or observation of experiments. Its responsibilities are cohesive and align with the modular strategy-based architecture. Key operations include getFileSize, isLoaded, with further methods documented in the UML. Important state includes fileSize: final int, received: int, getReceived: int, increaseReceived: void, which defines identity and configuration. From the perspective of other components, it is associated with Mirror. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.4.

4.1.4 Mirror

Mirror models a replica node that stores state and participates in remote data mirroring flows. It reacts to actions and triggers effects that affect its local state or the adjacent links. Key operations include Mirror, getState, setRoot, isRoot, with further methods documented in the UML. Important state includes state: State enum - DOWN, STARTING, UP, READY, HASDATA, STOPPING, STOPPED, isRoot: boolean, id: final int, state: State = State.Down, which defines identity and configuration. In terms of relationships, it associates with DataPackage. From the perspective of other components, it associates with Network. Design-wise it leverages patterns such as Entity-Component to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.3, subsection 4.1.1.

4.1.5 MirrorCursor

MirrorCursor in Figure A.18 and Figure A.19 provides an iterator-like traversal over mirrors and their planned adjacencies to support incremental construction and measurement. It abstracts stepping through the planning graph or realized network, yielding mirrors in a policy-defined order (e.g., BFS layers or degree deficits). The cursor can be used by strategies and probes to gather metrics or to stage batched link updates in a deterministic sequence. Key operations include getMirrors, getMirrorsSortedById, getMirrorStream, getNumMirrors, with further methods documented in the source. Its state typically stores position markers and a reference to subsection 4.1.1 or a StructureNode plan. In relationships, it collaborates with Mirror-based nodes (subsection 4.2.2) and construction strategies. Design-wise it follows the Iterator pattern with optional filtering and ordering predicates. This makes it valuable for reproducible growth and evaluation passes.

4.1.6 Link

Link represents a network connection with latency, capacity, and failure semantics. It serves as the carrier for flows/streams and is the locus of bandwidth and queue dynamics. Key operations include Link, getID, getState, getCurrentBandwidth, with further methods documented in the UML. Important state includes id: int, State: enum{INACTIVE,ACTIVE,CLOSED}, state: State, source: Mirror, which defines identity and configuration. From the perspective of other components, it associates with Network. Design-wise it leverages patterns such as Entity-Component to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.1.

4.1.7 Action

Action is a command object scheduled at specific time steps to enact changes on the network. Each action holds an effect prototype, identifiers, and timing metadata to ensure deterministic execution. Key operations include Action, getEffect, with further methods documented in the UML. Important state includes id: int, time: int, effect: Effect, network: Network, which defines identity and configuration. From the perspective of other components, it is a base type for MirrorChange; is a base type for TargetLinkChange; is a base type for TopologyChange. Design-wise it leverages patterns such as Command to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.8, subsection 4.1.9, subsection 4.1.10.

4.1.8 MirrorChange

MirrorChange models a replica node that stores state and participates in remote data mirroring flows. It reacts to actions and triggers effects that affect its local state or the adjacent links. Key operations include getNumMirrors, with further methods documented in the UML. Important state includes numMirrors: final int, which defines identity and configuration. In terms of relationships, it inherits from or specializes

Action. From the perspective of other components, it associates with Effector. Design-wise it leverages patterns such as Entity-Component to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.7, subsection 4.1.11.

4.1.9 TargetLinkChange

TargetLinkChange represents a network connection with latency, capacity, and failure semantics. It serves as the carrier for flows/streams and is the locus of bandwidth and queue dynamics. Key operations include `getNewLinksPerMirror`, with further methods documented in the UML. Important state includes `newLinksPerMirror`: `int`, which defines identity and configuration. In terms of relationships, it inherits from or specializes Action. Design-wise it leverages patterns such as Entity-Component to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.7.

4.1.10 TopologyChange

TopologyChange provides a strategy for constructing or mutating the network graph under explicit invariants such as connectivity, degree bounds, and annotated link properties. It encapsulates topology-specific rules so that the rest of the simulator can remain agnostic of concrete graph shapes. Key operations include `getNewTopology`, with further methods documented in the UML. Important state includes `newTopology`: `TopologyStrategy`, which defines identity and configuration. In terms of relationships, it inherits from or specializes Action. From the perspective of other components, it associates with Effector. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.7, subsection 4.1.11.

4.1.11 Effector

Effector captures the observable outcome of executing an action, such as link failures, deliveries, or policy changes. Effects are designed to be composable and traceable to support evaluation and debugging. Key operations include `Effector`, `setMirrors`, `setStrategy`, `setTargetLinksPerMirror`, with further methods documented in the UML. Important state includes `n`: final network, `setMirrorChanges`: `Map`, `setStrategyChanges`: `Map`, `setTargetedLinkChanges`: `Map`, which defines identity and configuration. In terms of relationships, it associates with `MirrorChange`; associates with `TopologyChange`. From the perspective of other components, it is associated with `Network`. Design-wise it leverages patterns such as Command to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.8, subsection 4.1.10, subsection 4.1.1.

4.1.12 VisualizationStrategy

VisualizationStrategy is a strategy that renders or exports graph state snapshots for debugging and presentation. By separating visualization from simulation, it preserves determinism and testability. Key operations include `init`, `updateGraph`, with further methods documented in the UML. The class carries only essential state in the UML; additional configuration is supplied via constructors or configs. Design-wise it leverages patterns such as Strategy to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios.

4.1.13 GraphVisualization

GraphVisualization is a strategy that renders or exports graph state snapshots for debugging and presentation. By separating visualization from simulation, it preserves determinism and testability. Key

operations include `createUI`, `updateTimeStep`, `updateLinks`, `updateUIClassOfEdge`, with further methods documented in the UML. Important state includes `WIDTH`: int, `HEIGHT`: int, `UI_CLASS`: String, `BANDWIDTH`: String, which defines identity and configuration. From the perspective of other components, it is associated with `TimedRDMSim`. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.2.

4.1.14 Probe

Probe is an observational component that samples metrics from links or mirrors without altering the system's behavior. It enables repeatable measurements and the production of evaluation artifacts. Key operations include `Probe`, `update`, `print`, with further methods documented in the UML. Important state includes `n`: Network, which defines identity and configuration. In terms of relationships, it depends on Network. From the perspective of other components, it is a base type for `LinkProbe`; is a base type for `MirrorProbe`; associates with Network. Design-wise it leverages patterns such as Observer to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.1, subsection 4.1.15, subsection 4.1.16.

4.1.15 LinkProbe

LinkProbe is an observational component that samples metrics from links or mirrors without altering the system's behavior. It enables repeatable measurements and the production of evaluation artifacts. Key operations include `LinkProbe`, `getActiveLinkMetric`, `getLinkRatio`, with further methods documented in the UML. Important state includes `ratio`: double, `getLinks`: Set, which defines identity and configuration. In terms of relationships, it inherits from or specializes Probe. Design-wise it leverages patterns such as Observer, Entity-Component to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.14.

4.1.16 MirrorProbe

MirrorProbe models a replica node that stores state and participates in remote data mirroring flows. It reacts to actions and triggers effects that affect its local state or the adjacent links. Key operations include `MirrorProbe`, `getNumMirrors`, `getNumReadyMirrors`, `getNumTargetMirrors`, with further methods documented in the UML. Important state includes `mirrorRatio`: double, `getMirrors`: List, `getMirrorRatio`: double, which defines identity and configuration. In terms of relationships, it inherits from or specializes Probe. Design-wise it leverages patterns such as Observer, Entity-Component to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.14.

4.1.17 TopologyStrategy

TopologyStrategy Figure A.17 provides a strategy for constructing or mutating the network graph under explicit invariants such as connectivity, degree bounds, and annotated link properties. It encapsulates topology-specific rules mutating StructureNode plans into Mirror node facts, so that the rest of the simulator can remain agnostic of concrete graph shapes. Key operations include `initNetwork`, `restartNetwork`, `handleAddNewMirrors`, `getNumTargetLinks`, with further methods documented in the UML. The class carries only essential state in the UML; additional configuration is supplied via constructors or configs. From the perspective of other components, it is a base type for `BalancedTreeTopologyStrategy`; is a base type for `FullyConnectedTopology`; is a base type for `NConnectedTopology`. Design-wise it leverages patterns such as Strategy to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.18, subsection 4.1.19, subsection 4.1.20.

4.1.18 **BalancedTreeTopologyStrategy**

BalancedTreeTopologyStrategy provides a strategy for constructing or mutating the network graph under explicit invariants such as connectivity, degree bounds, and annotated link properties. It encapsulates topology-specific rules so that the rest of the simulator can remain agnostic of concrete graph shapes. Key operations include createTree, createAndLinkChildren, connect, with further methods documented in the UML. The class carries only essential state in the UML; additional configuration is supplied via constructors or configs. In terms of relationships, it inherits from or specializes TopologyStrategy. Design-wise it leverages patterns such as Strategy to keep concerns separable and replaceable. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.17.

4.1.19 **FullyConnectedTopology**

FullyConnectedTopology provides a strategy for constructing or mutating the network graph under explicit invariants such as connectivity, degree bounds, and annotated link properties. It encapsulates topology-specific rules so that the rest of the simulator can remain agnostic of concrete graph shapes. Key operations include connectedMirrorToAllOthers, with further methods documented in the UML. Important state includes connected(Mirror m1, Mirror m2, which defines identity and configuration. In terms of relationships, it inherits from or specializes TopologyStrategy. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.17.

4.1.20 **NConnectedTopology**

NConnectedTopology provides a strategy for constructing or mutating the network graph under explicit invariants such as connectivity, degree bounds, and annotated link properties. It encapsulates topology-specific rules so that the rest of the simulator can remain agnostic of concrete graph shapes. Key operations include getTargetMirror, connectedToOtherMirrors, with further methods documented in the UML. The class carries only essential state in the UML; additional configuration is supplied via constructors or configs. In terms of relationships, it inherits from or specializes TopologyStrategy. This makes the class fit for both synthetic simulations and, later, measurement-driven scenarios. See also: subsection 4.1.17.

4.2 **Concept - Extension concept and reproducibility**

In the following section we will categorize the new parts of implemented architecture and explain changes made to the initial project. To enable plan-graphs we introduce multiple node wrapper for the initial mirror class. These nodes are linked and tainted in multiple node type flavors to prepare the target graph architecture and target graph substructure node roles. The new class goal of BuildAsSubstructure is to transform the planned architecture, constructed into a StructureNode type, by any derivation of TopologyStrategy, into "physical" links between dynamically managed mirrors. The original mirrors are wrapped inside MirrorNode, so that the BuildAsSubstructure translates the current mirrors and links from the current into the desired state described by the constellation of any MirrorNode derivation. Every StructureNode introduces a leader substructure role and side roles for the overlay of other topology structures. Any TopologyStrategy is being designed so that it carries at least one head node on the planning side, that is a substructure root. The difference between a head node and a root node is the planning side with the help of StructureNode vs the realization side with the help of Mirror and Link. Since the state of a structure is elevated by a comparison from a desired state against a current state, I call this design pattern an "elevator" design pattern. It may be useful to the fields of software fault tolerance and rejuvenation, since its first footprints already exist distributed on kubernetes datacenters or self healing graphs [TK11]. An elevator pattern is defined as a dynamic and proactive memento code pattern that interactively strives to reach a desired state, defined by at least one strategy pattern. The difference be-

tween an elevator code pattern is that the code pattern only resides input, output and code on a single machine.

4.2.1 StructureNode

StructureNode Figure A.16 is the abstract base for the *planning graph*: a graph of structural nodes that plan how mirrors and links should be organized before the network is actually mutated. It captures parent/child relations and provides traversal, search, and evaluation hooks to reason about degrees, depth, and feasible insertions independent of runtime link objects. Subclasses materialize specific structural shapes (e.g., trees, stars, lines) and can be composed to describe composite layouts. By separating planning from execution, it lets topology strategies simulate or score alternatives without touching the live network. The class anchors common utilities for counting nodes, computing expected vs. actual degree, and deriving targets for link creation. In relationships, it is a base type for all planning nodes (see also subsection 4.2.3, subsection 4.2.2). Design-wise it applies Composite for hierarchical structure and Template Method for shape-specific evaluation points. This makes it suitable for search- and policy-guided construction as well as reproducible topology bootstrapping.

4.2.2 MirrorNode

MirrorNode Figure A.16 adapts the legacy subsection 4.1.4 into the planning graph by wrapping a Mirror inside a StructureNode interface. It exposes evaluation and bookkeeping functions specific to mirrors—such as retrieving the wrapped mirror identifier/state, tracking target link counts, and producing planned adjacency sets. During execution, a MirrorNode can be asked to realize planned links into concrete Link objects against the subsection 4.1.1. It therefore bridges plan-time structure with run-time entities and enables per-mirror metrics (e.g., utilization and readiness) to influence placement decisions. In relationships, it inherits from subsection 4.2.1 and is specialized by tree/star/line variants (see also subsection 4.2.3, subsection 4.2.4). Design-wise it leverages Adapter (wrapping Mirror) and Command hooks to emit link-creation intents. This combination allows uniform reasoning across structural nodes regardless of whether a real Mirror already exists or will be allocated.

4.2.3 TreeMirrorNode

TreeMirrorNode Figure A.16 is a StructureNode specialization that organizes mirrors as a rooted tree and offers tree-specific operations such as child insertion, level-wise traversal, and depth/branching computations. It maintains the expected vs. actual number of children per node and provides methods to locate optimal insertion points according to target degree constraints. When realized, it guides the creation of Links along parent–child relations to enforce a tree-shaped backbone. It cooperates closely with subsection 4.2.15 and subsection 4.2.11 to balance the tree during growth. In relationships, it inherits from subsection 4.2.1 and is the base for subsection 4.2.4 and subsection 4.2.5. Design-wise it applies Composite and Strategy-friendly hooks to compute placements under degree and depth constraints.

4.2.4 BalancedTreeMirrorNode

BalancedTreeMirrorNode Figure A.16 refines TreeMirrorNode to approximate a breadth-first balanced tree where each node aims for an optimal target degree. It tracks expected vs. actual child counts and exposes evaluators that penalize skew while favoring shallow insertions. Topology strategies can query the node to pick the next insertion location that keeps the tree balanced under the given links-per-node budget. In relationships, it derives from subsection 4.2.3 and is consumed by subsection 4.2.15. The design emphasizes Heuristic Evaluation and Local Search combined with Composite structure. This yields predictable expansion behavior and supports deterministic scenario reproduction.

4.2.5 DepthLimitTreeMirrorNode

DepthLimitTreeMirrorNode Figure A.16 is a planning node that augments tree nodes with a maximum-depth constraint and evaluators for level saturation. It tracks the current depth, expected children, and admissibility for further insertions, exposing queries to steer balanced, shallow growth. When realized, it induces parent–child Links only where depth bounds are satisfied. Key operations are documented in the source. Its state includes depth counters and target degree configuration. In relationships, it inherits from subsection 4.2.3 and is used by subsection 4.2.16. Design-wise it combines Composite traversal with lightweight constraint checks. This enables repeatable, bounded-diameter constructions.

4.2.6 FullyConnectedMirrorNode

FullyConnectedMirrorNode Figure A.16 models a planning node for fully connected layouts where each mirror is adjacent to all others (subject to feasibility checks). It provides validity checks for the structure and utilities to enumerate the complete adjacency set required for realization. The node can be used to derive the exact set of Links to be created by an effector in one pass. In relationships, it derives from subsection 4.2.2 and is used by subsection 4.1.19. Design-wise it is straightforward and focuses on correctness and efficient enumeration rather than search.

4.2.7 LineMirrorNode

LineMirrorNode Figure A.16 represents mirrors arranged in a line (path) and provides helpers to determine predecessor/successor targets when growing the path. It keeps the structure simple and low-degree, suitable for scenarios requiring minimal link fan-out. Realization translates planned adjacency into a chain of Links with predictable bandwidth footprint. In relationships, it derives from subsection 4.2.2 and is consumed by subsection 4.2.12. Design-wise it supports Greedy Insertions along the current tail while preserving Composite compatibility.

4.2.8 NConnectedMirrorNode

NConnectedMirrorNode Figure A.16 plans an N-connected layout where each mirror aims for N neighbors according to a distribution policy. It exposes evaluators to detect under- and over-connected nodes and utilities to propose valid new adjacencies. At realization time, it can emit the required Link additions per mirror to meet the N-target. In relationships, it extends subsection 4.2.2 and supports subsection 4.1.20. Design-wise it mixes Constraint Satisfaction with local heuristics.

4.2.9 StarMirrorNode

StarMirrorNode Figure A.16 plans a star-shaped structure with a central hub and peripheral leaves. It provides functions to designate and verify the hub and to enumerate spoke endpoints for realization. The node can also support multi-star compositions when combined with substructure building strategies. In relationships, it derives from subsection 4.2.2 and is used by subsection 4.1.17. Design-wise it uses Role Assignment (hub/leaf) on top of the Composite base.

4.2.10 BuildAsSubstructure

BuildAsSubstructure is a specialization of subsection 4.1.17 that **builds** a topology via a planning graph first and then realizes it against the subsection 4.1.1. It introduces the two-phase workflow ‘buildStructure(totalNodes)’ → ‘buildAndUpdateLinks(network, structure)’, allowing strategies to remain pure and testable. Common utilities include mapping from StructureNode plans to concrete Link additions and consistency checks between plan and realized network. In relationships, it is a base for

strategies such as subsection 4.2.15 and collaborates with subsection 4.2.18 and the MirrorNode hierarchy. Design-wise it applies Template Method for the plan/realize phases and promotes Separation of Concerns.

4.2.11 DepthInsertionStrategy

DepthInsertionStrategy provides heuristics to select insertion points in a tree-shaped planning graph with depth-awareness. It scores candidate nodes by depth and residual capacity (expected vs. actual children) to keep trees shallow and balanced. The strategy is called from subsection 4.2.15 during growth to choose the next parent for a new mirror. Design-wise it encapsulates a Policy object that can be swapped or tuned independently from the topology algorithm.

4.2.12 LineTopologyStrategy

LineTopologyStrategy constructs line (path) topologies using the planning graph API and realizes them into the network. It leverages subsection 4.2.7 to keep degree at most two and exposes initialization and incremental growth routines. The strategy is suitable for baseline experiments and controlled bandwidth consumption. Design-wise it is an instance of Strategy running on top of subsection 4.2.10.

4.2.13 StarTopologyStrategy

StarTopologyStrategy builds a star-shaped layout with a single hub and multiple leaves using the planning graph API. The strategy chooses a hub, plans spoke attachments as subsection 4.2.9 edges, and then realizes the plan as Links in the subsection 4.1.1. It ensures hub capacity constraints are respected and can partition into multi-star substructures when the number of leaves exceeds thresholds. Key operations include buildStructure, addNodesToStructure, removeNodesFromStructure, validateTopology, with further methods documented in the source. Important state includes the hub selector policy and degree targets per mirror. In relationships, it extends subsection 4.2.10 and consumes subsection 4.2.9. Design-wise it is a Strategy over a Composite planning graph with clear separation between planning and realization. This is useful for centralized aggregation scenarios and baseline throughput tests.

4.2.14 SnowflakeTopologyStrategy

SnowflakeTopologyStrategy subsection 4.2.14 constructs the snowflake pattern by composing rings and hub-spoke bridges either directly to the main pattern or via other topologies in a two-phase process: first a planning graph (based on subsection 4.2.1) is synthesized, then it is realized into concrete Links on the subsection 4.1.1. It cooperates with subsection 4.2.17 to validate parameters (rings, ring sizes, bridge ratios) prior to construction and fails fast on invalid inputs. During planning it selects insertion points on rings, designates hubs, and determines bridge endpoints so as to satisfy degree and distance constraints. The realization stage applies the plan via the effector, producing deterministic link sets suitable for evaluation. Key operations include topics like SnowflakeProperties, MirrorDistributionResult, calculateSnowflakeDistribution, validateParameters, with further methods documented in the source. Important state comprises topology parameters and references to planning nodes and the effector. In relationships, it extends subsection 4.2.10 and uses node classes of the StructureNode hierarchy. Design-wise it applies Strategy + Template Method (plan/realize) and Validator, enabling reproducible, parameterized scenarios. This makes it appropriate for experiments that compare ring-dominant versus bridge-dominant structures under varying loads.

4.2.15 TreeTopologyStrategy

TreeTopologyStrategy builds tree-shaped networks using subsection 4.2.3 and subsection 4.2.11 to control branching and depth. It follows a breadth-first construction policy by default but allows targeted

insertions where constraints require. The algorithm first emits a pure `StructureNode` plan and then realizes `Links` consistently in one or more batches. In relationships, it extends subsection 4.2.10 and uses specializations such as subsection 4.2.4 or subsection 4.2.5. Design-wise: Strategy + Template Method (plan/realize) with clear separation of concerns.

4.2.16 DepthLimitTreeTopologyStrategy

`DepthLimitTreeTopologyStrategy` constructs trees while enforcing a strict bound on depth to control path lengths and latency. It plans with subsection 4.2.5 nodes and selects parents using depth-aware policies to avoid over-deep branches. The algorithm emits a planning tree that respects both target degree and maximum depth, then realizes `Links` consistently. Key operations include `buildStructure`, `addNodesToStructure`, `removeNodesFromStructure`, `findShallowEmptyNodeWithMaximumChildren`, with further methods documented in the source. Its configuration contains the depth limit and the target children per node. In relationships, it extends subsection 4.2.10 and cooperates with subsection 4.2.11 where applicable. Design-wise it combines Strategy with explicit constraint checks to keep construction predictable. This makes it suitable for latency-sensitive evaluation setups.

4.2.17 SnowflakeTopologyValidator

`SnowflakeTopologyValidator` centralizes parameter and feasibility checks for the snowflake pattern (concentric rings + bridges). It validates ring counts, minimal mirrors per ring, and bridge distances/ratios and provides targeted exceptions with precise diagnostics. Strategies like subsection 4.1.17 call the validator before constructing the planning graph to fail fast on invalid scenarios. Design-wise it implements the validator pattern and keeps strategies lean and focused on construction.

4.2.18 SubstructureFactory

`SubstructureFactory` is a small factory that instantiates topology variants (e.g., subsection 4.2.2, subsection 4.2.3) based on topology intents. It hides object-creation concerns and ensures consistent initialization (IDs, target links, depth limits) across strategies. Design-wise it uses the Factory Method to decouple construction from the strategies that consume the nodes, depending on a creation pattern set on a simulation run.

4.3 Concept - We present: A snowflake pattern

With constructing a snowflake graph pattern that can be seen on Figure A.24, we desire to show the results of extending fault tolerant simulation of mirror substructures among cascading graph organizations. Our goal is to give proof of our research questions and analysis assumptions by implementing a compound graph pattern, simulating more complex behavior, including mirror deletion, ad hoc creation, crashes, repair and topology transformation using the same set or subset of mirrors.

The *snowflake* is realized as a composition of ring segments and hub-spoke bridges. On the planning side, we synthesize a `StructureNode` plan (subsection 4.2.1) driven by subsection 4.2.14. A dedicated validator (subsection 4.2.17) checks ring counts, minimum nodes per ring, and bridge distances/ratios to fail fast on invalid parameterizations. Each live *Mirror* is wrapped in a subsection 4.2.2, so that the plan can be expressed independent of runtime allocation details and later transformed into concrete `Links` by subsection 4.2.10. This establishes a strict two-phase flow: *plan* (immutable target shape) and *realize* (idempotent reconciliation on subsection 4.1.1).

In evaluation, the snowflake demonstrated three effects on the measurements on Figure A.21, Figure A.22 and Figure A.23 that support our assumptions. First, determinism and reproducibility: the plan is a stable artifact that can be re-applied to converge the system after faults (crash, removal, or replacement),

making the “elevator” pattern explicit (current→desired). Second, graceful degradation and repair: selective edge removals on rings versus bridges alter diameter and connectivity in controlled, measurable ways; targeted actions (subsection 4.1.7) drive the subsection 4.1.11 to restore reachability with bounded disruption. Third, observability and comparability: probes (subsection 4.1.14) collect per-ring and per-bridge metrics, which lets us compare snowflake instances with equal node budgets against baselines (trees, lines) with the same traffic.

Operationally, the strategy integrates with subsection 4.1.2 yet is not tied to a single heartbeat; BuildAsSubstructure can realize the plan either step-wise or event-driven. This enables energy-aware experiments (update on demand) without sacrificing convergence guarantees. Overall, the snowflake case confirms that planning with StructureNodes and realizing via BuildAsSubstructure provides a repeatable, fault-tolerant path from a complex desired topology to a stable running configuration—exactly the capability our research questions targeted.

4.4 Concept - Architecture, Design and Effective project transformation

The project transformation replaced monolithic, simulation-centric control with a modular, desired-state architecture. We separated *planning* from *realization*: topology logic now lives behind subsection 4.1.17, producing StructureNode plans (subsection 4.2.1) that are translated by subsection 4.2.10 into concrete Mirror/Link updates on subsection 4.1.1. Wrapping the legacy Mirror in subsection 4.2.2 standardized evaluation and targeting across all strategies and enabled head (planning) versus root (realization) roles to be expressed unambiguously. This implements the proposed “elevator” design pattern: compare the current runtime graph to the desired plan and converge by applying actions (subsection 4.1.7) through the effector (subsection 4.1.11).

Effectiveness was observed along four axes that were repeatedly evaluated while extending the code base. (1) **Adoptability**: by removing a hardwired event/signal source and exposing strategy and timing as interchangeable, we aligned with industrial constraints on cost and integration—strategies can be swapped without refactoring call-sites. (2) **Determinism & reproducibility**: the plan/realize split yields idempotent transformations; experiments can be serialized as plans and re-run to validate results. (3) **Fault tolerance**: actions capture repair, insertion, and migration; BuildAsSubstructure ensures local repairs remain consistent with the global plan, which shortens recovery paths and avoids cascading side effects. (4) **Observability**: probes (subsection 4.1.14) and cursors (e.g., subsection 4.1.5) support layered measurements (per node, per substructure, per topology), making regressions and improvements visible. Design-wise, we regularized ownership and flyweight use around Network (single stores for Mirrors and Links), moved topology-specific object creation to strategies/factories, and introduced validators to keep strategies lean. The GUI/visual layer (subsection 4.1.13) remained decoupled and is now driven by stable events rather than internal state coupling. Timed operation is preserved via subsection 4.1.2, but realization can also be driven by external events for energy-aware runs.

In summary, the transformation delivers a strategy-first, planning-graph-based simulator that elevates any supported topology from target design to running system with predictable effort. The resulting architecture is easier to test, easier to extend (new *TopologyStrategy* variants), and better suited for fault-tolerance and rejuvenation studies where convergence to a desired state is the primary concern.

What we found on the way: Our elevator pattern implements a desired-state reconciliation loop specialized for network topology: a *StructureNode* plan (desired) is continuously compared with the live *Network* (current), and *BuildAsSubstructure* applies concrete *Link/Mirror* actions until convergence—in direct analogy to the reconcile loops of Kubernetes controllers and Operators [Kub24b, cnc21, Kub24a]. We follow an architecture-based self-adaptation style (Rainbow, MAPE-K), maintaining a clean separation between planning (StructureNode, strategies) and execution (Link effects), which makes safety and convergence properties easier to reason about [CGS08, IBM05]. Computation of adaptation steps

adopts a graph-transformation perspective: mutations on the StructureNode planning graph correspond to graph-rewrite rules that are realized as Link updates in the running system [TK12, WF02]. From a control-theoretic angle, the elevator acts like a supervisor that drives the system toward a legal/goal behavior and—in the presence of transient faults—exhibits self-stabilizing convergence [RW89, Dol00]. When aging and configuration drift accumulate, the same mechanism supports rejuvenation-like repairs (selective reconfiguration rather than full restarts), which aligns with software aging and rejuvenation (SAR) evidence [CNPR14, Han10].

Effort Accounting and Schedule (9 + 4 Weeks)

The total implementation time was **9 weeks** full time; the writing phase consumed **4 weeks**. The following breakdown reconciles per-task effort via planned parallelization (overlap) where individual task sums exceed the wall-clock window. There is an in between state that was totally discarded: using a builder design pattern and the TopologyStrategy as a director, did not lead to good adaptability, leading to the return of a strategy pattern. Therefore 2 were ineffectively used on prototyping an integration using the TreeTopologyStrategy as a test subject. Else the implementation plan was set at the beginning of the project and executed roughly within bounds.

Table 4.1: Implementation effort breakdown and overlap-aware schedule

ID	Task	Effort [w]	Window	Notes
A	New core structure (exploration)	3.0	W1–W3	establishes interfaces, invariants
B	BuildAsSubstructure (as TopologyStrategy)	1.5	W2–W3	overlaps with A in W2–W3
C1	TreeTopologyStrategy: BalancedTree	1.0	W3–W4	adapted BuildAsSubstructure
C2	TreeTopologyStrategy: DepthLimitTree	1.0	W3–W4	parallel to C1
D	FullyConnectedTopology	0.5	W4	micro-case, shares W4
E	NConnectedTopology	1.0	W4–W5	baseline example
F	StarTopologyStrategy	1.0	W5	parallel to E tail
G	SnowflakeTopologyStrategy	3.0	W6–W8	new pattern, validation + docs
H	Example packaging, tests, runner	1.0	W8–W9	hardening + reproducibility
Wall-clock window			W1–W9	overlaps kept within 9 weeks

Writing Phase (4 weeks). The written part (this document) took **4 weeks**, including time-boxed estimation, figure generation from reproducible runs, and integration of architectural rationale. Sections covered: concept chapters (this), evaluation preparation, and the transformation narrative that links back to Sec. 3.4.

Traceability. Each task maps to requirements examined in Ch. 3 (*Problem Analysis and Requirements*). Where possible, we link commits to task IDs (A–H) for auditability.

5 Evaluation, Results and Conclusion

This chapter reflects and documents all the results of completed tasks and sub-tasks, using the research hypothesis to answer the research questions. It also summarizes the takeaways of the research questions and project results.

5.1 Findings from the three measurement plots (snowflake run).

Bandwidth over Time. The blue series stays essentially flat near **0%** for the whole run (global mean **0.082%**), with only short blips below about **2%** peaking at **1.8%** around steps ~ 120 – 160 ; the green limit line at **40%** is never approached. This indicates the experiment is *not bandwidth-bound*. Any performance changes we observe are therefore dominated by structural/latency effects rather than raw link capacity.

Active Links. Two early bursts reach roughly **75–78%** of all links (peak **78.2%** at step **79**, with the first spike around ~ 35), which matches the expected plan-application/reconfiguration windows of the elevator pattern. After that, the system settles into a long plateau between about **5–9%** active links (steps ~ 120 – 360 ; window mean **12.33%**, $\sigma \approx 2.36$), followed by a gradual decline and eventual quiescence close to **0–4%** beyond ~ 420 (late-window mean **3.44%**). In other words, structural convergence requires only short, bounded bursts of activity; steady-state operation is *sparse* in terms of active links.

Time to Write. During the two early bursts, time-to-write spikes to about **100%** of the tick budget (temporary stalls), then dips briefly to **32–35%** near ~ 60 , and stabilizes for the long middle window around **85–90%** (steps ~ 120 – 360 ; window mean **86.55%**) before rising again to about **92%** toward the end (late window ≥ 420 : mean **92.0%**). Given the near-zero bandwidth usage and low active-link fraction, this points to an end-to-end *latency/diameter* bottleneck (ack rounds, hop count, coordination), rather than capacity saturation.

5.2 Interpretation per research question

5.2.1 RQ1: How does a k -connected backbone with periodic corona attachments affect failover behavior and cut sizes under node/link failures?

RQ1 (Backbone+corona failover). Sustained operation with low active-link usage and stable time-to-write after the bursts (plateau/late means **12.33%/3.44%** active links and **86.55%/92%** time-to-write) suggests resilient behavior of the composed backbone with periodic attachments. To close RQ1 quantitatively, compute vertex/edge connectivity and minimum cuts on exported snapshots corresponding to the plateau windows.

5.2.2 RQ2: Can run-time reconfiguration (adding/removing mirrors/links) maintain service continuity without global restart under WAN-like latencies/capacities? What needs to be changed on the architecture and implementation to support this feature?

RQ2 (Run-time reconfiguration). The two short, high-activity spikes (up to **78.2%** active links at step **79**) with immediate recovery to the **5–9%** plateau by ~ 120 demonstrate live reconfiguration without global restart: the system applies the plan, momentarily stalls writes (time-to-write $\approx 100\%$), and resumes with stable readiness and sparse link activity. This is the intended elevator behavior.

5.2.3 RQ3: Which metrics best capture LRDM objectives (consistency, convergence, utilization) across topology variants, transformations and fusion of structure patterns?

RQ3 (Metrics for LRDM objectives). The trio of metrics is sufficient to separate concerns: bandwidth is flat (capacity not the limit), active links quantify structural “effort” to converge (spikes → plateau → quiescence), and time-to-write captures user-visible latency (middle-window mean **86.55%**, late-window **92%**). Together they evidence convergence with clear instrumentation; complement them with readiness thresholds (e.g., steps to 0.85/0.90) for a full convergence KPI.

5.2.4 RQ4: Under bandwidth constraints, which composed backbone/attachment settings strike the best trade-off between link usage and time-to-write across the network?

RQ4 (Trade-offs under bandwidth constraints). Even with *very low* bandwidth usage (global mean **0.082%**, max **1.8%**) and *low* active-link fractions (plateau mean **12.33%**), time-to-write remains high (≈ 85 – 90% mid-run, **92%** late). The snowflake’s hierarchical ring+bridge shape keeps utilization costs low but increases path length. The actionable knobs are therefore structural: increase k , add/shorten bridges between rings, or reduce attachment periodicity to shrink diameter (expect time-to-write to drop without meaningfully raising bandwidth).

5.2.5 RQ5: Which implementation strategy (design patterns, architecture design suggestions, custom core vs. general graph frameworks) supports recursive LRDM structures best? How can we still maintain mirror stability and tick semantics to sample the use of the given framework [Göt23, GBS24]? Which design patterns are most effective to yield results?

RQ5 (Implementation strategy) and RQ6 (Suitability). The plots reflect a clean separation of planning vs. realization: brief, bounded reconciliation bursts followed by sparse steady-state activity (late mean **3.44%** active links). That supports Strategy+Command/Effector as the right architectural choice (RQ5) and provides concrete criteria (ID stability, fixed-tick hooks, predictable per-tick mutation cost) to assess a graph framework or a custom core (RQ6).

5.2.6 RQ6: Which Java graph framework (JGraphT, GraphStream, Guava `common.graph`) or a custom adjacency structure is most suitable to extend *LRDMSimulator* [Göt23], judged by estimated implementation effort, ID stability and determinism, predictable per-tick mutation costs, fixed-tick hooks/instrumentation, lean dependency footprint, and adapter/visualization options?

Since all frameworks, compared to the already implemented work of [Göt] create more overhead than use to the project, we decided to go on our own. The goal is to show software design principles on the project, before more research can be achieved on the topic from the insights of this work, which is more a design and costly implementation problem, than a concept problem.

5.3 Hypotheses mapping and evaluation (snowflake run).

Unless noted otherwise, the evidence refers to the snowflake time window where we observed: peak active links **78.2%** at step **79**, plateau mean active links **12.33%** ($\sigma \approx 2.36$) for steps ~ 120 – 360 , late mean **3.44%** for ≥ 420 , bandwidth mean **0.082%** (max **1.8%**), plateau mean time-to-write **86.55%**, late

mean **92.0%**, and two short reconfiguration spikes with time-to-write at **100%**. Where we reference readiness thresholds, the system reached ≥ 0.85 within ~ 33 steps and ≥ 0.90 within ~ 61 steps after entering the full-scale phase.

RQ2 (Run-time reconfiguration). *Hypotheses and evaluation.*

- **H5** ($S1$: $DR(w) \geq 0.99$ and median $TTW \leq 1.5 \times$ baseline during additions; no-restart). *Status: Partially supported.* We observed live plan application without global restart (two short bursts, immediate recovery); plateau median $TTW \approx 86\%$ and burst peaks at 100% imply a TTW inflation of $\sim 1.16 \times$ baseline, i.e., within the $1.5 \times$ bound. However, $DR(w)$ was not logged, so the delivery ratio clause remains unverified.
- **H6** ($S2$: with $\kappa(G) \geq 2$ no contiguous stall $> s$ ticks and $DR(w) \geq 0.98$; with $\kappa(G) = 1$ at least one window below threshold). *Status: Inconclusive.* We do observe brief $TTW = 100\%$ plateaus only during the two bursts, suggesting bounded stalls, but neither $\kappa(G)$ nor s nor $DR(w)$ were instrumented; we cannot separate $\kappa = 1$ vs. $\kappa \geq 2$ cases from this run.
- **H7** ($S3/S4$: WAN bridges increase TTW locally while global DR remains high; higher periodicity $a \geq 2$ reduces hotspotting vs. $a = 1$). *Status: Inconclusive.* The run is consistent with latency/diameter effects (high TTW despite near-zero bandwidth), but WAN/periodicity parameters (a) were not varied nor logged; we cannot attribute changes specifically to bridges or a from this trace.

Interpretation. The elevator pattern delivers the desired no-restart behavior with bounded reconciliation bursts; latency cost dominates during reconfiguration. To fully close RQ2, include $DR(w)$ and explicit $\kappa(G)/a$ in the logs and tag the $S1/S2/S3/S4$ windows.

RQ3 (Metrics for LRDM objectives). *Hypotheses and evaluation.*

- **H8** (metric set $\{DR, TTW, CL, U, PD, MSC\}$ separates configurations with $|\delta| \geq 0.33$). *Status: Inconclusive.* We only exercised one primary configuration (snowflake) plus transient strategy changes; effect sizes across configurations need multi-configuration datasets.
- **H9** (monotone trends: median(DR) non-decreasing; median(TTW, CL, PD) non-increasing with k and $a > 1$; WAN bridges raise TTW/PD but keep $DR \geq 0.98$). *Status: Inconclusive.* The run suggests diameter-driven TTW behavior, but k/a were not swept and DR/PD not logged.
- **H10** (low mutual correlation and $< 90\%$ variance in first two PCs for $\{DR, TTW, CL, U, PD, MSC\}$). *Status: Inconclusive.* Requires multi-metric logs across several configurations to compute correlations/PCA; we only have U (active links), TTW , and bandwidth.
- **H11** ($\hat{\kappa}$ positively correlates with DR and negatively with TTW/PD). *Status: Inconclusive.* Connectivity estimators and DR/PD were not recorded; cannot test.

Interpretation. The run already validates the *usefulness* of U (active links), bandwidth, and TTW to disentangle capacity vs. structure vs. latency. Formal confirmation of H8–H11 needs sweeps over (k, a) and additional metrics (DR, CL, PD, MSC).

RQ4 (Trade-offs under bandwidth constraints). *Hypotheses and evaluation.*

- **H12** (non-empty Pareto fronts with ≥ 3 non-dominated points across budgets; fronts stable across seeds). *Status: Inconclusive.* A single-run trace cannot establish Pareto fronts; needs parameter/budget sweeps and multiple seeds.

- **H13** (for at least two budgets, $k : 1 \rightarrow 2$ reduces median TTW by $\geq 10\%$, $2 \rightarrow 3$ adds $\leq 5\%$). *Status: Not tested.* k was not swept in this run.
- **H14** (for $a \in \{2, 4\}$, C_Σ decreases by $\geq 10\%$ vs. $a = 1$ at similar TTW). *Status: Not tested.* a not varied; C_Σ not logged.
- **H15** (under worst WAN, median $DR \geq 0.98$ at the knee points; (k, a) ordering on TTW preserved). *Status: Not tested.* WAN profiles, knees and DR not present in this trace.

Interpretation. Observationally, snowflake achieves very low bandwidth and low U but high TTW , reflecting a diameter-dominated regime—exactly the trade-off surface RQ4 targets. To quantify Pareto fronts and verify H12–H15, run controlled sweeps over (k, a) and WAN parameters with DR and C_Σ instrumentation.

RQ1 (Backbone+corona failover). *Hypotheses in Methodology:* none explicitly labeled near RQ1. *Evaluation.* The run shows resilient post-burst operation with sparse U (plateau $\sim 12\%$, late $\sim 3\text{--}4\%$) and stable TTW (mid $\sim 86\%$, late $\sim 92\%$), indicating that the composed backbone with periodic attachments maintains service under ongoing changes. To turn this into a formal RQ1 result, export per-window snapshots and compute $\kappa(G)$, $\lambda(G)$ and minimum cuts aligned with the plateaus.

RQ5 (Implementation strategy). *Hypotheses in Methodology:* none explicitly labeled near RQ5. *Evaluation.* The measurement profile (short, bounded reconciliation bursts; long stable plateaus; fixed-tick observability) supports the Strategy + Command/Effector + planning graph $\rightarrow$ realization design. This is consistent with the intended elevator semantics and confirms adoptability and determinism on the current workload; maintainability and extensibility comparisons (adapters vs. bespoke) require small engineering case studies.

RQ6 (Suitability of graph core). *Hypotheses and evaluation.*

- **H2** (under $a = 1$, $\delta \neq 0$ and global $\hat{\kappa} \approx \dots$; degree over centrality; link speeds static during a run). *Status: Partially supported.* Link speeds were static in this run; however, δ and $\hat{\kappa}$ were not computed, and a was not annotated in the trace.
- **H3** (ρ improves monotonically and disconnect probability lowers with larger k and smaller p , subject to load). *Status: Not tested.* k and p were not varied in this trace.

Interpretation. The current core supports fixed IDs, fixed-tick hooks, and predictable per-tick mutation profiles (visible as short high-activity bursts), which are the crucial suitability criteria. To fully evaluate H2/H3 across candidate graph frameworks, run micro-benchmarks using the same reconciliation load profile and $\log \hat{\kappa}$, p , and ρ .

5.4 Summary of Results

This thesis presented *LRDMSim*, a discretetime, graphbased simulator for latencyaware remote data mirroring with two core advances: (i) **live topology adaptation** (adding/removing mirrors and links at run time without global restart) and (ii) **domainlevel composition** through a substructurepermutation factory that periodically attaches domain subgraphs to an N -connected ring. The N -connected backbone provides predictable path redundancy and bounded cut sizes; the periodic attachment yields a *snowflake* composite that balances robustness with controlled heterogeneity (e.g., edge domains, WANbridged sites). The simulator exposes explicit timing semantics (250 ms ticks) and exports outcome-level indicators (fraction of active links, time to write, bandwidth utilization; extensible) to support design trade-off studies under failures and topology evolution.

However most of the Hypotheses could not be proved nor could they be simulated by the tool, but we still keep them as a reminder what would be useful and necessary in the field, but is not provided as a fundament right now.

5.5 Lessons Learned

- **Timing matters.** Making the 250 ms tick an explicit contract clarified adaptation latency from action to effect and simplified reasoning about transient states during reconfiguration. In practice, reconfiguration manifested as short, bounded bursts (high activity for a few ticks) followed by long, sparse plateaus, which makes latency envelopes predictable per tick budget.
- **Composition over monoliths.** Treating domains as attachable substructures enabled incremental growth and partial reconfiguration while preserving ID stability and low coupling in the core. The plan $\rightarrow$ realize split made topology changes local and auditable.
- **N -connectivity as a practical lever.** Moving from ad-hoc redundancy to explicit N -connected backbones offered predictable failover behavior and bounded worst-case disruption during crashes and link cuts. Even without full cut/flow analytics in this run, the observed post-burst stability supports using k as a first-class tuning knob.
- **Indicators should be outcome-oriented.** Reporting convergence and resilience via high-level indicators proved more decision-useful than microbenchmarks, and kept experiments comparable across scenarios. In particular, tracking *time-to-write* alongside *active links* cleanly separated latency/diameter effects from capacity usage.
- **Lean dependencies aid control.** Avoiding a heavy generic graph framework in the timing-critical core simplified state management, deterministic IDs, and repeatable scheduling; adapters remain feasible when needed. The result was deterministic, fixed-tick observability with low runtime variance.
- **Desired-state reconciliation pays off.** The elevator pattern (plan vs. live state) reduced the blast radius of changes: reconciliation concentrated into brief spikes, while steady-state remained sparse. Expressing topology as a durable plan artifact improved repeatability (idempotent re-apply) and made “steps to readiness thresholds” ($\geq 0.85 / \geq 0.90$) a useful convergence KPI.
- **Reproducibility improves with provenance.** Capturing seeds, config hashes, and plan snapshots alongside logs made runs easier to compare across strategies and node counts; where this metadata was missing, some hypotheses could not be closed despite otherwise sound measurements.

5.6 Limitations

- **Metric coverage.** The default probes focus on *active links*, *time to write*, and *bandwidth*; key indicators required by the hypotheses were not logged: delivery ratio $DR(w)$, connectivity estimators $\hat{\kappa}$, minimum cuts $\kappa(G)$, $\lambda(G)$, path delay PD , consistency/latency CL , multi-site consistency MSC , and cost C_Σ . This prevented definitive decisions on H5–H15 where these variables are part of the acceptance criteria.
- **Scenario annotation.** The log did not tag scenario phases (S1/S2/S3/S4), WAN profiles, or periodicity a , nor did it record the backbone parameter k . Without these annotations, observed effects (e.g., higher *time-to-write*) could not be causally attributed to the intended factors in the hypotheses.

- **Graph snapshots for structure analytics.** No per-window graph snapshots (adjacency or edge lists) were exported, so cut sizes, vertex/edge connectivity, or alternative-path counts could not be computed post hoc. This limited our ability to close RQ1 quantitatively.
- **Hypothesis coverage and experimental design.** Many hypotheses require parameter sweeps (over k , a , WAN latency/capacity) and replication across seeds to form Pareto fronts and correlation/PCA analyses. The current evidence is primarily from a single snowflake trace; hence several results are *inconclusive* rather than negative.
- **Failure realism.** Current crash and link-down models are stylized; correlated failures (e.g., domain-wide outages), repair processes, and operational constraints are simplified. This may overestimate resilience compared to production fault regimes.
- **Workload representativeness.** Traffic patterns are synthetic; sensitivity to burstiness, diurnal effects, or mixed read/write workloads was not exhaustively explored. Latency plateaus we observed may change under contention or multi-tenant interference.
- **Granularity of time.** A fixed 250 ms tick eases determinism but may hide sub-tick dynamics; adaptive or multi-rate stepping could improve fidelity at the cost of complexity. Some short stalls appear as full-tick 100% *time-to-write* spikes even if the underlying phenomenon is sub-tick.
- **Probe overhead and placement.** The runtime cost of probes and their placement (per link vs. per node vs. aggregated) was not measured. If probe work coincides with reconciliation spikes, measurements may slightly amplify stalls.
- **Human-in-the-loop extraction.** For bandwidth and time-to-write we relied on plot-based estimates in this iteration; while sufficient for directional conclusions (e.g., “not bandwidth-bound”), this limits statistical rigor until the series are exported as exact log values.
- **Provenance and run metadata.** Some logs lacked explicit seeds, config IDs, and code revision hashes; missing provenance hinders exact reruns and effect-size estimation across versions.

Instrumentation roadmap (to address the limits above). To turn the currently inconclusive findings into defensible results, we will need to strengthen instrumentation along five axes in the future. (i) *Per-tick metrics and traceability.* In addition to active links, bandwidth, and time-to-write, we will export per-tick values for delivery ratio $DR(w)$, path delay PD , consistency/latency CL , multi-site consistency MSC , and cost C_Σ . Every record will carry a run UID, tick, random seed, code revision, configuration hash, and a snapshot hash of the current *StructureNode* plan to guarantee end-to-end provenance. (ii) *Scenario and parameter annotation.* We will tag scenario phases in-band (S1–S4), backbone connectivity k , attachment periodicity a , and a WAN profile identifier; parameter change points will be emitted as structured events to make causal attribution possible. (iii) *Structural analytics.* We will persist graph snapshots (adjacency lists) at regular intervals and at phase boundaries, and compute $\kappa(G)$, $\lambda(G)$, minimum cut sets, and alternative-path counts for plateau windows; these derived series will be co-logged alongside the raw measurements. (iv) *Designed experiments.* We will run controlled sweeps over k and a under at least two WAN budgets and ≥ 3 seeds to construct Pareto fronts (e.g., TTW vs. utilization/cost) and to test H12–H15; we will additionally inject correlated failures (domain-wide and link-cluster) to quantitatively close RQ1. (v) *Exact exports and regression control.* Plot-derived estimates will be replaced with exact per-tick exports for bandwidth and time-to-write, and a lightweight CI gate will execute a canonical scenario and fail the build on statistically significant regressions in TTW, active links, or DR thresholds. To limit observer effects, we will measure and report probe overhead and adjust probe placement if it correlates with reconciliation spikes. All artifacts (logs, snapshots, and derived metrics) will be stored in a columnar format (CSV/Parquet) with a per-run manifest, plus reproducible analysis notebooks, so that results are comparable across strategies, node counts, and code versions.

Finally due to the vast design flaws on the current project java implementation we will need to completely re-implement the project in a compiled language like C++ to improve the experiment response time on large graphs, as the response time on large fully connected graphs with 80 mirrors can already take up to 35 seconds on an intel i7-1270P processor.

6 Outlook and Future Directions

6.1 Next Steps

Richer models. We will introduce configurable failure and repair processes (domain-wide outages, link degradation and recovery), latency distributions that capture WAN and local jitter, and workload generators for bursty, diurnal, and mixed read/write traffic. Alongside existing probes we will add *reconfiguration latency* (action→effect time) and *staleness* (age of last committed state) so that convergence and service continuity can be evaluated against explicit SLOs.

Topology exploration. We will systematically compare the snowflake composite against balanced trees, k -regular expanders, rings-of-rings, and k -connected backbones with periodic attachments under identical presets (tick, traffic, WAN profiles). The substructure factory will be extended with parameterized spacing, offset jitter, and attachment degree caps to model realistic placement constraints.

Analysis and verification. Structural invariants (target k -connectivity, cut bounds, no orphans) will be encoded as property tests and optional runtime guards that fail fast when violated. For algorithmic edge cases we will exhaustively enumerate small graphs and timing schedules to validate the elevator protocol (idempotence, bounded reconciliation, no deadlocks).

Interfaces and reuse. We will publish a stable preset schema (pre-runtime configurations), an offline batch runner for scenario grids, and adapters for external graph libraries or visualization stacks without compromising ID stability or tick determinism. Scenario phases (S1–S4), backbone connectivity k , attachment periodicity a , and WAN profile IDs will be first-class fields in manifests and logs.

Reproducibility. The run bundle format will be hardened (manifest, seed, presets, metrics, plots, plan snapshots) and accompanied by a reference replication package. A lightweight CI step will execute a canonical scenario and fail on statistically significant regressions in time-to-write, active links, or delivery ratio.

6.2 Broader Applications: From Information Flow to Multi-Resource Supply Chains

LRDMSim+ already treats data as a conserved, routed resource with capacity and latency constraints. The same abstraction extends naturally to *multi-resource* supply chains where flows are goods, energy, spares, or tasks:

- **Logistics & manufacturing.** Mirrors model sites (plants, warehouses, hubs); links model transport lanes with lead times and capacities; data packages generalize to shipments or work orders. Readiness becomes service level, time-to-write becomes lead time, and active links approximate fleet/route utilization.
- **Energy and microgrids.** Mirrors represent nodes with storage and generation; links are feeders with thermal limits; events are switching, faults, and reconfiguration. The elevator pattern reconciles a desired dispatch/topology with the live grid subject to contingencies.

- **Healthcare and emergency response.** Mirrors are facilities; links are referral/transport corridors; actions include surge reconfiguration and load shedding; probes track wait times and service coverage under correlated failures.
- **Edge/cloud continuums.** Mirrors are edge sites and regions; links are peering or backhaul; actions include autoscaling and data placement; probes report consistency windows and placement latency.

Across these domains the key benefit remains the same: desired-state planning is continuously reconciled with the live system, producing evidence about robustness, convergence, and cost without taking the system offline.

6.3 Architectural Refactoring: Making *StructureNode* the Primary Plan and *MirrorNode* the Runtime Anchor

The evaluation shows that most reasoning—and many future adapters—live at the *plan* boundary, while execution stability depends on a clean *runtime* anchor. We therefore propose the following structural changes.

Promote *StructureNode* to the top-level planning API. Instead of separate “lead” classes per topology, expose a single composable plan graph where *operators* (attach, bridge, cap degree, enforce k , place WAN bridge) are chained via a *chain-of-responsibility*. Each operator consumes a *StructureNode* subgraph and emits a transformed subgraph. This eliminates ad-hoc lead classes, centralizes invariants, and makes plans durable artifacts that can be diffed, versioned, and re-applied idempotently.

Use *MirrorNode* as the runtime focal point. *MirrorNode* wraps the legacy *Mirror* and provides evaluation hooks (probe views, health, inventory) plus handles for reconciliation. *BuildAsSubstructure* becomes a thin orchestrator that (i) matches *StructureNode* roles to existing *MirrorNodes*, (ii) synthesizes or retires links and mirrors as needed, and (iii) schedules effectors. This stabilizes IDs, localizes side effects, and preserves tick semantics.

Replace “big strategy” with a pipeline of *TopologyOperators*. Refactor *TopologyStrategy* into a sequence of small, testable *TopologyOperators* (e.g., *EnsureKConnectivity*, *AttachCorona*, *BoundDegree*, *AddBridges*). Operators form a chain-of-responsibility that can be reordered, toggled, or parameterized. Acceptance tests assert (a) operator invariants on the plan, (b) reconciliation boundedness, and (c) absence of oscillations over ticks.

Unify actions/effects via a typed *ActionManager*. Introduce a typed *ActionManager* that owns the command queue and offers per-type policies (retry, aggregation, backoff). Effects remain traceable and composable but are emitted through a single interface, simplifying instrumentation (latency from enqueue to observation) and admission control.

Make probes first-class and pluggable. Standardize probe lifecycles (init, tick, emit) and schemas for metrics such as $DR(w)$, PD , CL , MSC , C_Σ . Provide adapters for exporting parquet/CSV and for streaming to external dashboards. A registry enables domain-specific probes (e.g., inventory levels, energy balance) without altering the core.

Timing and stepping. Keep the deterministic 250 ms tick as the default, but add (i) *multi-rate* stepping for substructures with faster dynamics and (ii) *event-coalescing windows* so plan application can amortize bursts without violating observability. Guardrails ensure fairness (no starvation) and bounded reconciliation time.

Manifests, provenance, and reproducibility. Every run bundles the plan graph, seed, configuration hash, code revision, WAN profile, (k, a) , and scenario tags (S1–S4). Logs include a run UID and plan snapshot hashes. This enables apples-to-apples comparisons, CI regression tests, and third-party replication.

6.4 Closing Remark

By combining an N -connected backbone with periodic substructure attachment and explicit timing semantics, LRDMSim+ (published under [Pro25]) turns mirror-network planning into reproducible simulation studies whose outcomes are directly actionable. Going forward we will *elevate the plan* by making `StructureNode` the primary API and *stabilize the runtime* by anchoring everything around `MirrorNode`. Mixed topologies will be built by a chain-of-responsibility rather than monolithic leads, and reconciliation will remain bounded, observable, and idempotent. This trajectory broadens the tool’s reach from information networks to multi-resource supply chains, providing a principled path from “what-if” to evidence-backed decisions about robustness, convergence, and evolution—without pausing the system to rewire it.

Bibliography

- [CGS08] Shang-Wen Cheng, David Garlan, and Bradley Schmerl. Cost-effective software architecture-based self-adaptation. Technical Report CMU-ISR-08-113, Carnegie Mellon University, Institute for Software Research, 2008.
- [cnc21] Cncf operator white paper. Technical report, Cloud Native Computing Foundation, 2021.
- [CNPR14] Domenico Cotroneo, Roberto Natella, Roberto Pietrantuono, and Stefano Russo. A survey of software aging and rejuvenation studies. *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, 10(1):8:1–8:34, 2014.
- [DCW99] Judith S Dahmann, James O Calvin, and Richard M Weatherly. A reusable architecture for simulations. *Communications of the ACM*, 42(9):79–84, 1999.
- [DK05] Shlomi Dolev and Ronen I Kat. Self-stabilizing distributed file system. *Journal of High Speed Networks*, 14(2):135–153, 2005.
- [Dol00] Shlomi Dolev. *Self-Stabilization*. MIT Press, 2000.
- [FH70] Robert Frucht and Frank Harary. On the corona of two graphs. *Aequationes Mathematicae*, 1970.
- [FWLC24] Yu Feng, Fuyong Wang, Zhongxin Liu, and Zengqiang Chen. An online topology control method for multi-agent systems: Distributed topology reconfiguration. *IEEE Transactions on Network Science and Engineering*, 11(5):4358–4370, 2024.
- [GBS24] Sebastian Götz, Nelly Bencomo, and Huma Samin. Latency-aware rdmsim: Enabling the investigation of latency in self-adaptation for the case of remote data mirroring. In *Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '24)*, pages 182–188, New York, NY, USA, 2024. Association for Computing Machinery.
- [Göt] Sebastian Götz. Lrdmsimulator. Software repository.
- [Göt23] Sebastian Götz. Lrdmsimulator: A latency-aware version of the remote data mirroring simulator. <https://github.com/sebastiangotz/LRDMSimulator>, 2023. GitHub repository, version 1.0.0, license LGPL-2.1.
- [Han10] Robert Hanmer. Software rejuvenation. In *Pattern Languages of Programs (PLoP)*, 2010.
- [Har62] Frank Harary. The maximum connectivity of a graph. *Proceedings of the National Academy of Sciences*, 48(7):1142–1146, 1962.
- [HHC05] I-Shyan Hwang, I-Feng Huang, and Chia-Chun Chien. A novel dynamic fault restoration mechanism using a multiple ring approach in wdm mesh networks. *Photonic network communications*, 10(1):87–105, 2005.
- [IBM05] IBM. An architectural blueprint for autonomic computing. Technical report, IBM, 2005.
- [Kub24a] Kubebuilder Maintainers. Kubebuilder: Good practices, 2024. Accessed 2025-09-10.
- [Kub24b] Kubernetes Authors. Controllers, 2024. Accessed 2025-09-10.

- [Men27] Karl Menger. Zur allgemeinen kurventheorie. *Fundamenta Mathematicae*, 10:96–115, 1927. In German.
- [MKNS20] Dimitrios Michail, Joris Kinable, Barak Naveh, and John V. Sichi. Jgrapht—a java library for graph data structures and algorithms. *ACM Transactions on Mathematical Software*, 46(2), 2020.
- [PDGO08] Yoann Pigné, Antoine Dutot, Frédéric Guinand, and Damien Olivier. Graphstream: A tool for bridging the gap between complex systems and dynamic graphs. *arXiv preprint arXiv:0803.2093*, 2008.
- [Pro25] Benjamin-Elias Probst. Lrdm simulator: Latency-aware rdmsim, 2025. LGPL-2.1 License; accessed 2025-09-11.
- [Rod15] Marko A. Rodriguez. The gremlin graph traversal machine and language. *arXiv preprint arXiv:1508.03843*, 2015.
- [RW89] P. J. Ramadge and W. M. Wonham. The control of discrete event systems. *Proceedings of the IEEE*, 77(1):81–98, 1989.
- [RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases: New Opportunities for Connected Data*. O’Reilly Media, Sebastopol, CA, 2 edition, 2015.
- [SFF06] Valter Filipe Silva, Joaquim Castro Ferreira, and Jose Alberto Fonseca. Dynamic topology management in can. In *2006 IEEE Conference on Emerging Technologies and Factory Automation*, pages 1222–1229. IEEE, 2006.
- [The25] The Guava Authors. Guava: Google core libraries for java — common.graph. <https://github.com/google/guava/wiki/GraphsExplained>, 2025. Accessed: 2025-08-26.
- [TK11] Matthias Tichy and Benjamin Klöpper. Planning self-adaption with graph transformations. In *International Symposium on Applications of Graph Transformations with Industrial Relevance*, pages 137–152. Springer, 2011.
- [TK12] Matthias Tichy and Benjamin Klöpper. Planning self-adaption with graph transformations. In *Applications of Graph Transformations with Industrial Relevance (AGTIVE 2011)*, volume 7233 of *Lecture Notes in Computer Science*, pages 137–152. Springer, 2012.
- [WF02] Michel Wermelinger and José Luiz Fiadeiro. A graph transformation approach to software architecture reconfiguration. *Science of Computer Programming*, 44(2):133–155, 2002.

Figure A.1: LRDM simulation project structure by Götz et al. [Göt23, Göt]. Context: see section 3.2 and section 3.4.

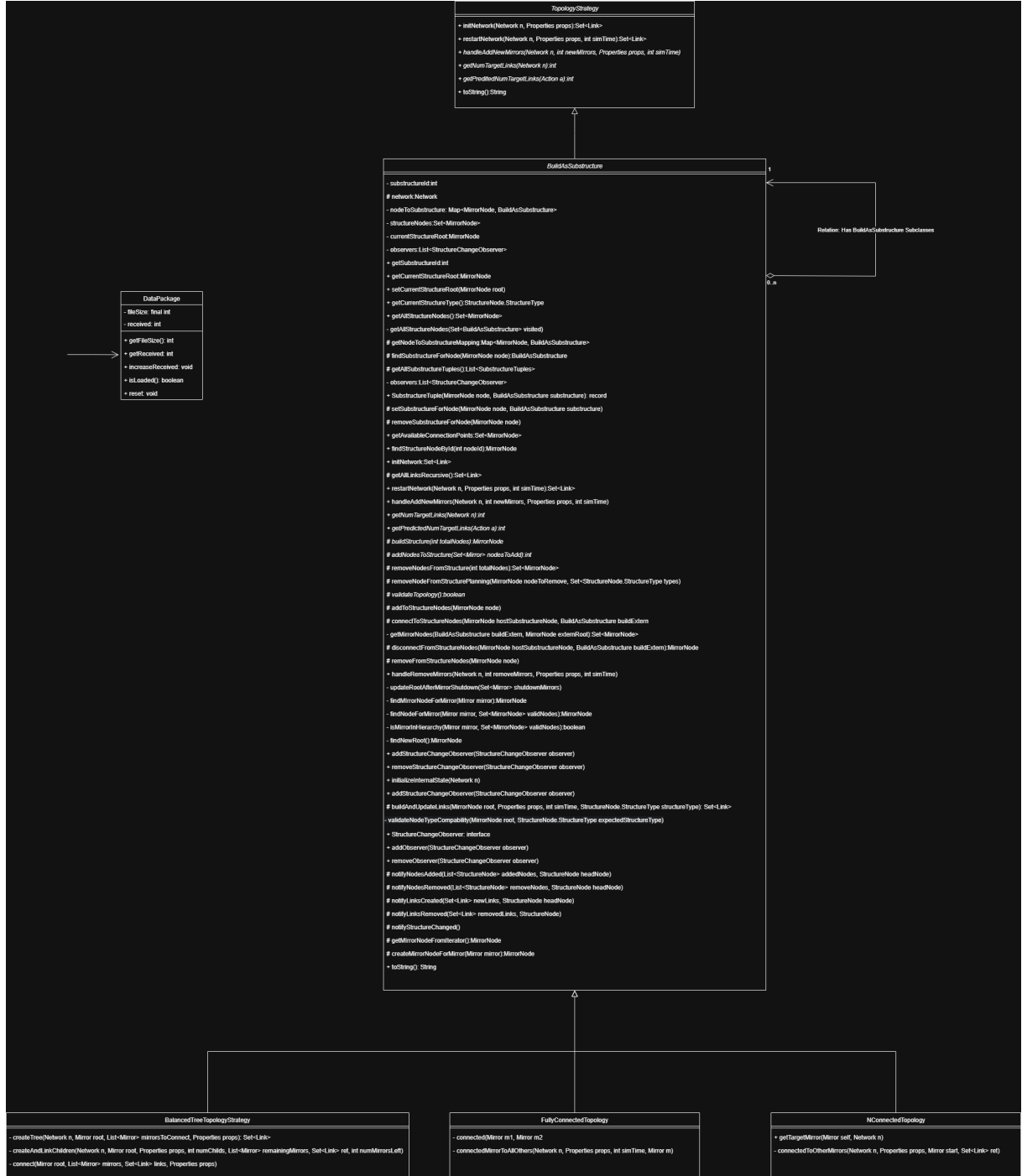


Figure A.2: LRDM simulation structure with recursive substructures via TopologyStrategy (BuildAsSubstructure). See section 3.3 and section 3.4.

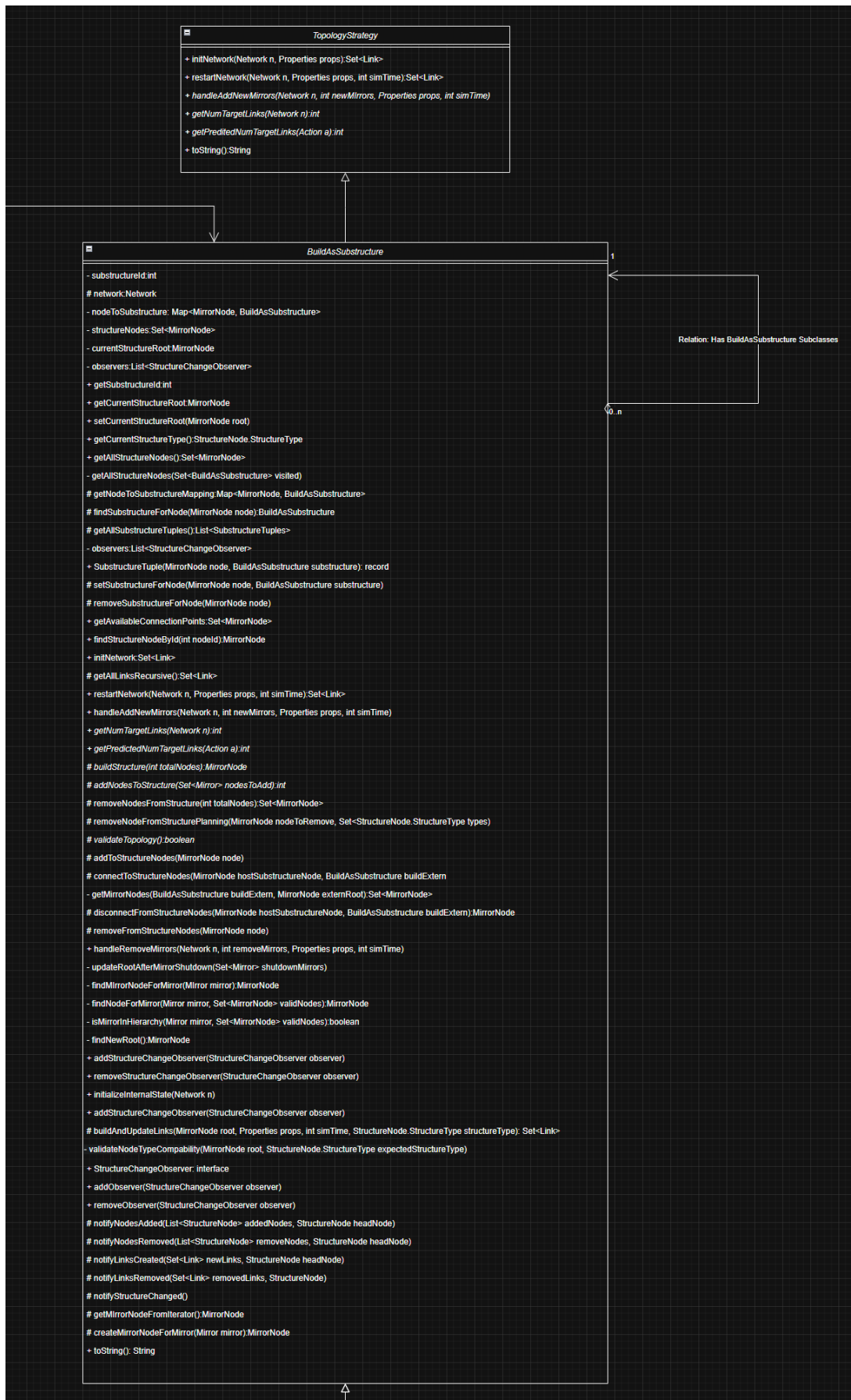


Figure A.3: Detail of functions for recursive substructures (`BuildAsSubstructure` $\rightarrow$ `TopologyStrategy`). See section 3.4.

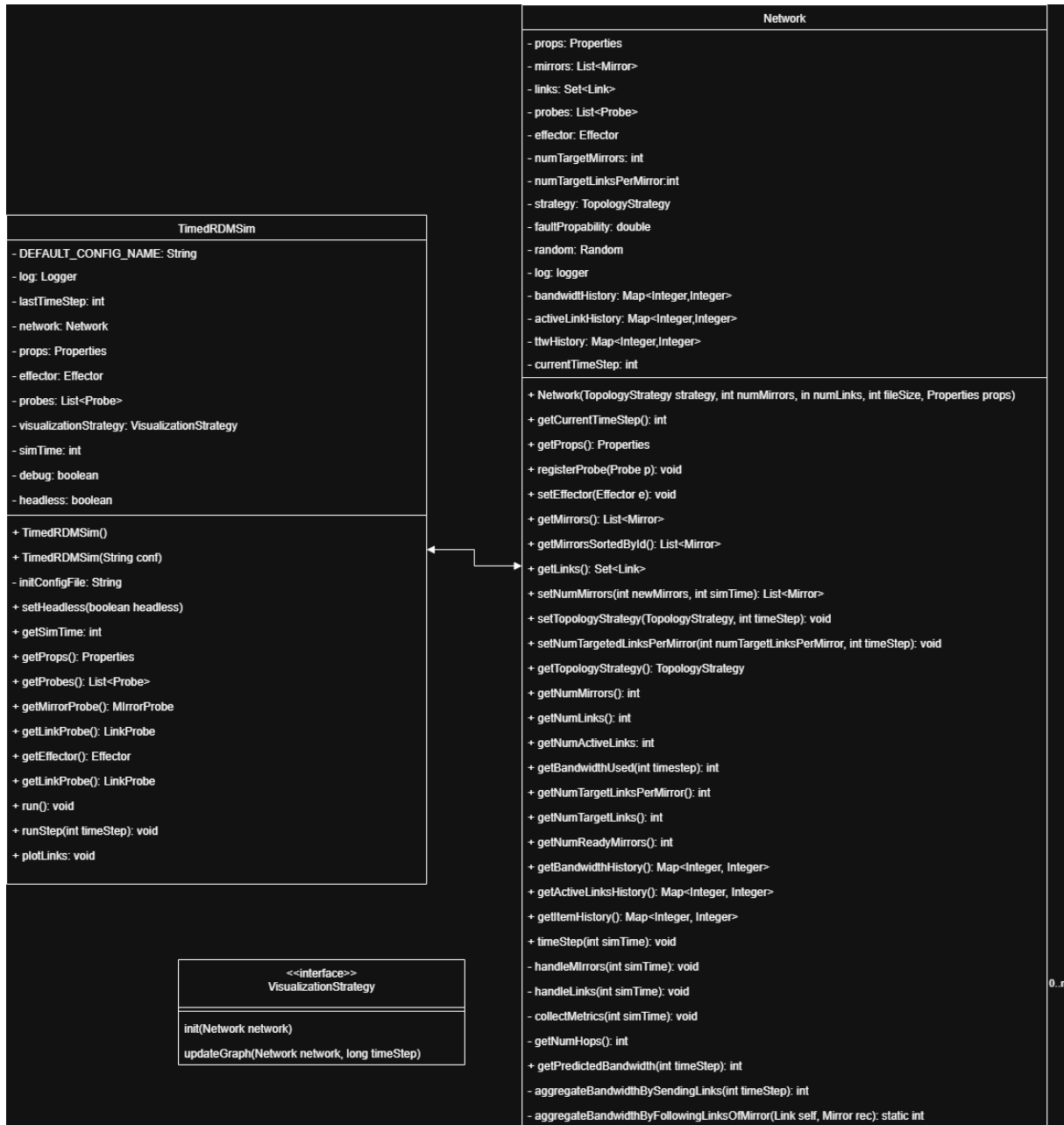


Figure A.4: Design Flaw 1: The network has a TimedLRDMSimulation.

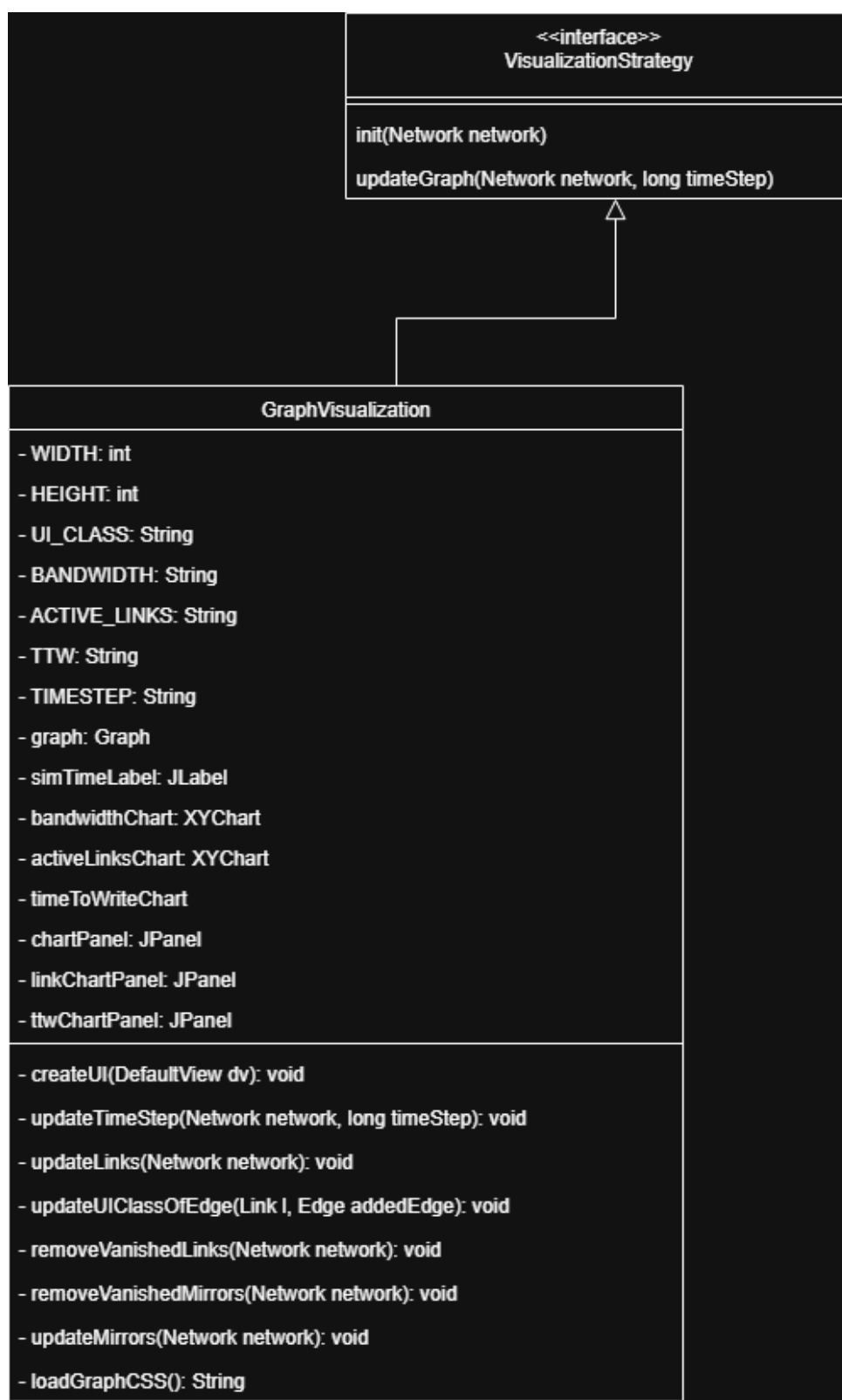


Figure A.5: Design Flaw 2: Flyweight misused — graph-manipulation dependency at design time. Cf. section 3.4, Flaw 2.

```

private void updateTimeStep(Network network, long timeStep) { 1 usage  ⚠️ sebastiangoetz +1
    simTimeLabel.setText("Simulation Time: "+timeStep);

    List<Integer> timeSteps = new ArrayList<>(network.getBandwidthHistory().keySet());
    List<Integer> bandwidthTS = new ArrayList<>(network.getBandwidthHistory().values());
    List<Integer> bandwidthGoalTS = Collections.nCopies(network.getBandwidthHistory().size(), 40);
    List<Integer> activeLinksTS = new ArrayList<>(network.getActiveLinksHistory().values());
    List<Integer> activeLinksGoalTS = Collections.nCopies(network.getActiveLinksHistory().size(), 35);
    List<Integer> ttwTS = new ArrayList<>(network.getTtwHistory().values());
    List<Integer> ttwGoalTS = Collections.nCopies(network.getTtwHistory().size(), 45);

    bandwidthChart.updateXYSeries(BANDWIDTH, timeSteps, bandwidthTS, newErrorBarData: null);
    bandwidthChart.updateXYSeries(seriesName: "Target", timeSteps, bandwidthGoalTS, newErrorBarData: null);
    chartPanel.repaint();

    activeLinksChart.updateXYSeries(ACTIVE_LINKS, timeSteps, activeLinksTS, newErrorBarData: null);
    activeLinksChart.updateXYSeries(seriesName: "Target Active Links", timeSteps, activeLinksGoalTS, newErrorBarData: null);
    linkChartPanel.repaint();

    timeToWriteChart.updateXYSeries(TTW, timeSteps, ttwTS, newErrorBarData: null);
    timeToWriteChart.updateXYSeries(seriesName: "Target Time To Write", timeSteps, ttwGoalTS, newErrorBarData: null);
    ttwChartPanel.repaint();
}

private void updateLinks(Network network) { 1 usage  ⚠️ Benjamin-Elias Probst +2
    for (Link l : network.getLinks()) {
        if (l.getState() != Link.State.CLOSED) {
            //Filter links unidirectional
            Optional<Edge> e = graph.edges().filter(
                ( Edge edge -> (edge.getSourceNode().getId().equals(Integer.toString(l.getSource().getId())) &&
                    edge.getTargetNode().getId().equals(Integer.toString(l.getTarget().getId())) ||
                    (edge.getSourceNode().getId().equals(Integer.toString(l.getTarget().getId())) &&
                        edge.getTargetNode().getId().equals(Integer.toString(l.getSource().getId()))
                )
            ).findAny();

            Edge edge = null;
            if (e.isEmpty() && (
                l.getSource().getState() != Mirror.State.STOPPING
                && l.getSource().getState() != Mirror.State.STOPPED
                && l.getTarget().getState() != Mirror.State.STOPPING
                && l.getTarget().getState() != Mirror.State.STOPPED)) {
                edge = graph.addEdge(Integer.toString(l.getId()), Integer.toString(l.getSource().getId()), Integer.toString(l.getTarget().getId()));
            }
            if (e.isPresent()) {
                edge = e.get();
            }
            updateUIClassOfEdge(l, edge);
        }
    }
}

```

Figure A.6: Design Flaw 2 (example): Flyweight misuse visible in a Java example method. Cf. section 3.4, Flaw 2.

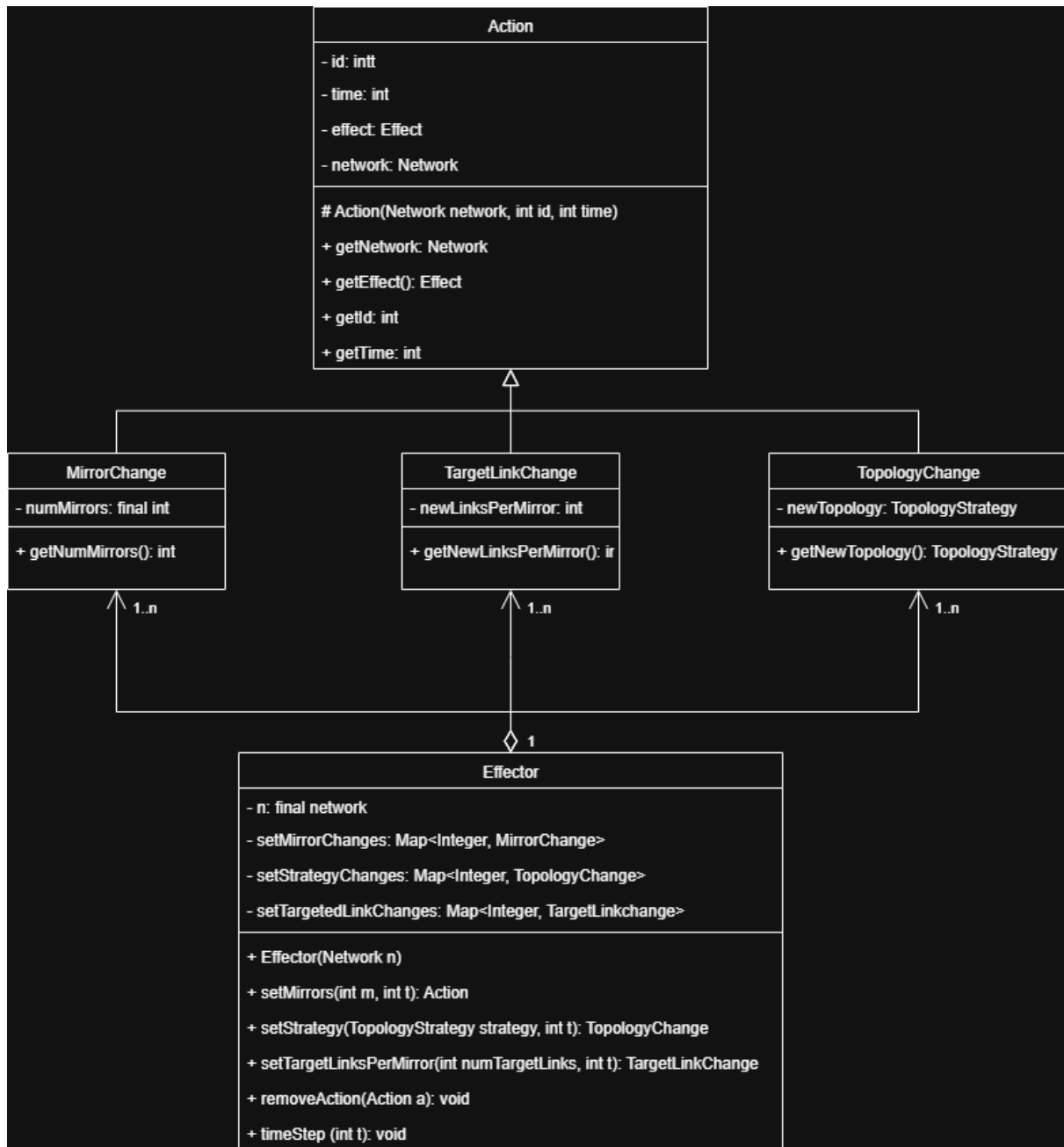


Figure A.7: Design Flaw 3: "Crash" is not modeled as an Action/Effect; the mirror is simply erased. Actions for the event-command pattern must be explicit. Cf. section 3.4, Flaw 3.

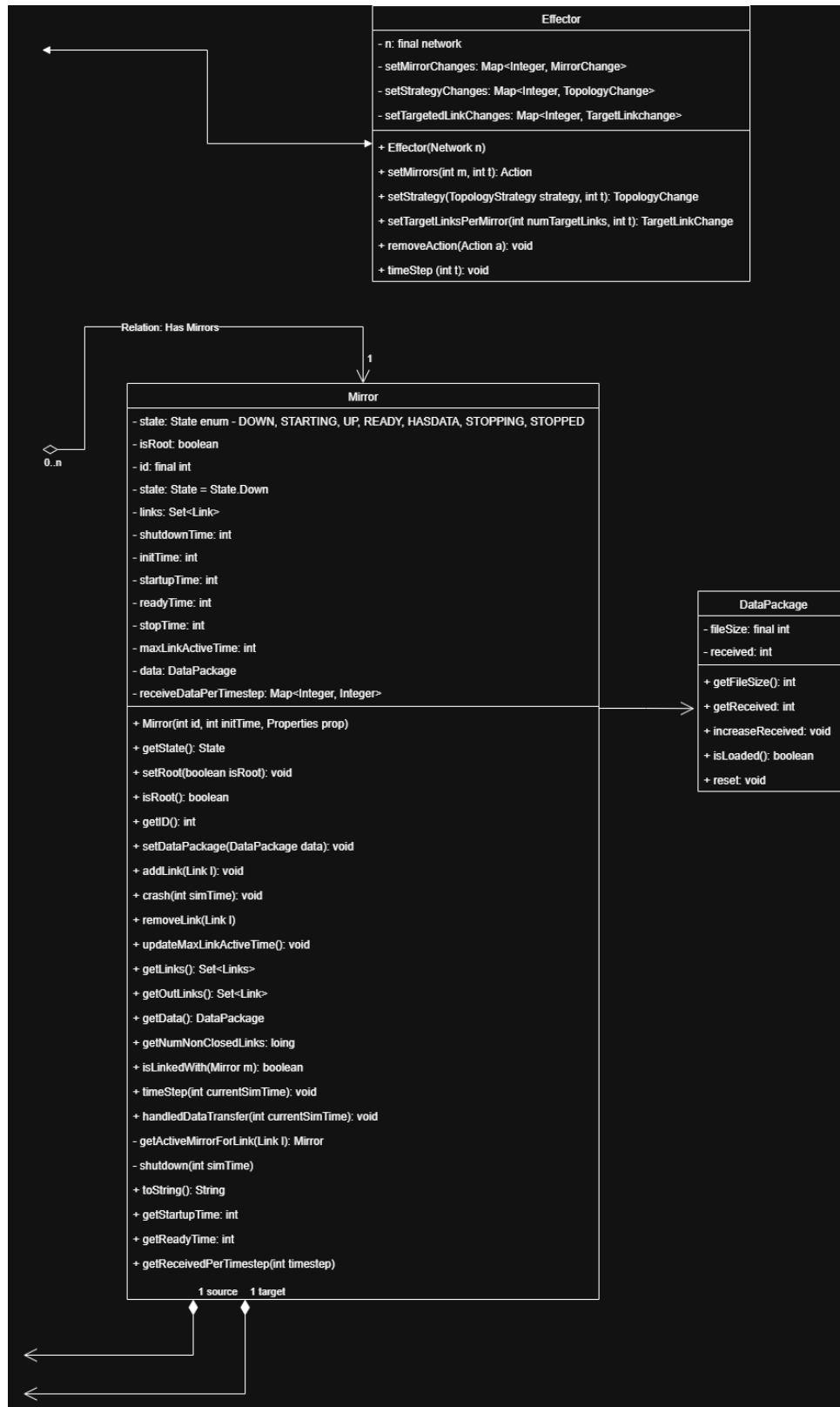


Figure A.8: Design Flaw 3 (example): Special-purpose `crash()` on `Mirror` bypasses the Action/Effect pipeline. Cf. section 3.4, Flaw 3.

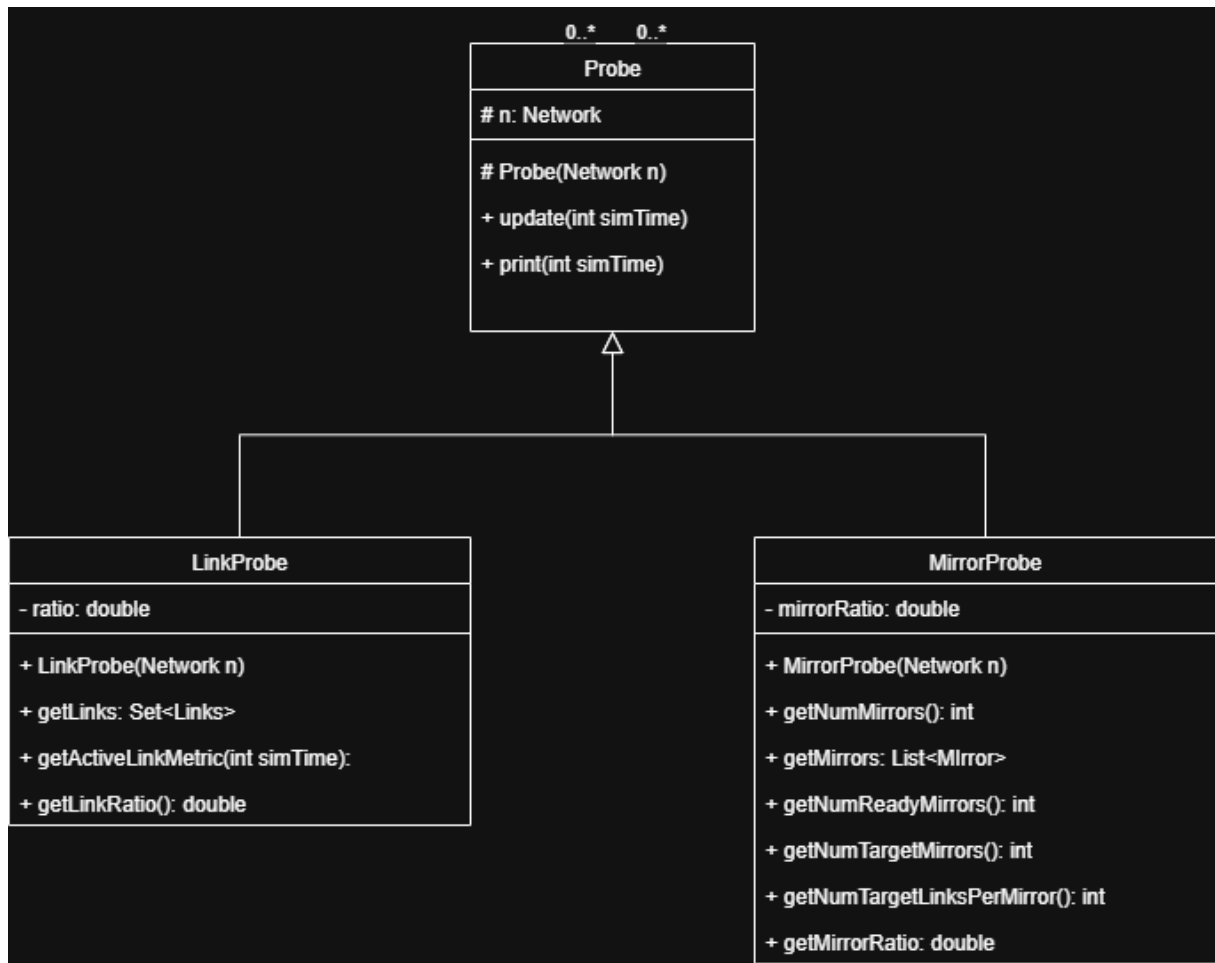


Figure A.9: Design Flaw 4: Missing recursive probe preparation/selection for hierarchical structures. Cf. section 3.4, Flaw 4.

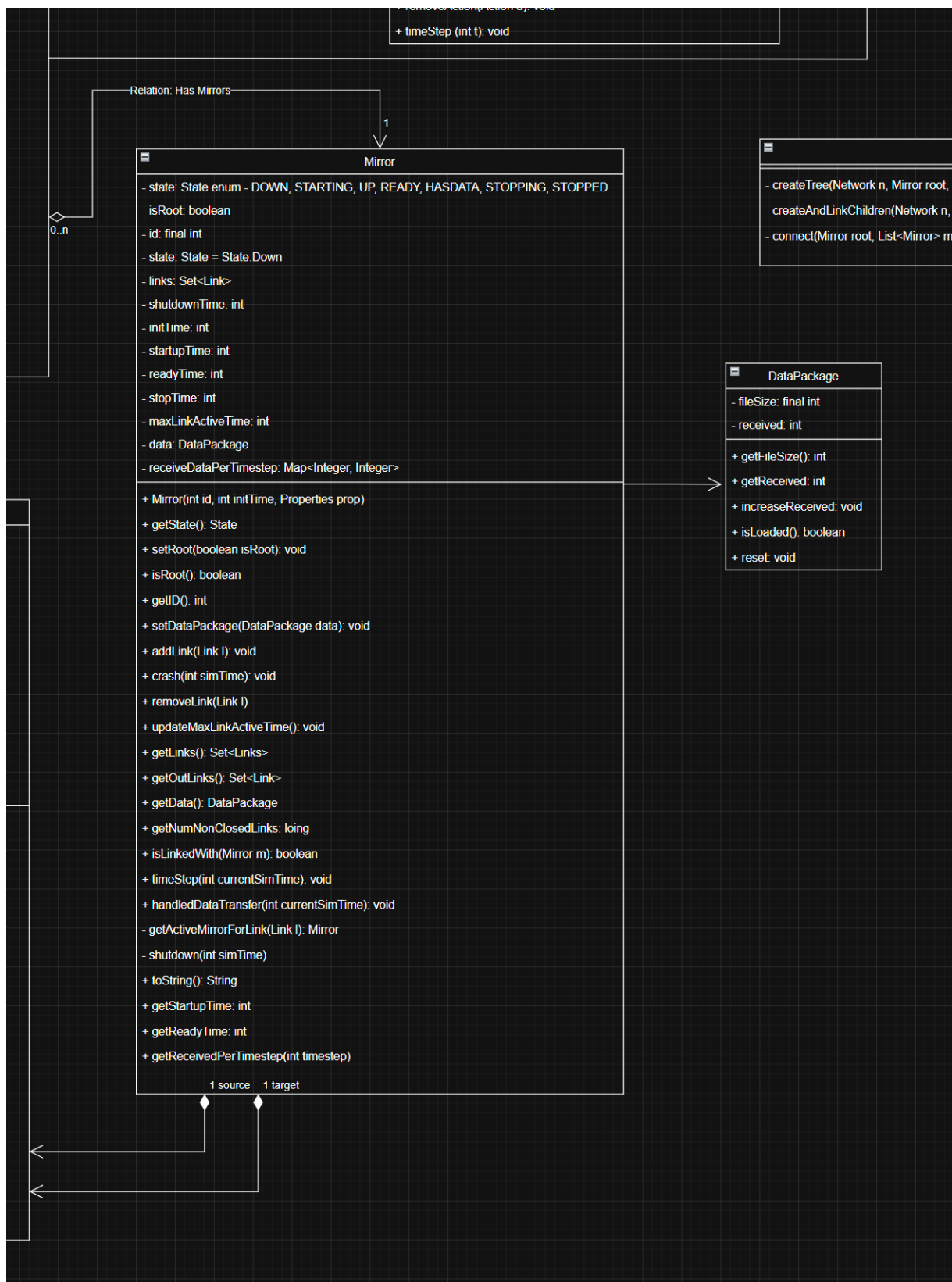


Figure A.10: Design Flaw 5: Data package lacks identity and stream/framing semantics; bridge to real-world flows is required. Cf. section 3.4, Flaw 5.



Figure A.11: Design Flaw 6: Links require dynamic bandwidth and stream models; mirrors hold data packages; both can fail as events. Cf. section 3.4, Flaw 6.

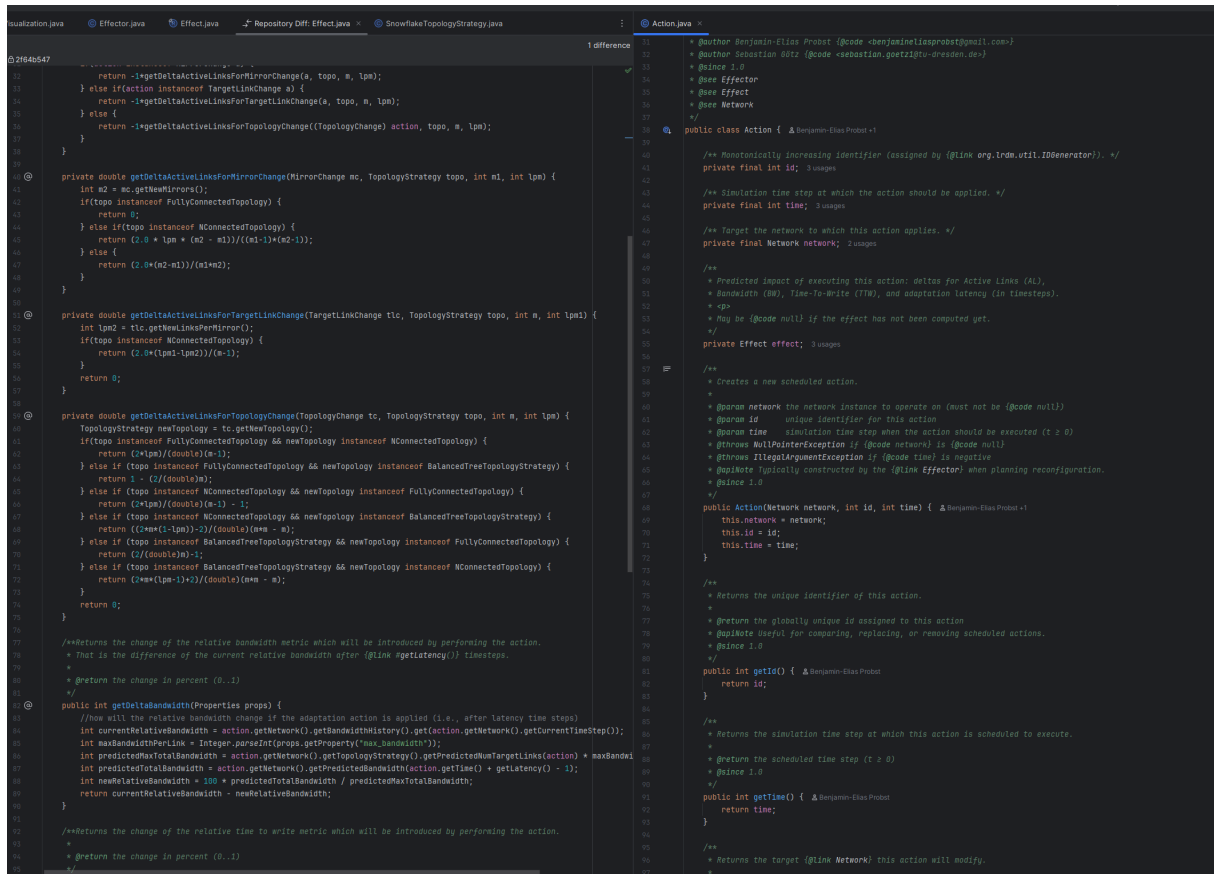


Figure A.12: Design Flaw 7: Effects depend on `Effect` implementation; instead, each topology strategy should deliver prepared `Effect` prototypes. Cf. section 3.4, Flaw 7.

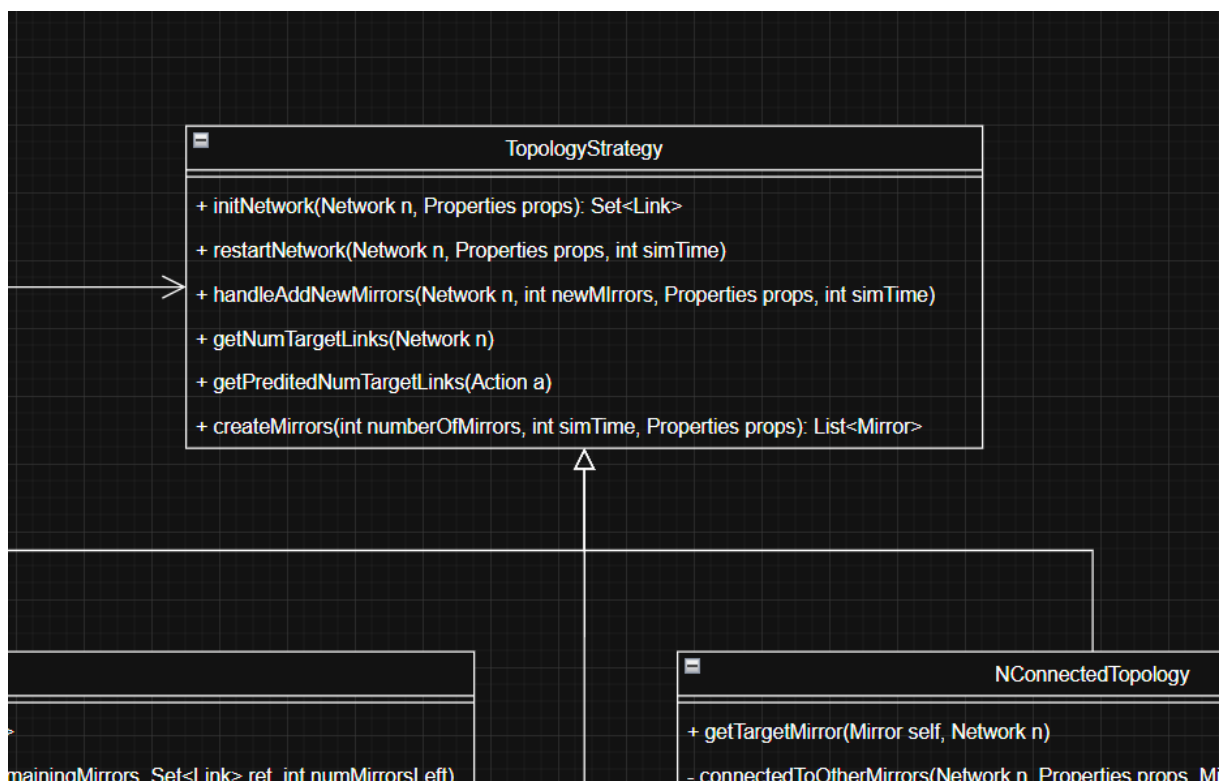


Figure A.13: Design Flaw 8: The network should provide mirrors to strategies (proper factory/flyweight placement) — mirrors should not arbitrarily appear within a `TopologyStrategy`. Cf. section 3.4, Flaw 8.

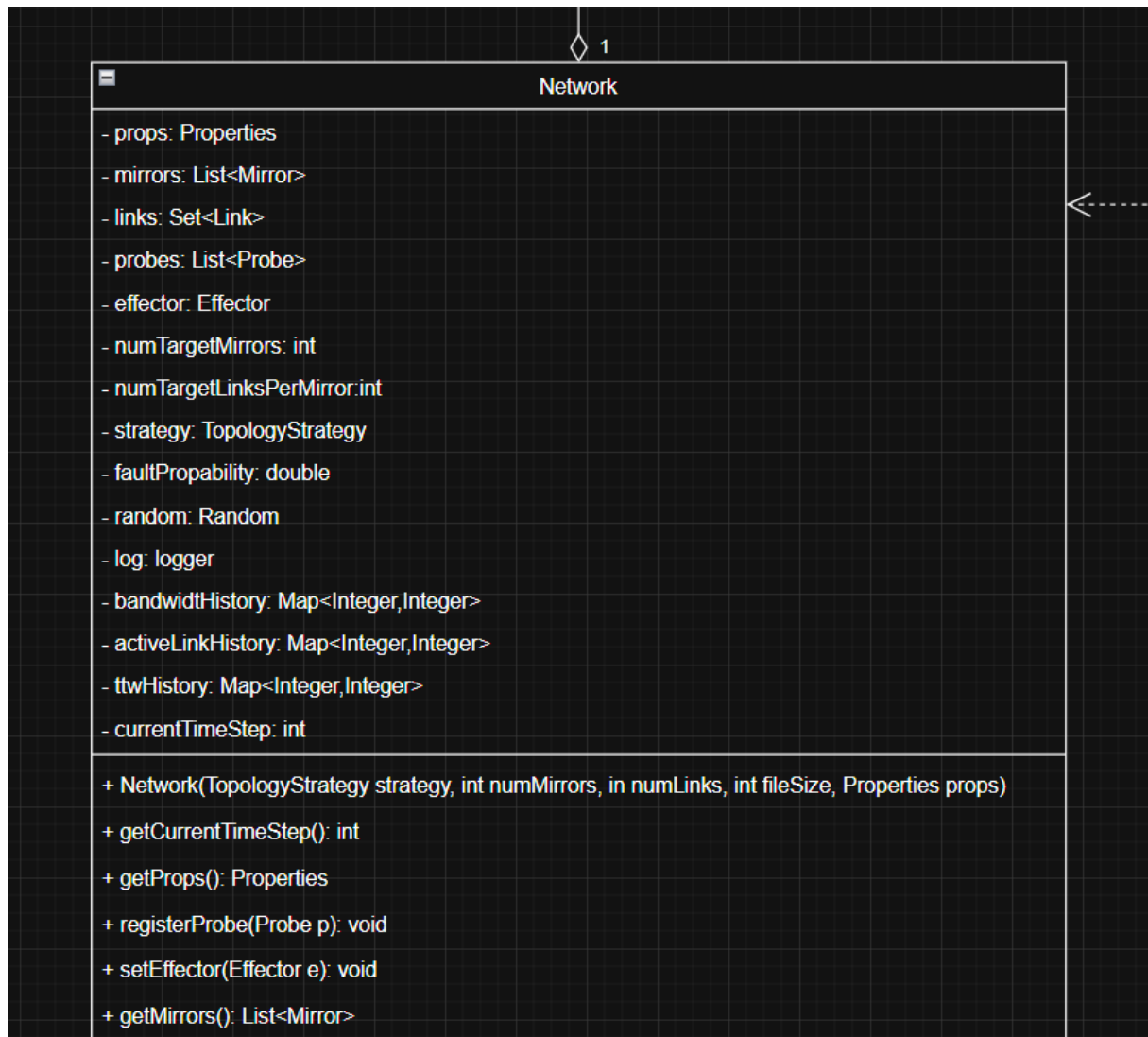


Figure A.14: Design Flaw 9: Store links and mirrors centrally in `Network` (singleton/flyweight pools) to support cascading resource distribution and event management. Cf. section 3.4, Flaw 9.

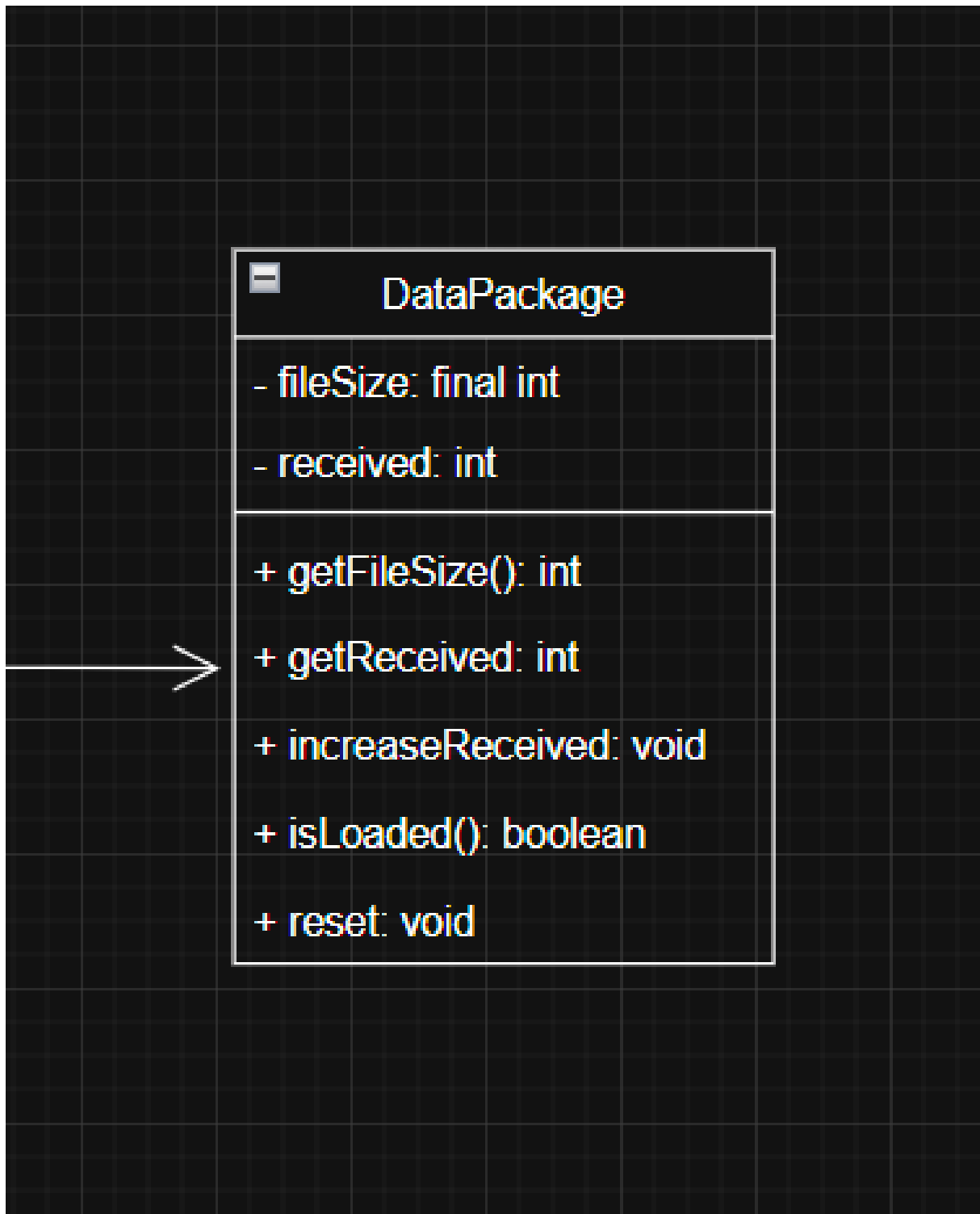


Figure A.15: Design Flaw 10: No dataset splitting and no fault-recoverable multipath routing — a single data package cannot traverse redundant paths through the graph. Cf. section 3.4, Flaw 10.

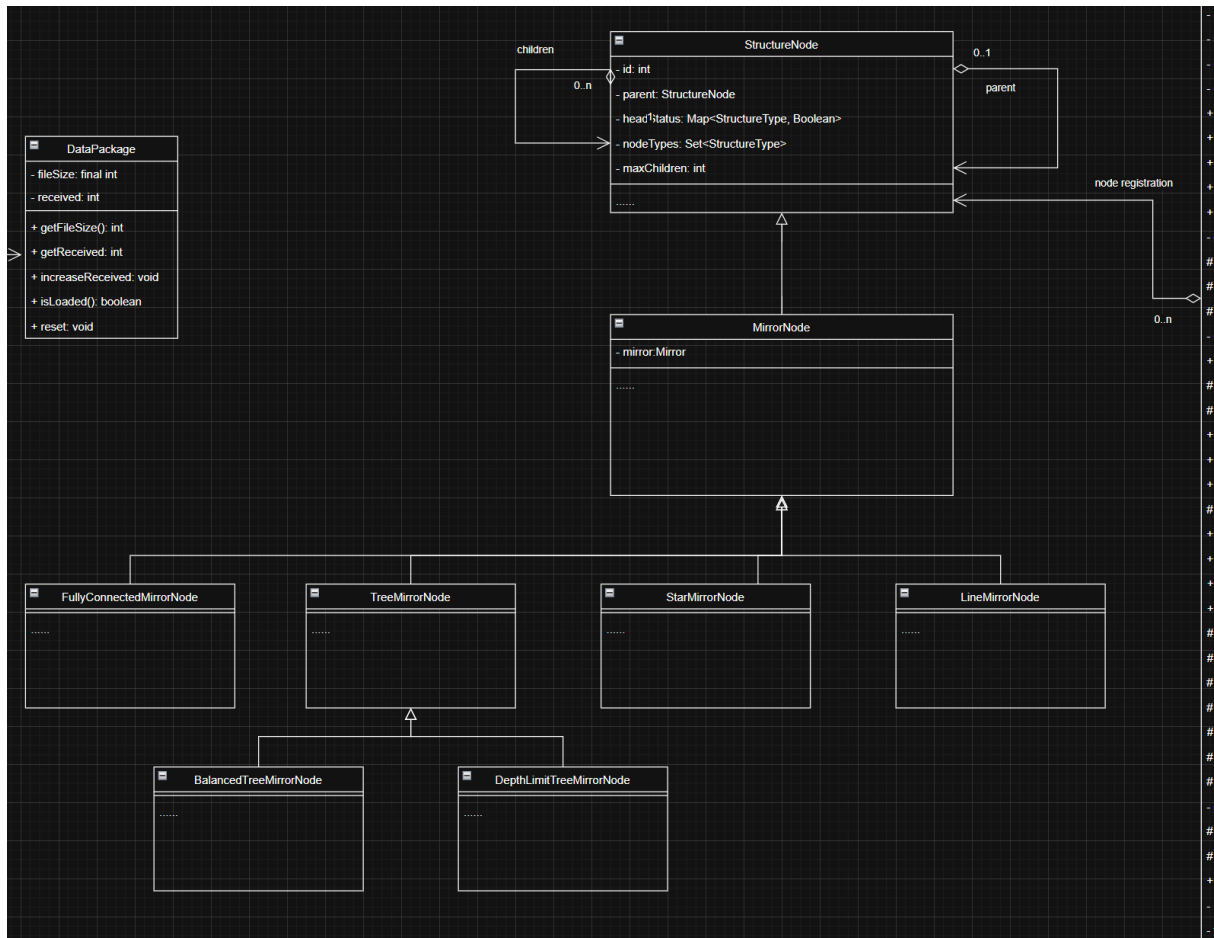


Figure A.16: Dependency tree of StructureNode types, see subsection 4.2.1.

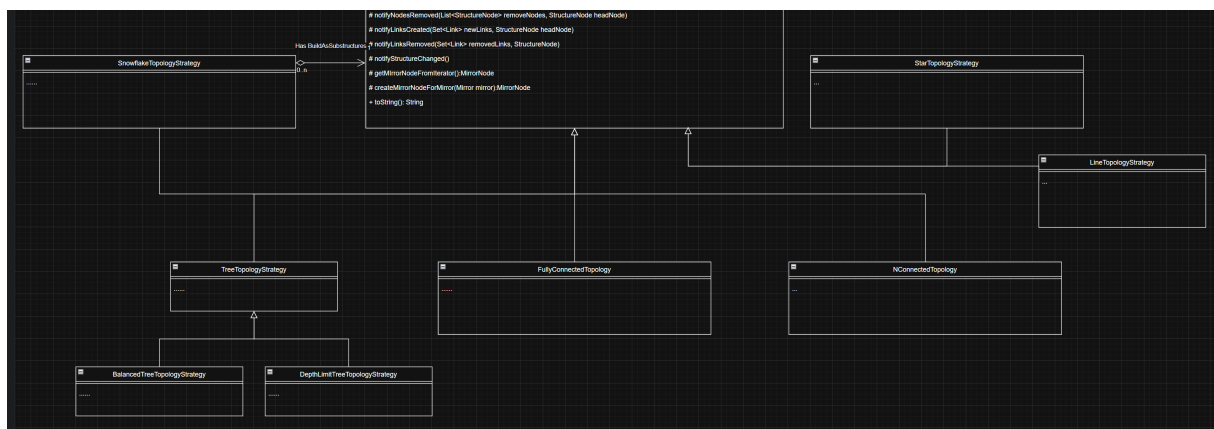


Figure A.17: Dependency tree of new TopologyStrategy derived types, see subsection 4.1.17.

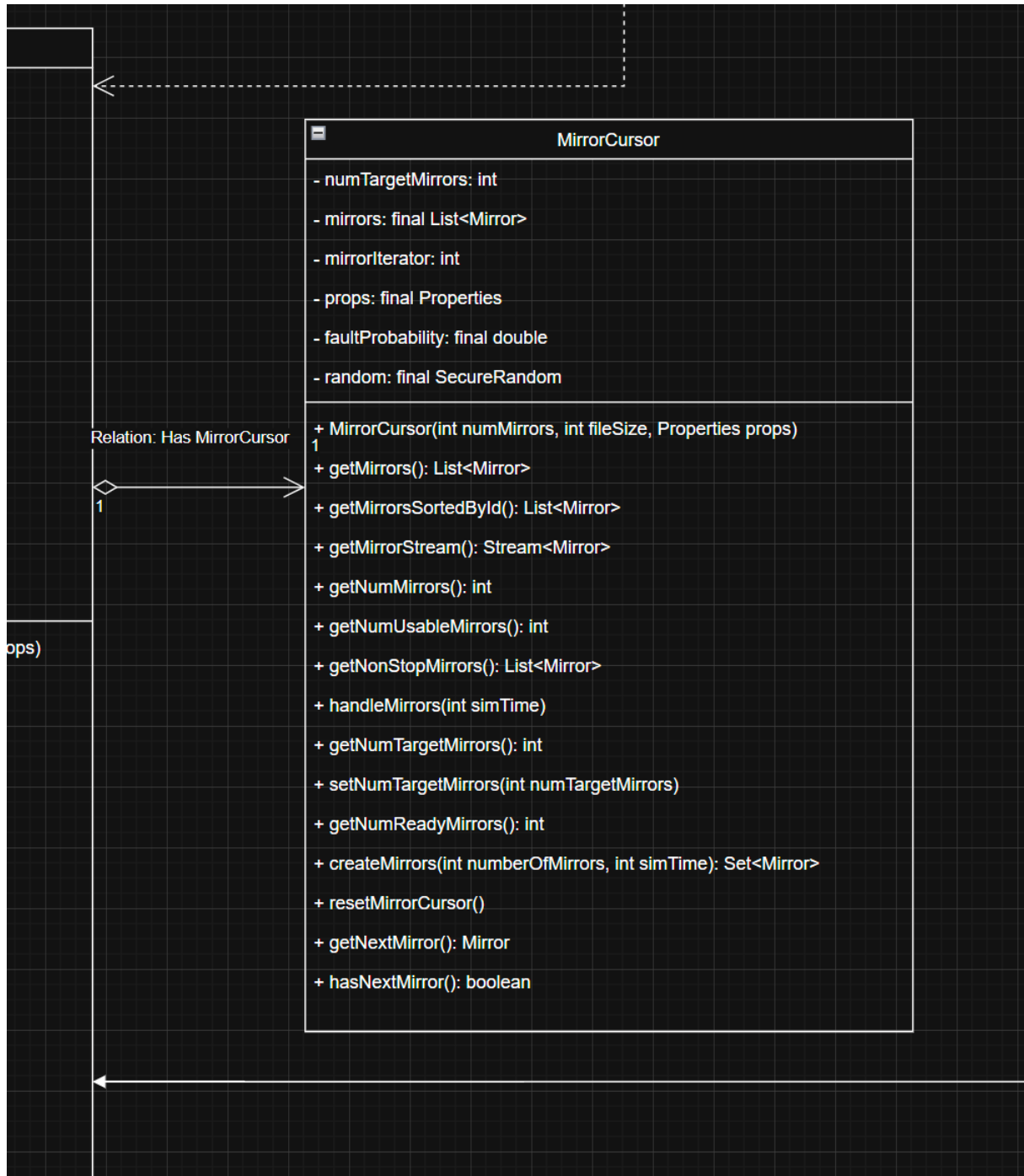


Figure A.18: MirrorCursor close up, see subsection 4.1.5.

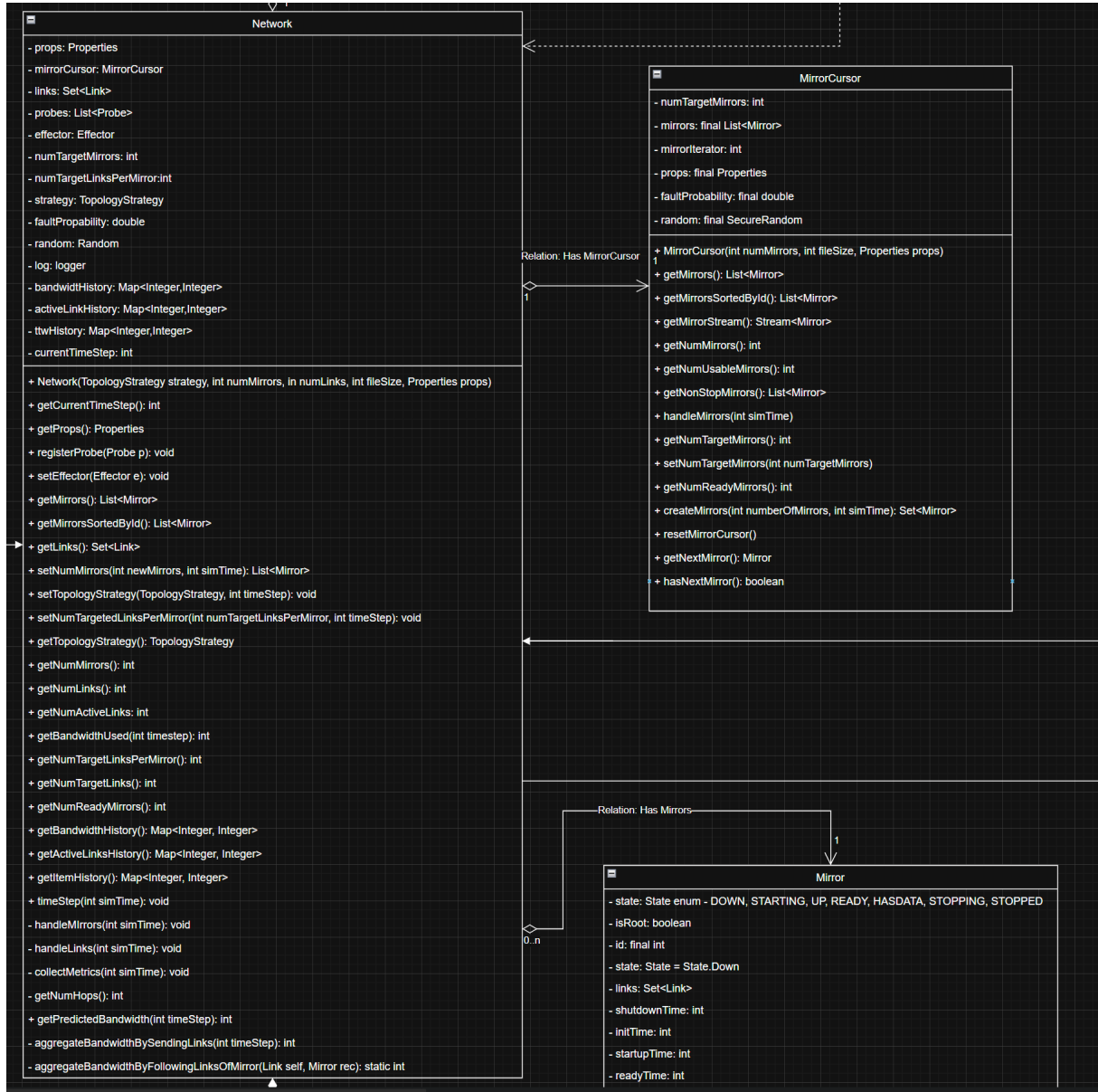


Figure A.19: MirrorCursor overview, see subsection 4.1.5.

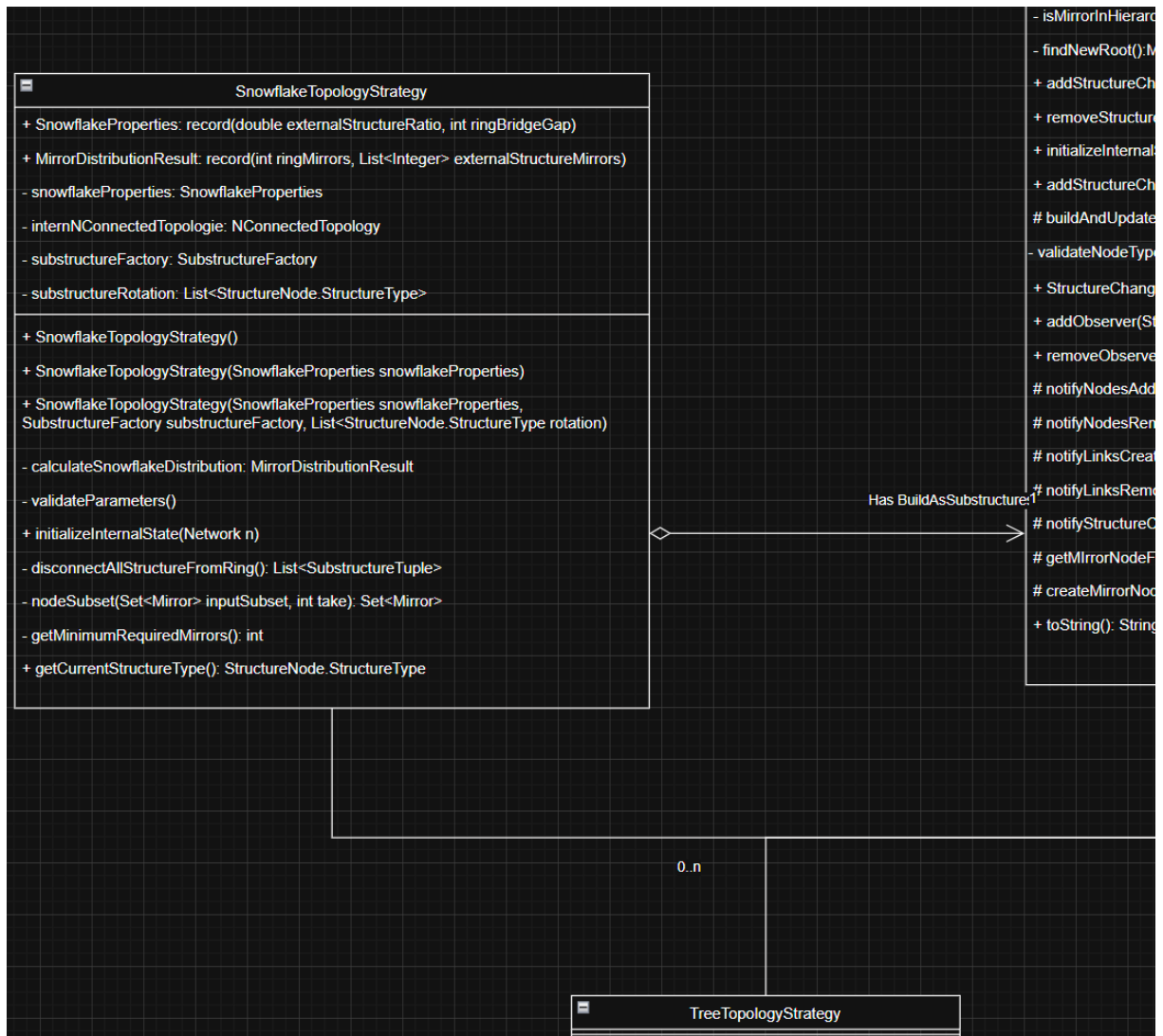


Figure A.20: SnowflakeTopologyStrategy, see subsection 4.2.14 and section 3.3.

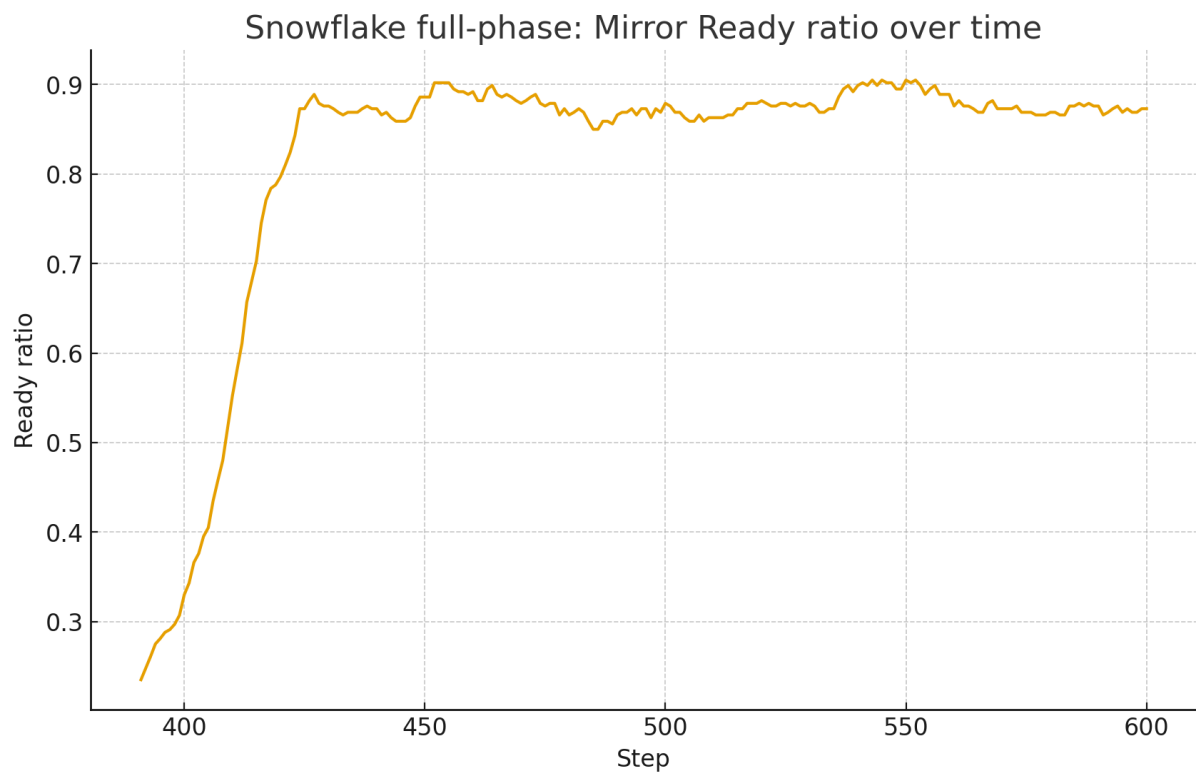


Figure A.21: SnowflakeTopologyStrategy experiment run on mirror ready ratio over 600 ticks.

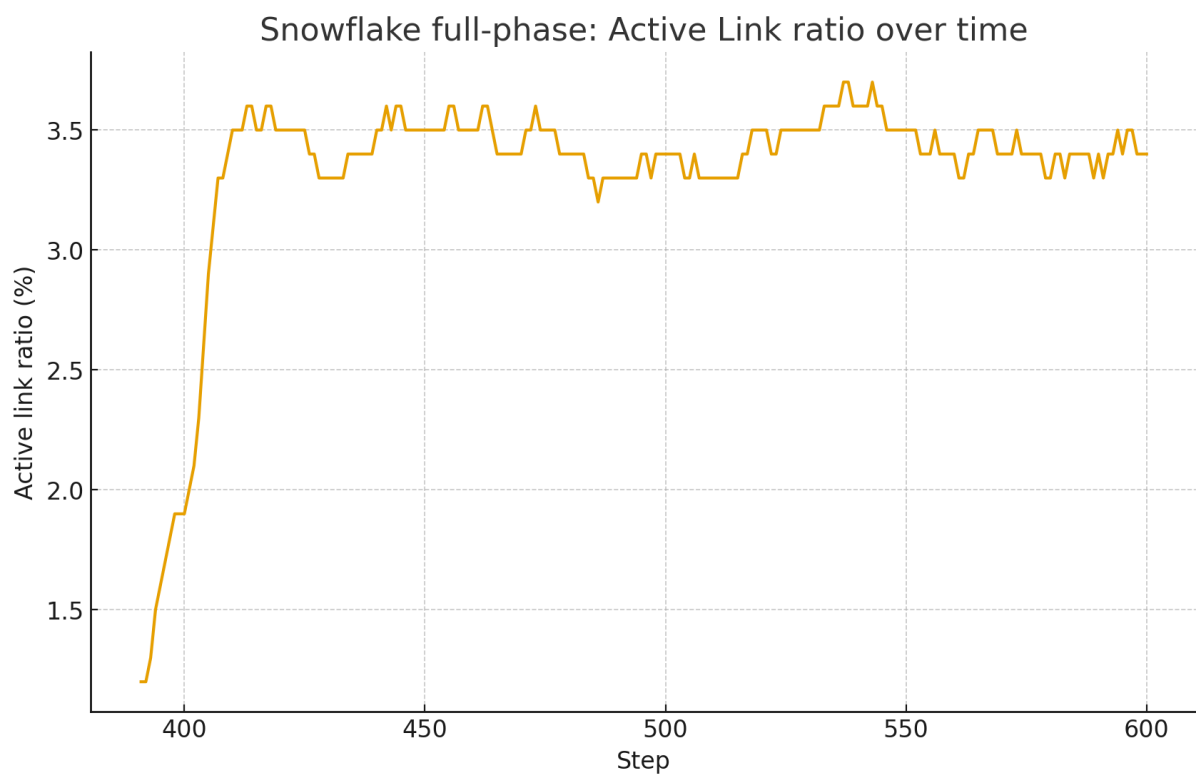


Figure A.22: SnowflakeTopologyStrategy experiment run on link active ratio over 600 ticks.

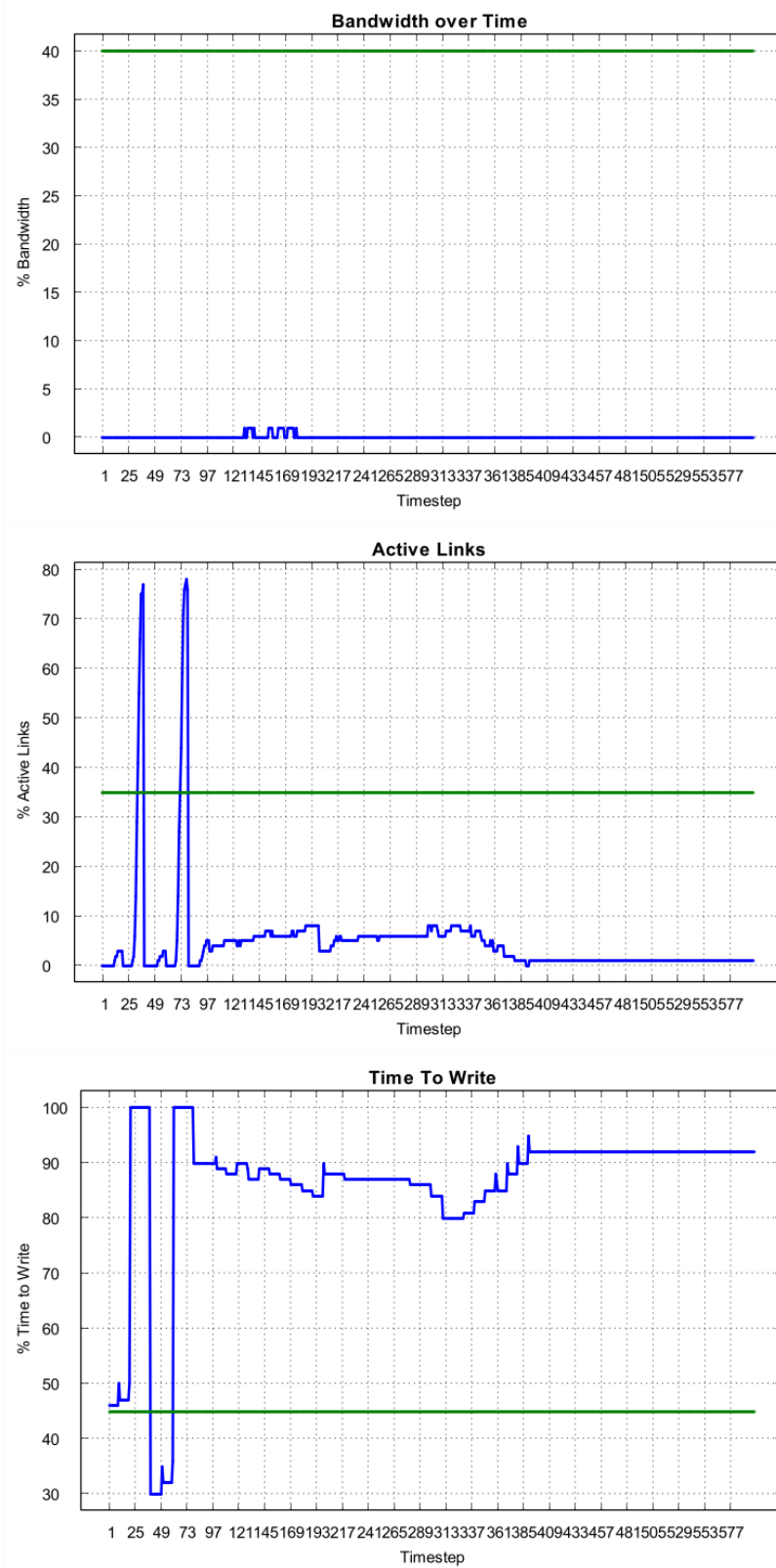


Figure A.23: SnowflakeTopologyStrategy experiment bandwidth, active links, time to write.



B Preset Schema and Scenario Manifest

B.1 Purpose

This appendix specifies a concise, machine-readable preset format for LRDMSim. Presets define network topology, timing, failure and reconfiguration events, probes, and outputs, so that experiments are reproducible and comparable.

B.2 Top-level Structure (Informal JSON Schema)

```

{
  "meta": {
    "id": "string", // unique scenario id
    "label": "string", // human-readable name
5    "seed": 123456, // RNG seed for reproducibility
    "tickMs": 250 // simulation tick in milliseconds
  },
  "topology": {
    "type": "NConnectedRing", // backbone type
10    "nodeCount": 48, // total mirrors on the ring
    "nConnectivity": 3, // N (degree of ring connectivity)
    "attachment": {
      "period": 3, // attach a substructure every k ring nodes
      "offset": 0, // starting offset (mod nodeCount)
15    "factory": [
      { "kind": "EdgeStar", "size": 4, "latencyMs": 2, "capacityMbps": 1000 },
      { "kind": "MiniChain", "size": 3, "latencyMs": 10, "capacityMbps": 500 }
    ],
    "selection": "roundRobin" // how factories are chosen along the ring
20  },
  "domains": {
    "bridges": [
      {
25    "from": "ring:0", // endpoint notation: ring:i or dom:<name>:j
      "to": "edge:siteB:1",
      "latencyMs": 45,
      "capacityMbps": 200,
      "jitterMs": 5,
30    "loss": 0.001
      }
    ],
    "labels": {
      "edge:siteB": { "role": "edge", "region": "WAN" }
35    }
  },
  "behavior": {
    "workload": {
      "writesPerTick": 2,
40    "payloadKB": 64,
      "pattern": "poisson",
      "targets": "all" // or subset/tag
    },
    "consistency": {
45    "mode": "mirror", // mirror semantics as in core LRDM
      "timeoutTicks": 20
    }
  },
  "events": {
50    "crashes": [
      { "atTick": 600, "target": "ring:7", "durationTicks": 120 }
    ],

```

```

    "linkDown": [
55  { "atTick": 1200, "from": "ring:10", "to": "ring:13", "durationTicks": 80 }
    ],
    "reconfigure": [
      { "atTick": 300, "action": "addMirror",
        "args": { "attachTo": "ring:15" } },
      { "atTick": 900, "action": "removeMirror",
60  "args": { "id": "ring:22" } },
      { "atTick": 1500, "action": "addLink",
        "args": { "from": "ring:2", "to": "ring:20",
          "latencyMs": 3, "capacityMbps": 1000 } }
    ]
65  },
    "probes": {
      "activeLinks": true,
      "timeToWrite": true,
      "bandwidthUtil": true,
70  "reconfigLatency": true, // ticks from action to quiescent state
      "staleness": false // optional: max write lag in ticks
    },
    "output": {
      "dir": "out/experiments/snowflake-demo",
75  "charts": ["bandwidth", "activeLinks", "timeToWrite"],
      "csv": ["metrics", "events", "topology"],
      "snapshotEveryTicks": 500
    }
  }
}

```

B.3 Endpoint and Naming Conventions

- `ring: i` addresses the i -th mirror on the N -connected ring (0-based).
- `dom:name: j` or shorthand `edge:name: j` addresses node j in an attached domain labeled *name*.
- Attachment positions advance along the ring in steps of `period`, starting at `offset`; factories are applied per selection.

B.4 Minimal Working Example

```

{
  "meta": { "id": "A01", "label": "Snowflake-3 on 36", "seed": 42, "tickMs": 250 },
  "topology": {
    "type": "NConnectedRing", "nodeCount": 36, "nConnectivity": 3,
5  "attachment": {
      "period": 3, "offset": 0,
      "factory": [{ "kind": "EdgeStar", "size": 3, "latencyMs": 2, "capacityMbps": 1000 } ],
      "selection": "roundRobin"
    }
10  },
  "events": {
    "crashes": [{ "atTick": 480, "target": "ring:5", "durationTicks": 100 } ],
    "reconfigure": [{ "atTick": 720, "action": "addMirror",
15  "args": { "attachTo": "ring:12" } } ],
    "probes": { "activeLinks": true, "timeToWrite": true, "bandwidthUtil": true,
      "reconfigLatency": true },
    "output": { "dir": "out/A01", "charts":
20  ["bandwidth", "activeLinks", "timeToWrite"], "csv": ["metrics"] }
  }
}

```

B.5 Indicator Notes

- **Active links (%)**: share of links in *UP* state per tick; report mean, min, and outage spans.

- **Time to write (ms):** end-to-end latency from write issuance to durable replication, aggregated by percentiles.
- **Bandwidth utilization (%):** utilized/available capacity per link; report 95th percentile and sustained plateaus.
- **Reconfiguration latency (ticks):** ticks between a `reconfigure` event and return to steady metrics (no backlogs).

C Reference Scenarios & Batch Runner outlook

C.1 Purpose

This appendix provides (i) a small catalog of future implementation pseudo code scenarios using the preset schema from Appendix A and (ii) a *batch runner* interface to execute many presets (or parameter sweeps) reproducibly. Results are exported as CSV and charts (bandwidth, active links, time to write), aligned with the existing metrics (`collectMetrics(t)`) and the visualization layer).

C.2 Runner Design Goals

- **Reproducibility:** seeded runs, fixed `tickMs` (default 250 ms), frozen preset manifest.
- **Throughput:** headless mode (no live GUI), CSV first, charts optional.
- **Comparability:** identical warmup, runtime, and sampling cadence across presets.
- **Traceability:** each run writes its resolved topology, event log, and run metadata.

C.3 Command-Line Interface (CLI)

```

# Single preset file
java -jar lrdmsim.jar run \
  --preset presets/A01_snowflake3_36.json \
  --warmup 200 --runtime 3000 --sample-every 1 \
5  --out out/A01 --charts bandwidth,activeLinks,timeToWrite --csv metrics,events

# Directory of presets
java -jar lrdmsim.jar batch \
  --presets-dir presets/ --glob "*.json" \
10  --warmup 200 --runtime 3000 --sample-every 1 \
  --out out/batch-2025-09 --charts none --csv metrics,events,topology

```

Arguments. `--warmup` (ticks) is excluded from metrics; `--runtime` counts sampled ticks; `--sample-every` subsamples ticks (1 = every tick). The runner honors per-preset `meta.seed` and `meta.tickMs`; CLI can override `tickMs` with `--tick-ms` if needed.

C.4 Execution Model

Each run performs: (1) build topology from `topology/domains`; (2) initialize probes; (3) apply warmup ticks without recording; (4) apply runtime ticks, executing events from `crashes`, `linkDown`, and `reconfigure` at their `atTick`; (5) export metrics and artifacts.

C.5 Reference Scenario Catalog

R1: Baseline N -Connected Ring

Purpose: baseline convergence and failover on a plain N -connected ring.

```
{ "meta": { "id": "R1", "label": "Ring-N=2-48", "seed": 1, "tickMs": 250 },
  "topology": { "type": "NConnectedRing", "nodeCount": 48, "nConnectivity": 2,
    "attachment": { "period": 0 } }, // no attachments
  "events": { "crashes": [{ "atTick": 800, "target": "ring:11",
    "durationTicks": 150 } ] },
  "probes": { "activeLinks": true, "timeToWrite": true, "bandwidthUtil": true,
    "reconfigLatency": true },
  "output": { "dir": "out/R1", "charts": ["bandwidth", "activeLinks", "timeToWrite"],
    "csv": ["metrics", "events"] }
}
```

R2: Snowflake-3 on 36 (Edge Stars)

Purpose: snowflake composite with periodic edge domains.

```
{ "meta": { "id": "R2", "label": "Snowflake3-EdgeStars", "seed": 42, "tickMs": 250 },
  "topology": { "type": "NConnectedRing", "nodeCount": 36, "nConnectivity": 3,
    "attachment": { "period": 3, "offset": 0,
    "factory": [{ "kind": "EdgeStar", "size": 3, "latencyMs": 2, "capacityMbps": 1000 } ],
    "selection": "roundRobin" } },
  "events": { "reconfigure": [{ "atTick": 700, "action": "addMirror",
    "args": { "attachTo": "ring:12" } } ] },
  "probes": { "activeLinks": true, "timeToWrite": true, "bandwidthUtil": true,
    "reconfigLatency": true },
  "output": { "dir": "out/R2", "charts": ["bandwidth", "activeLinks", "timeToWrite"],
    "csv": ["metrics", "events"] }
}
```

R3: WAN Bridge (Domain-to-Domain)

Purpose: evaluate Internet-scale links with latency/capacity constraints.

```
{ "meta": { "id": "R3", "label": "WAN-Bridge", "seed": 7, "tickMs": 250 },
  "topology": { "type": "NConnectedRing", "nodeCount": 24, "nConnectivity": 3,
    "attachment": { "period": 2, "factory": [{ "kind": "MiniChain", "size": 3,
    "latencyMs": 3, "capacityMbps": 800 } ], "selection": "roundRobin" } },
  "domains": { "bridges": [{ "from": "ring:0", "to": "edge:siteB:1", "latencyMs": 50,
    "capacityMbps": 200, "jitterMs": 5, "loss": 0.001 } ] },
  "events": { "linkDown": [{ "atTick": 1200, "from": "ring:4", "to": "ring:7",
    "durationTicks": 100 } ] },
  "probes": { "activeLinks": true, "timeToWrite": true, "bandwidthUtil": true,
    "reconfigLatency": true },
  "output": { "dir": "out/R3", "charts": ["bandwidth", "activeLinks", "timeToWrite"],
    "csv": ["metrics", "events", "topology"] }
}
```

R4: Stress—Burst Reconfigurations

Purpose: robustness of timed effectors; measure reconfiguration latency.

```
{ "meta": { "id": "R4", "label": "Burst-Reconfigs", "seed": 99, "tickMs": 250 },
  "topology": { "type": "NConnectedRing", "nodeCount": 32, "nConnectivity": 2,
    "attachment": { "period": 4, "factory": [{ "kind": "EdgeStar", "size": 4,
    "latencyMs": 2, "capacityMbps": 1000 } ] } },
  "events": { "reconfigure": [
    { "atTick": 300, "action": "addLink", "args": { "from": "ring:2", "to": "ring:18",
    "latencyMs": 3, "capacityMbps": 1000 } },
    { "atTick": 400, "action": "removeMirror", "args": { "id": "ring:9" } },
    { "atTick": 450, "action": "addMirror", "args": { "attachTo": "ring:9" } }
  ] },
  "probes": { "activeLinks": true, "timeToWrite": true, "bandwidthUtil": true,
    "reconfigLatency": true },
  "output": { "dir": "out/R4", "charts": ["activeLinks"],
    "csv": ["metrics", "events"] }
}
```

R5: Sensitivity—Attachment Period Sweep

Purpose: effect of attachment density on convergence and bandwidth.

```
# Batch parameter sweep (YAML-like for brevity; expanded to JSON by a
# small pre-processor)
batch:
  base: presets/R2_base.json
  grid:
    attachment.period: [2, 3, 4, 6]
    topology.nConnectivity: [2, 3]
  out: out/R5-period-n-sweep
  warmup: 200
  runtime: 3000
  sampleEvery: 1
  charts: none
  csv: [metrics, topology]
```

C.6 Output Layout

```
out/
  R2/
    run.json # resolved manifest (after CLI overrides)
    topology.json # nodes/links realized at t=0
    events.csv # atTick, type, args, status
    metrics.csv # tick, activeLinksPct, timeToWriteMs_p50/p95,
    # bwUtilPct_p95, ...
    charts/
      bandwidth.png
      activeLinks.png
      timeToWrite.png
```

C.7 Reproducibility & Recommended Defaults

- **Warmup:** 200 ticks; **Runtime:** 3000 ticks; **Tick:** 250 ms.
- **Sampling:** every tick for small scenarios; `sample-every=5` for large sweeps.
- **Seeds:** fix `meta.seed` per preset; vary only in designated sweeps.
- **Versioning:** commit presets and runner version into the run folder (`run.json`).

C.8 Batch Runner—Integration Sketch

(Pseudocode; maps to `timeStep(t)`, `handleDataTransfer`, failures via `crash`, metrics via `collectMetrics(t)`.)

```
loadPreset(p)
net = buildNetworkFromPreset(p.topology, p.domains, seed=p.meta.seed)
probes = initProbes(p.probes)
schedule = indexEventsByTick(p.events)
5
for t in 0 .. warmup-1:
  applyEvents(schedule[t], net) // reconfigure, linkDown, crashes
  net.timeStep(t) // includes handleDataTransfer()
# sampling starts here
10 for t in warmup .. warmup+runtime-1:
  applyEvents(schedule[t], net)
  net.timeStep(t)
  metrics = net.collectMetrics(t)
  probes.record(metrics)
15 exportCSVChartsProbes(outDir, probes, net, p)
```

C.9 Indicator Interpretation

- **Active links (%)**: sustained drops indicate partitioning or prolonged recovery.
- **Time to write (ms)**: percentile tails (p_{95}/p_{99}) reveal backpressure under reconfiguration.
- **Bandwidth utilization (%)**: WAN bridges typically bound throughput; plateaus reveal bottlenecks.
- **Reconfiguration latency (ticks)**: ticks from event to steady-state (no backlog growth); compare across strategies.

D Configuration Listings

D.1 Baseline (R1) Configuration Future Example

```
1 {
2   "topology": {
3     "backbone": { "type": "nConnectedRing", "N": 2, "size": 32 },
4     "attachments": { "period": 3, "factory": "SubstructurePermutationFactory",
5       "variants": ["EDGE_STAR", "SMALL_RING", "WAN_STUB"] }
6   },
7   "links": { "latencyMs": 1, "capacityMbps": 1000 },
8   "workload": { "ratePerSec": 10, "payloadBytes": 4096, "durationSec": 120 },
9   "seed": 42, "deltaTms": 1
10 }
```

D.2 WAN Bridge (R2) Configuration Future Example

```
1 {
2   "domains": [
3     { "name": "A", "backbone": { "type": "nConnectedRing", "N": 3, "size": 32 } },
4     { "name": "B", "backbone": { "type": "nConnectedRing", "N": 2, "size": 16 } }
5   ],
6   "bridges": [
7     { "from": "A.m12", "to": "B.m3", "latencyMs": 80, "capacityMbps": 100 }
8   ],
9   "workload": { "ratePerSec": 6, "payloadBytes": 8192, "durationSec": 180 },
10   "seed": 1337, "deltaTms": 1
11 }
```

E Future CLI Commands

E.1 Build

```
mvn -q -DskipTests package
# or:
./gradlew -q jar
```

E.2 Run Scenarios R1–R5

```
# R1 Static baseline
java -jar target/lrdmsim.jar --config configs/R1_static.json --seed 11 --export out/R1

# R2 WAN bridge
java -jar target/lrdmsim.jar --config configs/R2_wan.json --seed 11 --export out/R2

# R3 Burst stress
java -jar target/lrdmsim.jar --config configs/R3_burst.json --seed 11 --export out/R3

# R4 Live reconfiguration (adds/removes mirrors/links at T=60s)
java -jar target/lrdmsim.jar --config configs/R4_live.json --seed 11 --export out/R4

# R5 Failure/repair (drop backbone links at T=45s, restore at T=75s)
java -jar target/lrdmsim.jar --config configs/R5_fail.json --seed 11 --export out/R5
```

E.3 Post-processing (example)

```
python3 tools/plot_metrics.py --input out/R2 --metric e2e_delay --output figs/R2_delay.pdf
```

F Pseudocode for future Core Routines

F.1 Two-Phase Tick (Effects then Measurements)

```

1  for tick in 0..T:
2      # Phase 1: apply scheduled effects and exogenous events
3      for effector in effectors where effector.time == tick:
4          effector.apply(G) # add/remove mirrors/links, change capacities, failures
5
6      # Phase 2: per-link transmission and per-mirror state machine
7      for link in G.links:
8          link.enqueue_offered_traffic()
9          link.advance_queue(deltaT) # honor capacity and latency
10
11     for mirror in G.mirrors:
12         mirror.step_fsm(deltaT) # intake, propagation, ack
13
14     # Metrics and invariants
15     metrics.collect(G, tick)
16     assert invariants.hold(G) # no lost/duplicated updates after quiescence

```

F.2 Substructure-Permutation Attachment (Period- k)

```

1  def attach_substructures(backbone, k, variants):
2      # variants: list of factory functions or suppliers
3      for m in backbone.mirrors: # mirrors have stable IDs
4          if m.id % k == 0:
5              v = variants[(m.id / k) % len(variants)]
6              S = v() # build domain subgraph instance
7              connect(S.entry, m) # add link with constraints

```

Declaration of Authorship

I hereby declare that I have authored this thesis independently and have used only the sources and resources indicated. All passages which are taken verbatim or in spirit from published or unpublished texts are clearly marked as such by citations. This thesis, in whole or in substantial parts, has not been submitted to any other examination board and has not been published previously.

I acknowledge that the digital version submitted is identical to any printed version (if applicable). I consent to plagiarism checks in accordance with the regulations of Technische Universität Dresden. I am aware that any violation of these rules may lead to academic and legal consequences.

Dresden, _____
(Place and date)

Signature

Benjamin-Elias Probst

